

Python for AI

A Practical Feature Guide, Every Example Runnable

The Python language features that matter most for machine learning and AI engineering, each with a short, self-contained example whose output was captured by actually running the code.

Built with Claude Code

How to read this book

This guide is organised around Python *language features* rather than around any single AI library, because the features are what transfer across PyTorch, TensorFlow, JAX, scikit-learn, Hugging Face and the LLM API clients you will actually use. For every feature you get a one- or two-line explanation of what it is, why it shows up constantly in AI code, and a small program you can run.

Every code block marked with an OUTPUT panel was executed while this PDF was being generated. The green panel is the real captured output of the grey code above it, so if it prints here, it runs on your machine too (Python 3.10+ recommended; the numeric chapters use NumPy and pandas).

The examples are deliberately tiny and dependency-light. Where a line shows how a real framework uses a feature (for example a PyTorch `nn.Module` or an async LLM call), it is clearly marked as illustrative and is not executed.

Tip for Kindle: use the built-in table of contents (the ≡ navigation menu) to jump between chapters. Every chapter and part is a bookmark. Rotating to landscape enlarges the code panels.

Contents

Part I: Python Core for AI

1. Numbers, Booleans and the Object Model	9
2. Strings and f-strings for Text Data	13
3. Lists, Tuples, Sets and Dicts	18
4. The collections Module: Counter, defaultdict, deque	23
5. Comprehensions and Generator Expressions	26
6. Control Flow, Truthiness, Unpacking and the Walrus	30
7. Structural Pattern Matching (match / case)	35

Part II: Functions and Functional Programming

8. Functions: Arguments, Defaults and the Mutable-Default...	40
9. Lambdas, map, filter and sorted(key=)	47
10. functools: partial, lru_cache, reduce, wraps	52
11. itertools: Batching and Combining Iterables	60

Part III: Iteration, Generators and Data Pipelines

12. Iterables, Iterators and the Iterator Protocol	69
13. Generators and yield: Streaming Datasets and Mini-Ba...	74
14. Generator Expressions, yield from and Lazy Pipelines	81
15. Building a Streaming Data Pipeline End to End	86

Part IV: Objects and Abstractions

16. Classes, Dunder Methods and __call__: the nn.Module...	91
17. Inheritance, super(), Abstract Base Classes and Prope...	99
18. Dataclasses for Configs and Records	104
19. Enums and NamedTuples	109
20. Decorators: Timing, Caching and no_grad-style Wrap...	113
21. Context Managers and the with Statement	118

Part V: Typing and Robustness

22. Type Hints and the typing Module	124
23. Protocols and Structural Typing (Duck Typing, Check...	130
24. TypedDict and Typed Configs / Messages	133

25. Errors, Exceptions and Custom Exception Types	136
26. Logging Instead of print for Training Runs	142
Part VI: Data, Files and Serialization	
27. File I/O and pathlib	146
28. JSON and CSV: Configs and Datasets	151
29. pickle and Model / Artifact Serialization	156
Part VII: Concurrency and Performance	
30. Threads, Processes and concurrent.futures for Data... ..	162
31. async / await for Concurrent LLM and API Calls	168
32. Choosing Concurrency: I/O-bound vs CPU-bound	175
Part VIII: The Numeric AI Stack	
33. NumPy Arrays, dtypes and Vectorization	178
34. Broadcasting, Indexing and Linear Algebra: the Math... ..	183
35. Random, Statistics and Activation Functions from Scr... ..	189
36. pandas Series and DataFrame: Loading Datasets	194
37. pandas Cleaning, Grouping and Feature Engineering	200
38. Visualization and the Deep-Learning Frameworks	205
Part IX: Capstone: A Tiny Transformer LLM	
39. Building and Training a Tiny Transformer LLM	212
40. Running Inference with the Trained Transformer	220

PART I

Python Core for AI

Numbers, Booleans and the Object Model

Every value in Python is an **object** with a type, and even integers have unbounded precision. In AI you constantly move between Python's `int/float` and the fixed-width numeric types used by tensors (`float32`, `int64`), so it helps to know what the pure-Python side actually guarantees.

Integers never overflow

Unlike C or the fixed-width dtypes inside NumPy, a Python `int` grows as large as memory allows. This is why counting tokens or computing large factorials in plain Python never silently wraps around.

ARBITRARY PRECISION INTEGERS

```
n = 2 ** 200
print(n)
print('bit length:', n.bit_length())
# floor division and modulo, useful for batching
samples, batch = 1003, 32
print('full batches:', samples // batch, 'remainder:', samples % batch)
```

OUTPUT

```
1606938044258990275541962092341162602522202993782792835301376
bit length: 201
full batches: 31 remainder: 11
```

Floats, and why comparisons need tolerance

Floating-point math is inexact. Loss values and probabilities almost never compare exactly, so you test *closeness* with `math.isclose` (or `numpy.allclose` for arrays) instead of `==`.

NEVER COMPARE FLOATS WITH `==`

```
import math
a = 0.1 + 0.2
print('a =', a)
print('a == 0.3 ?', a == 0.3)
print('isclose ?', math.isclose(a, 0.3, rel_tol=1e-9))
```

OUTPUT

```
a = 0.30000000000000004
a == 0.3 ? False
isclose ? True
```

Rounding, divmod and conversions

Python's `round` uses **banker's rounding** (round-half-to-even), which reduces bias when you round many numbers, exactly what you want for statistics. `divmod` returns the quotient and remainder together, handy for turning a flat step count into epochs and offsets. Conversions between `int` and `float` truncate toward zero, so use `math.floor/ceil` when you need a specific direction.

BANKER'S ROUNDING AND DIVMOD

```
print('round(0.5) =', round(0.5))    # -> 0, ties go to even
print('round(1.5) =', round(1.5))    # -> 2, ties go to even
print('round(2.675, 2) =', round(2.675, 2)) # float artifact
epochs, offset = divmod(1003, 32)
print('divmod(1003, 32) =', (epochs, offset))
print('int(3.9) =', int(3.9), '| float(7) =', float(7))
```

OUTPUT

```
round(0.5) = 0
round(1.5) = 2
round(2.675, 2) = 2.67
divmod(1003, 32) = (31, 11)
int(3.9) = 3 | float(7) = 7.0
```

Infinity, NaN, and complex numbers

`math.inf` is a useful sentinel for a starting best-loss. `math.nan` is

special: it is **not equal to itself**, which is how you detect a training run whose loss has diverged to NaN. Python also has native **complex** numbers (written `2 + 3j`), which appear in signal processing and Fourier transforms.

INF, NAN AND COMPLEX

```
import math
best = math.inf                # nothing beats infinity
print('0.9 < inf ?', 0.9 < best)
nan = math.nan
print('nan == nan ?', nan == nan)  # always False!
print('isnan      ?', math.isnan(nan))
z = 2 + 3j
print('complex:', z, '| magnitude:', abs(z))
```

OUTPUT

```
0.9 < inf ? True
nan == nan ? False
isnan      ? True
complex: (2+3j) | magnitude: 3.605551275463989
```

Booleans are integers

True and **False** are **1** and **0**. Summing a list of boolean conditions is the idiomatic way to *count* how many are true, which is exactly how you compute accuracy from a list of correct/incorrect predictions.

COUNTING CORRECT PREDICTIONS

```
preds = [1, 0, 1, 1, 0, 1]
labels = [1, 0, 0, 1, 0, 1]
correct = sum(p == y for p, y in zip(preds, labels))
accuracy = correct / len(labels)
print(f'{correct} correct out of {len(labels)}')
print(f'accuracy = {accuracy:.2%}')
```

OUTPUT

```
5 correct out of 6
accuracy = 83.33%
```

Why it matters for AI: metrics like accuracy, precision and recall are all just sums of boolean conditions divided by a count. Master `sum(cond for ...)` and you can write most metrics from scratch without a library.

CHAPTER 2

Strings and f-strings for Text Data

Text is the raw material of NLP and of every LLM prompt. Python strings are immutable Unicode sequences, and f-strings are the fastest, most readable way to build prompts and format numbers in logs.

f-strings: interpolation and formatting

BUILDING A PROMPT AND FORMATTING METRICS

```
role, topic = 'assistant', 'linear algebra'
prompt = f'You are a helpful {role}. Explain {topic} simply.'
print(prompt)

loss, lr = 0.03718, 3e-4
# format spec: fixed decimals, scientific, padding
print(f'loss={loss:.4f} lr={lr:.1e}')
for step in [1, 50, 1000]:
    print(f'step {step:>5} | {step/1000:6.1%} done')
```

OUTPUT

```
You are a helpful assistant. Explain linear algebra simply.
loss=0.0372 lr=3.0e-04
step      1 |   0.1% done
step     50 |   5.0% done
step    1000 | 100.0% done
```

The `=` specifier is a debugging superpower: `f'{x=}'` prints both the expression and its value.

SELF-DOCUMENTING DEBUG OUTPUT

```
temperature = 0.7
top_p = 0.95
print(f'{temperature=}, {top_p=}')
```

OUTPUT

```
temperature=0.7, top_p=0.95
```

Rich format specs: alignment, separators, conversions

The format spec after `:` is a mini-language. Use `,` or `_` for thousands separators (readable parameter counts), `</^/>` with a fill character for aligned tables, and the `!r/!s` conversions to force `repr()` or `str()`. A field width can itself be a variable via a nested `{}`.

SEPARATORS, FILL AND CONVERSIONS

```
params = 6_738_415_616
print(f'params: {params:,}')          # 6,738,415,616
print(f'params: {params/1e9:.2f}B')
width = 10
for name, val in [('loss', 0.31), ('acc', 0.88)]:
    print(f'name:<{width}|{val:>8.3f}') # nested width
label = 'cat'
print(f'raw={label!r} str={label!s}') # repr vs str
```

OUTPUT

```
params: 6,738,415,616
params: 6.74B
loss      |    0.310
acc       |    0.880
raw='cat' str=cat
```

str.format and format_map for prompt templates

When a template lives in a config file or database rather than in source code, you cannot use an f-string. `str.format` and `str.format_map` fill named placeholders from arguments or a dict, the classic way to store reusable prompt templates.

FILLING A STORED PROMPT TEMPLATE

```
template = 'You are a {role}. Answer about {topic}.'
print(template.format(role='tutor', topic='calculus'))
fields = {'role': 'critic', 'topic': 'poetry'}
print(template.format_map(fields)) # fill from a dict
```

OUTPUT

```
You are a tutor. Answer about calculus.
You are a critic. Answer about poetry.
```

Bytes and encoding: what tokenizers really see

Text on disk and on the wire is **bytes**, not characters. `str.encode` turns a string into UTF-8 bytes and `bytes.decode` reverses it. Modern byte-level tokenizers (BPE) operate on these bytes directly, which is why a single emoji can cost several tokens.

ENCODING TEXT TO UTF-8 BYTES

```
text = 'caf\u00e9 \U0001f600' # 'café' + accent + emoji
raw = text.encode('utf-8')
print('chars:', len(text), '| bytes:', len(raw))
print('utf-8:', raw)
print('round-trip:', raw.decode('utf-8') == text)
```

OUTPUT

```
chars: 6 | bytes: 10
utf-8: b'caf\xc3\xa9 \xf0\x9f\x98\x80'
round-trip: True
```

translate, removeprefix and removesuffix

`str.translate` deletes or remaps many characters in one pass (fast punctuation stripping). `removeprefix/removesuffix` (3.9+) safely trim known markers such as a `<s>` sentinel or a file extension, unlike `strip`, which removes *any* of the given characters.

STRIPPING PUNCTUATION AND MARKERS

```
import string
drop = str.maketrans('', '', string.punctuation)
print('Hello, world!!!'.translate(drop)) # no punctuation
tag = '<s>hello</s>'
print(tag.removeprefix('<s>').removesuffix('</s>'))
```

OUTPUT

```
Hello world
hello
```

textwrap: fitting prompts to a width

WRAPPING LONG PROMPT TEXT

```
import textwrap
note = ('Large language models predict the next token given '
       'all previous tokens in the context window.')
for line in textwrap.wrap(note, width=40):
    print(line)
print('---')
print(textwrap.shorten(note, width=45, placeholder=' ...'))
```

OUTPUT

```
Large language models predict the next
token given all previous tokens in the
context window.
---
Large language models predict the next ...
```

Common text-cleaning methods

NORMALISING TEXT BEFORE TOKENISING

```
raw = ' Hello, WORLD! \n'
clean = raw.strip().lower().replace(',', ' ')
print(repr(clean))
tokens = clean.split()
print('tokens:', tokens)
print('joined:', '|'.join(tokens))
print('starts with hi?', clean.startswith('hi'))
```

OUTPUT


```
'hello world!'
tokens: ['hello', 'world!']
joined: hello|world!
starts with hi? False
```

Why it matters for AI: prompt templates, chat formatting and cleaning scraped text are all string work. f-strings keep prompt-building readable, and `str.strip/split/join/replace` cover most preprocessing before a real tokenizer runs.

CHAPTER 3

Lists, Tuples, Sets and Dicts

Four built-in containers carry almost all data in Python before it becomes a tensor. Knowing which to reach for is half of writing clean data code: **lists** for ordered, mutable sequences (a batch of samples); **tuples** for fixed, immutable records (an array's [shape](#)); **sets** for fast membership and de-duplication (a vocabulary); **dicts** for keyed lookup (token to id, config values).

Lists: the workhorse sequence

BUILDING AND SLICING A LIST OF SAMPLES

```
data = [5, 2, 9, 1, 7, 3]
data.append(8)
print('list :', data)
print('first 3 :', data[:3])      # slice
print('last 2  :', data[-2:])
print('every 2nd:', data[::2])
print('reversed :', data[::-1])
```

OUTPUT

```
list : [5, 2, 9, 1, 7, 3, 8]
first 3 : [5, 2, 9]
last 2  : [3, 8]
every 2nd: [5, 9, 7, 8]
reversed : [8, 3, 7, 1, 9, 2, 5]
```

Tuples: immutable, hashable records

Tuples cannot change after creation, which makes them safe to use as dict keys and as the fixed shapes and coordinates you see all over NumPy and PyTorch.

TUPLES AS SHAPES AND DICT KEYS

```

shape = (32, 3, 224, 224) # batch, channels, H, W
batch, channels, h, w = shape # unpacking
print('channels =', channels, '| pixels =', h * w)
cache = {(0, 0): 'origin', (1, 2): 'point'} # tuple keys
print(cache[(1, 2)])

```

OUTPUT

```

channels = 3 | pixels = 50176
point

```

Sets: membership and de-duplication

BUILDING A VOCABULARY WITH A SET

```

words = ['cat', 'dog', 'cat', 'bird', 'dog', 'cat']
vocab = set(words)
print('unique   :', sorted(vocab))
print('size     :', len(vocab))
print("'cat' in?  :", 'cat' in vocab) # 0(1) lookup
a, b = {1, 2, 3}, {2, 3, 4}
print('overlap  :', a & b, '| union:', a | b)

```

OUTPUT

```

unique   : ['bird', 'cat', 'dog']
size     : 3
'cat' in? : True
overlap  : {2, 3} | union: {1, 2, 3, 4}

```

Dicts: keyed lookup and the config pattern

Dicts map keys to values with $O(1)$ lookup and preserve insertion order. They back token-to-id vocabularies, hyperparameter configs, and the **`**kwargs`** you pass into models.

A CONFIG DICT AND SAFE ACCESS

```

config = {'lr': 3e-4, 'epochs': 10, 'model': 'resnet'}
print('lr      : ', config['lr'])
print('dropout? : ', config.get('dropout', 0.0)) # default
config['batch_size'] = 64 # add a key
for key, value in config.items():
    print(f'  {key:12} = {value}')
stoi = {ch: i for i, ch in enumerate('abc')} # char -> id
print('token ids : ', stoi)

```

OUTPUT

```

lr      : 0.0003
dropout? : 0.0
  lr      = 0.0003
  epochs  = 10
  model   = resnet
  batch_size = 64
token ids : {'a': 0, 'b': 1, 'c': 2}

```

Dict tricks:.setdefault, merging, sorting by value

`setdefault` inserts a key with a default only if it is absent, the one-liner for grouping. The `|` operator merges two dicts into a new one and `|=` updates in place (3.9+), the modern way to layer a base config with overrides. To rank a frequency map, sort its `.items()` by value with a `key` function.

MERGE, GROUP AND RANK A DICT

```

base = {'lr': 1e-3, 'epochs': 10}
override = {'lr': 3e-4, 'batch': 64}
print('merged:', base | override) # override wins
groups = {}
for label, x in [('a', 1), ('b', 2), ('a', 3)]:
    groups.setdefault(label, []).append(x)
print('groups:', groups)
freq = {'the': 9, 'cat': 2, 'sat': 5}
ranked = sorted(freq.items(), key=lambda kv: kv[1], reverse=True)
print('ranked:', ranked)

```

OUTPUT

```
merged: {'lr': 0.0003, 'epochs': 10, 'batch': 64}
groups: {'a': [1, 3], 'b': [2]}
ranked: [('the', 9), ('sat', 5), ('cat', 2)]
```

sort vs sorted, and a tuple immutability caveat

`list.sort()` reorders in place and returns `None`; `sorted()` returns a new sorted list and works on any iterable. A subtle gotcha: a tuple is immutable, but if it *contains* a mutable object (a list), that inner object can still change, so such a tuple is no longer hashable and cannot be a dict/set key.

IN-PLACE SORT VS COPY; TUPLE CAVEAT

```
data = [5, 2, 9, 1]
print('sorted() copy :', sorted(data), '| original:', data)
data.sort(reverse=True)
print('after .sort() :', data)
t = (1, [2, 3])          # tuple holding a mutable list
t[1].append(4)           # the inner list still mutates
print('mutated tuple :', t)
try:
    hash(t)              # unhashable: contains a list
except TypeError as e:
    print('hash error  :', e)
```

OUTPUT

```
sorted() copy : [1, 2, 5, 9] | original: [5, 2, 9, 1]
after .sort() : [9, 5, 2, 1]
mutated tuple : (1, [2, 3, 4])
hash error  : unhashable type: 'list'
```

Set algebra and frozenset keys

Beyond union and intersection, `difference (-)` and `symmetric_difference (^)` answer 'what is only in A' and 'what differs between A and B', useful for comparing predicted vs true label sets. A `frozenset` is an immutable, hashable set, so it can serve as a dict key, for example caching a value per unordered

feature combination.

DIFFERENCE, SYMMETRIC DIFFERENCE, FROZENSET

```
pred = {'cat', 'dog', 'bird'}
true = {'cat', 'dog', 'fish'}
print('only predicted :', pred - true)
print('disagreements  :', pred ^ true)
cache = {frozenset({'a', 'b'}): 0.9}
print('order-free key :', cache[frozenset({'b', 'a'})])
```

OUTPUT

```
only predicted : {'bird'}
disagreements  : {'bird', 'fish'}
order-free key : 0.9
```

Why it matters for AI: a token-to-id **dict**, a **set** vocabulary, a **list** batch and a **tuple** shape appear in essentially every data-loading and tokenisation pipeline. Picking the right container makes that code both faster and clearer.

CHAPTER 4

The collections Module: Counter, defaultdict, deque

The standard-library `collections` module adds three specialised containers that show up constantly in AI and NLP code: `Counter` for frequency counting, `defaultdict` for grouping without key checks, and `deque` for fixed-size rolling buffers. They are pure Python, fast, and dependency-free.

Counter: token and word frequencies

`Counter` counts hashable items in one pass. `.most_common(k)` returns the top-k by frequency, the first step of building a vocabulary. Counters also support arithmetic, so you can add the word counts of two documents or subtract a stop-word profile.

COUNTING TOKENS WITH COUNTER

```
from collections import Counter
text = 'the cat sat on the mat the cat ran'
counts = Counter(text.split())
print('counts      : ', counts)
print('top 2       : ', counts.most_common(2))
print('the -> ', counts['the'], ' | missing -> ', counts['dog'])
doc_b = Counter('the dog ran'.split())
print('combined    : ', counts + doc_b)  # counter arithmetic
```

OUTPUT

```
counts      : Counter({'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'm
at': 1, 'ran': 1})
top 2       : [('the', 3), ('cat', 2)]
the -> 3 | missing -> 0
combined    : Counter({'the': 4, 'cat': 2, 'ran': 2, 'sat': 1, '
on': 1, 'mat': 1, 'dog': 1})
```

defaultdict: grouping and accumulating

A `defaultdict` supplies a default value the first time a key is touched, so you never write an `if key not in d` check. Use `defaultdict(list)` to bucket samples by label and `defaultdict(int)` to accumulate counts or running sums.

GROUPING SAMPLES BY LABEL

```
from collections import defaultdict
samples = [('spam', 'buy now'), ('ham', 'hi mom'),
           ('spam', 'free cash'), ('ham', 'call me')]
by_label = defaultdict(list)
for label, msg in samples:
    by_label[label].append(msg)  # no key check needed
for label, msgs in by_label.items():
    print(f'{label}: {msgs}')
totals = defaultdict(int)
for label, _ in samples:
    totals[label] += 1
print('class counts:', dict(totals))
```

OUTPUT

```
spam: ['buy now', 'free cash']
ham: ['hi mom', 'call me']
class counts: {'spam': 2, 'ham': 2}
```

deque: sliding windows and rolling buffers

A `deque` (double-ended queue) appends and pops from both ends in $O(1)$. With `maxlen` it becomes a fixed-size window: pushing a new item automatically drops the oldest. This is exactly a rolling loss buffer for smoothing, an n -gram window over tokens, or a replay buffer in reinforcement learning.

A ROLLING LOSS WINDOW WITH MAXLEN


```

from collections import deque
window = deque(maxlen=3)          # keep only last 3
stream = [0.9, 0.7, 0.6, 0.55, 0.5]
for loss in stream:
    window.append(loss)           # oldest drops off
    avg = sum(window) / len(window)
    print(f'loss={loss:.2f} window={list(window)} avg={avg:.3f} ←
        ')

```

OUTPUT

```

loss=0.90 window=[0.9] avg=0.900
loss=0.70 window=[0.9, 0.7] avg=0.800
loss=0.60 window=[0.9, 0.7, 0.6] avg=0.733
loss=0.55 window=[0.7, 0.6, 0.55] avg=0.617
loss=0.50 window=[0.6, 0.55, 0.5] avg=0.550

```

Why it matters for AI: **Counter** builds vocabularies and computes class balance, **defaultdict** groups data and tallies counts without boilerplate, and a **deque(maxlen=...)** is the canonical sliding window for smoothed metrics, n-grams and replay buffers. These three cover a huge share of everyday data-wrangling in ML pipelines.

CHAPTER 5

Comprehensions and Generator Expressions

Comprehensions are Python's signature feature for building a collection from an iterable in one readable line. In AI code they replace most manual `for` loops used to transform, filter and reshape data, and they are faster than the equivalent loop with `.append()`.

List comprehensions: map and filter in one line

TRANSFORM AND FILTER TOGETHER

```
nums = [1, 2, 3, 4, 5, 6]
squares = [x * x for x in nums]
evens    = [x for x in nums if x % 2 == 0]
labels   = ['pos' if x > 3 else 'neg' for x in nums]
print('squares:', squares)
print('evens   :', evens)
print('labels  :', labels)
```

OUTPUT

```
squares: [1, 4, 9, 16, 25, 36]
evens   : [2, 4, 6]
labels  : ['neg', 'neg', 'neg', 'pos', 'pos', 'pos']
```

Nested comprehensions: flattening batches

FLATTEN A LIST OF SEQUENCES

```

batches = [[1, 2], [3, 4], [5, 6]]
flat = [x for batch in batches for x in batch]
print('flat  :', flat)
# a 3x3 identity-like grid
grid = [[1 if i == j else 0 for j in range(3)] for i in range(3)
        ])
for row in grid:
    print(row)

```

OUTPUT

```

flat  : [1, 2, 3, 4, 5, 6]
[1, 0, 0]
[0, 1, 0]
[0, 0, 1]

```

Dict and set comprehensions

INVERT A VOCABULARY; COLLECT LENGTHS

```

stoi = {'a': 0, 'b': 1, 'c': 2}
itos = {i: ch for ch, i in stoi.items()} # invert mapping
print('id->char:', itos)
words = ['hi', 'bye', 'hi', 'ok']
lengths = {w: len(w) for w in set(words)} # dict comp
print('lengths :', lengths)

```

OUTPUT

```

id->char: {0: 'a', 1: 'b', 2: 'c'}
lengths : {'ok': 2, 'bye': 3, 'hi': 2}

```

Dict comp from zip, and the walrus in a comprehension

Pairing two lists with `zip` inside a dict comprehension is the idiomatic way to build a mapping from parallel columns. A walrus (`:=`) lets you compute an expensive value once and both filter on it and keep it, avoiding a double call.

ZIP INTO A DICT; WALRUS FILTER

```
names = ['loss', 'acc', 'f1']
vals = [0.31, 0.88, 0.79]
metrics = {k: v for k, v in zip(names, vals)}
print('metrics:', metrics)
raw = [' hi ', '', ' ok', ' ']
# strip once, keep only non-empty results
kept = [s for x in raw if (s := x.strip())]
print('kept   :', kept)
```

OUTPUT

```
metrics: {'loss': 0.31, 'acc': 0.88, 'f1': 0.79}
kept   : ['hi', 'ok']
```

Filtering vs conditional value (and not over-nesting)

Two very different **ifs** appear in comprehensions. An **if** at the **end filters** which items survive. An **if/else** in the **value** position transforms every item but chooses between two outputs. You can combine them, but if a comprehension needs more than one level of nesting or a guard, a plain loop is usually clearer, readability beats cleverness.

FILTER VS CONDITIONAL EXPRESSION

```
nums = [-2, -1, 0, 1, 2, 3]
positives = [x for x in nums if x > 0]           # filter
signs = ['pos' if x > 0 else 'non-pos' for x in nums] # map
# combine: transform only the survivors
inv = [round(1 / x, 2) for x in nums if x != 0]
print('filter :', positives)
print('map    :', signs)
print('combo  :', inv)
```

OUTPUT

```
filter : [1, 2, 3]
map    : ['non-pos', 'non-pos', 'non-pos', 'pos', 'pos', 'pos']
combo  : [-0.5, -1.0, 1.0, 0.5, 0.33]
```

Generator expressions: lazy and memory-cheap

Swap the square brackets for parentheses and you get a **generator expression**: it produces items one at a time instead of building the whole list, so it uses almost no memory. This is how you stream over a dataset too large to fit in RAM, and it plugs straight into `sum`, `max` and `any`.

STREAMING AGGREGATION WITHOUT A LIST

```
import sys
n = 1_000_000
gen = (x * x for x in range(n))      # nothing computed yet
total = sum(gen)                     # consumed lazily
print('sum of squares:', total)
lst = [x * x for x in range(n)]
print('list bytes ~', sys.getsizeof(lst))
print('gen bytes ~', sys.getsizeof(x*x for x in range(n)))
```

OUTPUT

```
sum of squares: 3333328333333500000
list bytes ~ 8448728
gen bytes ~ 208
```

Why it matters for AI: comprehensions are the idiomatic way to preprocess datasets (`[clean(x) for x in raw]`), and generator expressions let you stream huge corpora with a flat memory footprint. You will read and write these constantly.

CHAPTER 6

Control Flow, Truthiness, Unpacking and the Walrus

Beyond the obvious `if/for/while`, a handful of small Python conveniences show up so often in AI code that they are worth calling out on their own: truthiness, iterable unpacking, `enumerate/zip`, and the walrus operator.

Truthiness: empty things are False

Empty containers, `0`, and `None` are all *falsey*. This lets you write `if batch:` instead of `if len(batch) > 0:`. Note the important exception for arrays below.

FALSY VALUES IN GUARDS

```
for value in [[], [1], '', 'x', 0, 3, None]:
    print(f'{str(value)!r:>6} -> {bool(value)}')
batch = []
if not batch:
    print('skip: empty batch')
```

OUTPUT

```
'[]' -> False
'[1]' -> True
'' -> False
'x' -> True
'0' -> False
'3' -> True
'None' -> False
skip: empty batch
```

enumerate and zip: the loop duo

INDEXING AND PAIRING DURING ITERATION

```
names = ['loss', 'acc', 'lr']
vals  = [0.31, 0.88, 3e-4]
for i, (name, val) in enumerate(zip(names, vals)):
    print(f'{i}: {name:5} = {val}')
```

OUTPUT

```
0: loss  = 0.31
1: acc   = 0.88
2: lr    = 0.0003
```

Unpacking with * (star)

SPLITTING HEAD/TAIL AND MERGING

```
first, *rest = [10, 20, 30, 40]
print('first:', first, '| rest:', rest)
*init, last = [10, 20, 30, 40]
print('init :', init, '| last:', last)
a = [1, 2]; b = [3, 4]
merged = [*a, *b] # splat into a new list
cfg = {'**{'lr': 1e-3}, 'lr': 3e-4} # later key wins
print('merged:', merged, '| cfg:', cfg)
```

OUTPUT

```
first: 10 | rest: [20, 30, 40]
init : [10, 20, 30] | last: 40
merged: [1, 2, 3, 4] | cfg: {'lr': 0.0003}
```

The walrus operator := (assign inside an expression)

The walrus operator assigns and returns a value in the same expression, which avoids recomputing or duplicating a call, a common pattern in read-loops and early-stopping checks.

ASSIGN-AND-TEST IN ONE STEP

```

losses = [0.9, 0.6, 0.61, 0.60, 0.60]
best = float('inf')
for epoch, loss in enumerate(losses):
    if (improvement := best - loss) > 0.02:
        best = loss
        print(f'epoch {epoch}: improved by {improvement:.2f}')
    else:
        print(f'epoch {epoch}: no significant gain')

```

OUTPUT

```

epoch 0: improved by inf
epoch 1: improved by 0.30
epoch 2: no significant gain
epoch 3: no significant gain
epoch 4: no significant gain

```

for/else and while/else

A loop's **else** block runs only if the loop finished **without** `break`. This reads naturally as a search: 'scan the items; if none triggered the break, run the else'. It removes the usual `found = False` flag.

SEARCH WITH A LOOP ELSE

```

vocab = ['cat', 'dog', 'bird']
target = 'fish'
for word in vocab:
    if word == target:
        print('found:', word)
        break
else:
    print(target, 'is out of vocabulary')

```

OUTPUT

```

fish is out of vocabulary

```

Chained comparisons and the ternary

Python lets you chain comparisons the way maths does: `0 <= x < 1` is one expression that evaluates `x` once, ideal for validating

a probability or a learning rate. The ternary `a if cond else b` picks a value inline.

RANGE CHECKS AND INLINE CHOICE

```
for p in [-0.1, 0.5, 1.0, 1.4]:
    ok = 0.0 <= p <= 1.0          # chained comparison
    status = 'valid' if ok else 'OUT OF RANGE'
    print(f'p={p:>5} -> {status}')
```

OUTPUT

```
p= -0.1 -> OUT OF RANGE
p=  0.5 -> valid
p=  1.0 -> valid
p=  1.4 -> OUT OF RANGE
```

zip(strict=True): catch silent length mismatches

By default `zip` stops at the shortest input, which can silently drop data when your features and labels drift out of sync. Passing `strict=True` (3.10+) raises a `ValueError` instead, a cheap guard against a subtle data bug.

STRICT ZIP GUARDS AGAINST DRIFT

```
features = [[1], [2], [3]]
labels = [0, 1]          # one label missing!
try:
    pairs = list(zip(features, labels, strict=True))
    print(pairs)
except ValueError as e:
    print('length mismatch:', e)
```

OUTPUT

```
length mismatch: zip() argument 2 is shorter than argument 1
```

Why it matters for AI: `enumerate/zip` drive training loops, `*-unpacking` merges configs and splits sequences, and the walrus keeps early-stopping and streaming reads concise. Caution: never use `if array:` on a NumPy/torch tensor with more than one element, it raises an ambiguity error, use `.any()/all()` instead.

CHAPTER 7

Structural Pattern Matching (match / case)

Introduced in Python 3.10, `match/case` deconstructs data by *shape*, not just by value. It is a clean way to dispatch on the many differently-shaped messages that flow through AI systems: chat turns, tool-call payloads, config variants and streaming API events.

Matching values and structures

DISPATCH ON A CHAT MESSAGE SHAPE

```
def render(msg):
    match msg:
        case {'role': 'system', 'content': c}:
            return f'[system] {c}'
        case {'role': 'user', 'content': c}:
            return f'user> {c}'
        case {'role': 'tool', 'name': n}:
            return f'(tool call: {n})'
        case _:
            return '<unknown>'
for m in [{'role': 'system', 'content': 'Be brief.'},
          {'role': 'user', 'content': 'Hi'},
          {'role': 'tool', 'name': 'search'}]:
    print(render(m))
```

OUTPUT

```
[system] Be brief.
user> Hi
(tool call: search)
```

Capturing, guards and sequence patterns

MATCH ON STRUCTURE WITH AN IF-GUARD

```
def describe(point):
    match point:
        case (0, 0):
            return 'origin'
        case (x, 0):
            return f'on x-axis at {x}'
        case (x, y) if x == y:
            return f'diagonal at {x}'
        case (x, y):
            return f'point ({x}, {y})'
for p in [(0, 0), (5, 0), (3, 3), (1, 2)]:
    print(p, '->', describe(p))
```

OUTPUT

```
(0, 0) -> origin
(5, 0) -> on x-axis at 5
(3, 3) -> diagonal at 3
(1, 2) -> point (1, 2)
```

Class patterns with `__match_args__`

`match` can destructure objects by type and attribute. A `dataclass` supplies `__match_args__` automatically, so you can write positional class patterns, a clean way to dispatch on typed events or model configs.

MATCHING DATACLASS INSTANCES

```

from dataclasses import dataclass
@dataclass
class TextEvent:
    content: str
@dataclass
class ToolCall:
    name: str
    args: dict
def handle(ev):
    match ev:
        case TextEvent(content=c):
            return f'text: {c}'
        case ToolCall(name=n):
            return f'call: {n}'
for ev in [TextEvent('hi'), ToolCall('search', {'q': 'x'})]:
    print(handle(ev))

```

OUTPUT

```

text: hi
call: search

```

OR-patterns, as-capture and mapping rest

The `|` operator merges alternatives in one case, `as` binds the matched value to a name, and `**rest` captures the leftover keys of a mapping, exactly what you need when an API payload carries extra fields you want to keep.

OR-PATTERNS AND CAPTURING THE REST

```

def classify(event):
    match event:
        case {'type': ('start' | 'resume') as kind}:
            return f'run event: {kind}'
        case {'type': 'metric', 'name': n, **rest}:
            return f'metric {n}, extra={rest}'
        case _:
            return 'unhandled'
events = [{'type': 'resume'},
          {'type': 'metric', 'name': 'loss', 'step': 5}]
for e in events:
    print(classify(e))

```

OUTPUT

```
run event: resume  
metric loss, extra={'step': 5}
```

Why it matters for AI: LLM tool-calling and streaming responses arrive as tagged, differently-shaped dicts/events. `match` reads far better than a long `if/elif` chain when you branch on the *structure* of a payload.

PART II

Functions and Functional Programming

CHAPTER 8

Functions: Arguments, Defaults and the Mutable-Default Trap

Functions are the unit of reuse in every AI codebase, from a one-line loss to a full training step. Python's calling conventions are flexible: positional and keyword arguments, defaults, `*args`, `**kwargs`, and keyword-only parameters. Knowing them well is what lets you pass a whole config into a model with `model(**config)`.

Positional, keyword and default arguments

Arguments can be given by position or by name. Defaults make parameters optional, which is how hyperparameters get sensible fallbacks.

DEFAULTS AND KEYWORD CALLS

```
def train_step(lr=1e-3, momentum=0.9, clip=None):
    msg = f"lr={lr}, momentum={momentum}, clip={clip}"
    return msg

print(train_step())           # all defaults
print(train_step(0.01))      # positional lr
print(train_step(clip=1.0, lr=0.1)) # keyword, any order
```

OUTPUT

```
lr=0.001, momentum=0.9, clip=None
lr=0.01, momentum=0.9, clip=None
lr=0.1, momentum=0.9, clip=1.0
```

*args and **kwargs

`*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict. Frameworks use

them everywhere to forward options down a stack of layers without listing every parameter.

COLLECTING VARIADIC ARGUMENTS

```
def build_layer(name, *shape, **options):
    print("name :", name)
    print("shape :", shape)      # a tuple
    print("opts  :", options)    # a dict

    build_layer("dense", 784, 128, activation="relu", bias=False)
```

OUTPUT

```
name  : dense
shape : (784, 128)
opts  : {'activation': 'relu', 'bias': False}
```

Keyword-only arguments

Anything after a bare `*` in the signature must be passed by name. This prevents accidentally swapping two positional flags and makes call sites self-documenting.

FORCING CALLERS TO BE EXPLICIT

```
def sample(logits, *, temperature=1.0, greedy=False):
    return f"temp={temperature}, greedy={greedy}"

print(sample([0.1, 0.9], temperature=0.7))
try:
    sample([0.1, 0.9], 0.7)          # positional -> error
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
temp=0.7, greedy=False
TypeError: sample() takes 1 positional argument but 2 were give ←
n
```

The mutable-default trap

A default value is evaluated **once**, when the function is defined,

not on every call. A mutable default like `[]` is therefore *shared* across calls, a bug that silently corrupts accumulated data. The fix is the `None`-sentinel pattern.

THE BUG AND THE FIX

```
def buggy(x, acc=[]):          # DON'T: shared list
    acc.append(x)
    return acc

print("buggy:", buggy(1), buggy(2), buggy(3))

def fixed(x, acc=None):        # DO: fresh list each call
    if acc is None:
        acc = []
    acc.append(x)
    return acc

print("fixed:", fixed(1), fixed(2), fixed(3))
```

OUTPUT

```
buggy: [1, 2, 3] [1, 2, 3] [1, 2, 3]
fixed: [1] [2] [3]
```

Unpacking arguments at the call site

The mirror image of `*args/**kwargs`: use `*` to spread a sequence into positional arguments and `**` to spread a dict into keyword arguments. This is the idiom behind `model(**config)`.

SPREADING A CONFIG DICT

```
def make_optimizer(lr, weight_decay, betas):
    return f"Adam(lr={lr}, wd={weight_decay}, betas={betas})"

config = {"lr": 3e-4, "weight_decay": 0.01, "betas": (0.9, 0.999)}
print(make_optimizer(**config))    # dict -> keyword args

pair = (0.9, 0.999)
print("betas unpacked:", *pair)    # tuple -> positional args
```

OUTPUT

```
Adam(lr=0.0003, wd=0.01, betas=(0.9, 0.999))
betas unpacked: 0.9 0.999
```

Returning multiple values

A function returns one object, but that object can be a tuple, which Python packs and unpacks automatically. This is how a training step hands back both loss and accuracy, or how a data split returns train and test sets at once.

TUPLE RETURNS AND UNPACKING

```
def evaluate(preds, labels):
    correct = sum(p == y for p, y in zip(preds, labels))
    acc = correct / len(labels)
    loss = 1.0 - acc                # toy loss
    return loss, acc                # packed into a tuple

loss, acc = evaluate([1, 0, 1, 1], [1, 0, 0, 1])
print(f"loss={loss:.2f}, acc={acc:.2f}")
```

OUTPUT

```
loss=0.25, acc=0.75
```

First-class functions

In Python functions are ordinary objects: you can store them in variables, put them in a dict, and pass them as arguments. This is what makes a registry of activations or a table of metrics possible, and it underpins [map](#), [sorted\(key=\)](#) and decorators.

FUNCTIONS AS VALUES IN A REGISTRY

```
def relu(x):
    return max(0.0, x)

def leaky(x):
    return x if x > 0 else 0.01 * x

ACTIVATIONS = {"relu": relu, "leaky": leaky}  # a dispatch table

def apply(name, x):
    fn = ACTIVATIONS[name]  # look up a function by name
    return fn(x)

print("relu(-3) =", apply("relu", -3.0))
print("leaky(-3):", apply("leaky", -3.0))
```

OUTPUT

```
relu(-3) = 0.0
leaky(-3): -0.03
```

Closures: functions that remember state

A function defined inside another function *captures* the enclosing variables, forming a **closure**. The inner function keeps a live reference to that state after the outer function returns. A learning-rate scheduler factory is the classic example: it captures the base rate and decay, and each call advances a private step counter via `nonlocal`.

A LEARNING-RATE SCHEDULER FACTORY

```
def make_scheduler(base_lr, decay=0.5, every=2):
    step = 0                                # captured, private state
    def next_lr():
        nonlocal step
        lr = base_lr * (decay ** (step // every))
        step += 1
        return lr
    return next_lr                           # a closure over base_lr/s ←
    tep

sched = make_scheduler(1.0, decay=0.5, every=2)
print([round(sched(), 3) for _ in range(6)])
```

OUTPUT

```
[1.0, 1.0, 0.5, 0.5, 0.25, 0.25]
```

Positional-only parameters with /

Parameters before a `/` in the signature can only be passed by position, never by name. This lets you rename them freely later, and mirrors many C-implemented builtins. It pairs with the keyword-only `*` to give precise control over how a function is called.

LOCKING A PARAMETER TO POSITION

```
def scale(x, /, factor=2.0):
    return x * factor                        # x is positional-only

print("by position:", scale(10))
print("keyword factor:", scale(10, factor=3.0))
try:
    scale(x=10)                             # naming x -> error
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
by position: 20.0
keyword factor: 30.0
TypeError: scale() got some positional-only arguments passed as ←
keyword arguments: 'x'
```

Why it matters for AI: config-driven training relies on `**config` unpacking, and the mutable-default trap is a real source of data-leak bugs (e.g. a metric list that never resets between runs). Prefer `None` sentinels for any list/dict/set default. Closures capture scheduler and counter state cleanly, and first-class functions let you build registries of layers, losses and metrics keyed by name.

CHAPTER 9

Lambdas, map, filter and sorted(key=)

A `lambda` is a small anonymous function, most useful as the `key` = argument to `sorted`, `min` and `max`. Combined with `map` and `filter`, lambdas express data transforms compactly, though comprehensions are often clearer.

lambda basics

ANONYMOUS ONE-LINERS

```
square = lambda x: x * x
scale = lambda x, s=2.0: x * s
print(square(5), scale(3), scale(3, 10))
```

OUTPUT

```
25 6.0 30
```

map and filter (and why comprehensions win)

`map` applies a function to every item; `filter` keeps items where a predicate is true. Both return lazy iterators. For readability, a list/generator comprehension usually expresses the same thing more clearly.

MAP/FILTER VS COMPREHENSION

```
nums = [-2, -1, 0, 1, 2, 3]

# functional style
sq = list(map(lambda x: x * x, filter(lambda x: x > 0, nums)))
print("map/filter :", sq)

# comprehension style (usually preferred)
sq2 = [x * x for x in nums if x > 0]
print("comprehension:", sq2)
```

OUTPUT

```
map/filter : [1, 4, 9]
comprehension: [1, 4, 9]
```

sorted(key=): the workhorse

`key=` maps each element to a sort key. Sorting samples by sequence length (to build efficient batches) or ranking candidates by score are everyday uses.

SORT SAMPLES BY LENGTH

```
batch = ["hi", "hello there", "hey", "good morning all"]
by_len = sorted(batch, key=len)
print("shortest first:", by_len)
print("longest first :", sorted(batch, key=len, reverse=True))
```

OUTPUT

```
shortest first: ['hi', 'hey', 'hello there', 'good morning all']
longest first : ['good morning all', 'hello there', 'hey', 'hi']
```

Top-k with sorted, min and max

`max/min` also take `key=`. Selecting the best prediction or the top-k scoring candidates is a one-liner.

BEST AND TOP-K CANDIDATES

```
candidates = [("cat", 0.12), ("dog", 0.71), ("fox", 0.55), ("owl", 0.33)]

best = max(candidates, key=lambda c: c[1])
print("argmax:", best)

top2 = sorted(candidates, key=lambda c: c[1], reverse=True)[:2]
print("top-2 :", top2)
```

OUTPUT

```
argmax: ('dog', 0.71)
top-2 : [('dog', 0.71), ('fox', 0.55)]
```


operator: faster, clearer sort keys

The `operator` module provides ready-made key functions: `itemgetter(i)` pulls an element by index or key, `attrgetter('x')` pulls an attribute, and `methodcaller('m')` calls a method. They read better than a `lambda` and, being written in C, run faster on large datasets, exactly the sort keys you want when ordering millions of samples.

ITEMGETTER VS LAMBDA

```
from operator import itemgetter

candidates = [("cat", 0.12), ("dog", 0.71), ("fox", 0.55)]
by_score = sorted(candidates, key=itemgetter(1), reverse=True)
print("by score:", by_score)

# clearer and faster than: key=lambda c: c[1]
best = max(candidates, key=itemgetter(1))
print("argmax  :", best)
```

OUTPUT

```
by score: [('dog', 0.71), ('fox', 0.55), ('cat', 0.12)]
argmax   : ('dog', 0.71)
```

Multi-key sort with itemgetter

Pass several indices to `itemgetter` to sort by a tuple of keys: `itemgetter(1, 0)` sorts by field 1, breaking ties with field 0. Python's sort is **stable**, meaning equal keys keep their original order, so you can also get a secondary sort by sorting twice, least-significant key first.

SORT BY SCORE, THEN NAME

```

from operator import itemgetter

rows = [("dog", 2), ("cat", 1), ("ant", 2), ("bee", 1)]

# single pass: primary count, secondary name (both ascending)
combined = sorted(rows, key=itemgetter(1, 0))
print("count then name:", combined)

# two-pass via stable sort: sort by name, then by count
tmp = sorted(rows, key=itemgetter(0))           # secondary key
first
stable = sorted(tmp, key=itemgetter(1))         # primary key la
st
print("stable two-pass:", stable)

```

OUTPUT

```

count then name: [('bee', 1), ('cat', 1), ('ant', 2), ('dog', 2)
)
stable two-pass: [('bee', 1), ('cat', 1), ('ant', 2), ('dog', 2)
)

```

attrgetter and methodcaller

`attrgetter` sorts objects by an attribute, common when your samples are dataclasses. `methodcaller('m', *args)` builds a key that calls a method on each element, for example sorting strings case-insensitively via `str.lower`.

SORT OBJECTS AND BY METHOD

```

from operator import attrgetter, methodcaller
from dataclasses import dataclass

@dataclass
class Sample:
    text: str
    length: int

data = [Sample("BB", 2), Sample("a", 1), Sample("CCC", 3)]
print("by length:", [s.text for s in sorted(data, key=attrgetter(
    r("length")))])

words = ["banana", "Apple", "cherry"]
print("caseless :", sorted(words, key=methodcaller("lower")))

```

OUTPUT

```

by length: ['a', 'BB', 'CCC']
caseless : ['Apple', 'banana', 'cherry']

```

Why it matters for AI: `sorted(..., key=...)` and `max(..., key=...)` implement argmax, ranking, top-k selection and length-bucketing without any library. Reach for a comprehension over `map/filter` unless you already have a named function to pass, and prefer `operator.itemgetter/attrgetter` over `lambda` for sort keys, they are clearer and faster on large datasets.

CHAPTER 10

functools: partial, lru_cache, reduce, wraps

The `functools` module contains higher-order tools that show up constantly: freezing arguments with `partial`, memoizing with `lru_cache`, folding sequences with `reduce`, and preserving metadata with `wraps`.

partial: freezing hyperparameters

`functools.partial` returns a new callable with some arguments already filled in. It is the clean way to preconfigure an optimizer, activation or metric before passing it somewhere that will call it with the remaining arguments.

PRE-BINDING ARGUMENTS

```
from functools import partial

def scaled_dropout(x, rate, training):
    return f"x={x}, rate={rate}, training={training}"

# freeze rate=0.1; callers only pass x and training
dropout = partial(scaled_dropout, rate=0.1)
print(dropout(1.0, training=True))
print(dropout(2.0, training=False))
```

OUTPUT

```
x=1.0, rate=0.1, training=True
x=2.0, rate=0.1, training=False
```

lru_cache / cache: memoization

`@lru_cache` stores results keyed by arguments, so repeated calls with the same inputs return instantly. Great for expensive pure functions like tokenizer lookups or recursive computations. `@cache` is an unbounded shortcut (Python 3.9+).

CACHE EXPENSIVE PURE CALLS

```

from functools import lru_cache

calls = {"n": 0}

@lru_cache(maxsize=None)
def fib(n):
    calls["n"] += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print("fib(30) =", fib(30))
print("actual calls:", calls["n"])    # linear, not exponential
print("cache info :", fib.cache_info())

```

OUTPUT

```

fib(30) = 832040
actual calls: 31
cache info : CacheInfo(hits=28, misses=31, maxsize=None, currsi ←
                ze=31)

```

reduce: folding a sequence

`functools.reduce` collapses a sequence to a single value by repeatedly applying a two-argument function. Useful for chaining transforms or multiplying a shape into a flat size.

PRODUCT OF A TENSOR SHAPE

```

from functools import reduce

shape = (32, 3, 224, 224)                # (batch, channels, H, ←
    W)
num_elements = reduce(lambda a, b: a * b, shape)
print("elements per batch tensor:", num_elements)

```

OUTPUT

```

elements per batch tensor: 4816896

```

reduce with an initializer

A third argument to `reduce` is the starting accumulator. It fixes

the result type and, crucially, makes `reduce` safe on an empty sequence (without it, an empty input raises). Here we compose a list of preprocessing steps into one pipeline function, starting from the identity.

FOLD TRANSFORMS INTO A PIPELINE

```
from functools import reduce

steps = [lambda s: s.strip(),
         lambda s: s.lower(),
         lambda s: s.replace(" ", "_")]

def compose(f, g):
    return lambda s: g(f(s))          # apply f, then g

pipeline = reduce(compose, steps, lambda s: s)  # start = identity
print(repr(pipeline(" Hello World ")))

# initializer also guards the empty case
print("empty sum:", reduce(lambda a, b: a + b, [], 0))
```

OUTPUT

```
'hello_world'
empty sum: 0
```

cache vs lru_cache and eviction

`@cache` is an unbounded memo (never evicts, keeps everything). `@lru_cache(maxsize=N)` keeps only the `N` most-recently-used entries and evicts the least-recently-used when full, trading some recomputation for bounded memory. Watch the `misses` and `currsize` fields of `cache_info()` to see eviction happen.

BOUNDED LRU EVICTION IN ACTION

```

from functools import lru_cache, cache

@lru_cache(maxsize=2)                # room for 2 entries only
def embed(token):
    return len(token)                # pretend: expensive lookup

for tok in ["a", "bb", "a", "ccc", "bb"]:
    embed(tok)
print("lru :", embed.cache_info())  # evictions once 3rd key seen

@cache                               # unbounded, never evicts
def square(n):
    return n * n

square(2); square(3); square(2)
print("cache:", square.cache_info())

```

OUTPUT

```

lru : CacheInfo(hits=1, misses=4, maxsize=2, currsize=2)
cache: CacheInfo(hits=1, misses=2, maxsize=None, currsize=2)

```

partial on methods

partial also freezes the first argument of a bound method, letting you turn a general method into a specialised callable, for example a metric object whose threshold is fixed once and then called on many prediction batches.

SPECIALISING A BOUND METHOD

```

from functools import partial

class Metric:
    def score(self, preds, threshold):
        hits = sum(p >= threshold for p in preds)
        return hits / len(preds)

m = Metric()
at_half = partial(m.score, threshold=0.5)  # freeze the threshold
old
print("frac >= 0.5:", at_half([0.2, 0.6, 0.9, 0.4]))
print("frac >= 0.5:", at_half([0.7, 0.8]))

```

OUTPUT

```

frac >= 0.5: 0.5
frac >= 0.5: 1.0

```

singledispatch: type-based dispatch

`@singledispatch` turns a function into a generic that picks an implementation based on the type of its **first** argument. It is a clean alternative to a chain of `isinstance` checks, ideal for a preprocessing function that must accept strings, numbers or lists and normalise each differently.

ONE FUNCTION, MANY INPUT TYPES


```
from functools import singledispatch

@singledispatch
def to_tokens(x):
    raise TypeError(f"unsupported: {type(x).__name__}")

@to_tokens.register
def _(x: str):
    return x.split()

@to_tokens.register
def _(x: int):
    return [str(x)]

@to_tokens.register(list)
def _(x):
    return [t for item in x for t in to_tokens(item)]

print(to_tokens("hello world"))
print(to_tokens(42))
print(to_tokens(["a b", 7]))
```

OUTPUT

```
['hello', 'world']
['42']
['a', 'b', '7']
```

cached_property: compute once per instance

`@cached_property` turns a method into an attribute that is computed on first access and then stored on the instance, so later reads are free. Handy for a derived value like a vocabulary size that is expensive to build but never changes.

LAZY, MEMOIZED ATTRIBUTE

```

from functools import cached_property

class Corpus:
    def __init__(self, docs):
        self.docs = docs
    @cached_property
    def vocab(self):
        print("(building vocab...)"          # runs only once
        return sorted({w for d in self.docs for w in d.split()})
        )

c = Corpus(["a b", "b c"])
print(c.vocab)                          # builds
print(c.vocab)                          # cached, no rebuild

```

OUTPUT

```

(bbuilding vocab...)
['a', 'b', 'c']
['a', 'b', 'c']

```

wraps: honest decorators

When you write a decorator (see the Decorators chapter), the wrapper replaces the original function and loses its name and docstring. `functools.wraps` copies that metadata back, which matters for debugging and introspection.

PRESERVING `__NAME__` AND `__DOC__`

```
from functools import wraps

def logged(fn):
    @wraps(fn)                                # copy metadata from fn
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    return wrapper

@logged
def predict(x):
    "Run the model forward pass."
    return x

print("name:", predict.__name__)             # 'predict', not 'wrapper'
print("doc :", predict.__doc__)
```

OUTPUT

```
name: predict
doc : Run the model forward pass.
```

Why it matters for AI: [partial](#) preconfigures optimizers and transforms in data pipelines, [lru_cache](#) avoids recomputing tokenization or feature lookups, and [wraps](#) keeps your training-loop decorators debuggable. [singledispatch](#) gives clean type-based preprocessing, and [cached_property](#) memoizes expensive derived values like vocabularies per instance.

CHAPTER 11

itertools: Batching and Combining Iterables

`itertools` provides memory-efficient building blocks for iteration. They operate lazily on streams, so you can process datasets far larger than RAM. The stars for AI work are `chain`, `islice`, `product`, `combinations` and `batched`.

chain: concatenate iterables

`chain` treats several iterables as one continuous stream, for example merging train shards without building a giant list.

MERGE DATASET SHARDS LAZILY

```
from itertools import chain

shard_a = ["a1", "a2"]
shard_b = ["b1", "b2", "b3"]
for i, item in enumerate(chain(shard_a, shard_b)):
    print(i, item)
```

OUTPUT

```
0 a1
1 a2
2 b1
3 b2
4 b3
```

islice and count: windows into infinite streams

`count` is an infinite counter and `islice` takes a finite slice from any iterator, so you can peek at the first N items of an endless generator without materialising it.

TAKE THE FIRST N OF A STREAM

```

from itertools import count, islice

steps = count(start=0, step=10)          # 0, 10, 20, ... forever
first5 = list(islice(steps, 5))
print("first 5 lr-warmup steps:", first5)

```

OUTPUT

```
first 5 lr-warmup steps: [0, 10, 20, 30, 40]
```

product: hyperparameter grids

`product` is the Cartesian product, i.e. nested loops without the nesting. It is the natural way to enumerate a grid search over hyperparameters.

GRID SEARCH OVER HYPERPARAMETERS

```

from itertools import product

lrs = [1e-2, 1e-3]
batch_sizes = [16, 32]
for lr, bs in product(lrs, batch_sizes):
    print(f"run: lr={lr}, batch_size={bs}")

```

OUTPUT

```

run: lr=0.01, batch_size=16
run: lr=0.01, batch_size=32
run: lr=0.001, batch_size=16
run: lr=0.001, batch_size=32

```

combinations: pairs without repetition

`combinations` yields unordered subsets, handy for building positive/negative pairs in contrastive learning or comparing every model against every other.

ALL UNIQUE MODEL PAIRS

```
from itertools import combinations

models = ["A", "B", "C"]
for m1, m2 in combinations(models, 2):
    print(f"compare {m1} vs {m2}")
```

OUTPUT

```
compare A vs B
compare A vs C
compare B vs C
```

batched: grouping a stream into mini-batches

Since Python 3.12, `itertools.batched` chops an iterable into tuples of a fixed size, which is exactly what a data loader does to turn a stream of samples into mini-batches. The final batch may be shorter.

STREAM -> MINI-BATCHES

```
from itertools import batched

samples = range(10)                                # pretend: 10 samples
for step, mb in enumerate(batched(samples, 4)):
    print(f"step {step}: batch={mb} (size {len(mb)})")
```

OUTPUT

```
step 0: batch=(0, 1, 2, 3) (size 4)
step 1: batch=(4, 5, 6, 7) (size 4)
step 2: batch=(8, 9) (size 2)
```

accumulate: running totals

`accumulate` yields a running reduction, a cumulative sum by default (like `cumsum`), or any binary function you pass. Use it to turn per-step increments into a schedule, or per-batch counts into a cumulative sample position.

CUMULATIVE SUM OF A SCHEDULE

```

from itertools import accumulate
import operator

warmup = [0.1, 0.2, 0.3, 0.4]
print("cumsum  :", list(accumulate(warmup)))
print("running max:", list(accumulate([3, 1, 4, 1, 5], max)))
print("cumprod  :", list(accumulate([1, 2, 3, 4], operator.mul)
))

```

OUTPUT

```

cumsum  : [0.1, 0.30000000000000004, 0.6000000000000001, 1.0]
running max: [3, 3, 4, 4, 5]
cumprod  : [1, 2, 6, 24]

```

groupby: group consecutive items

`groupby` groups **adjacent** items sharing a key, so the input must be sorted by that key first. It is the standard way to bucket samples by label or split a stream into runs.

GROUP SAMPLES BY LABEL

```

from itertools import groupby
from operator import itemgetter

samples = [("cat", 0), ("dog", 1), ("cat", 2), ("dog", 3)]
samples.sort(key=itemgetter(0))          # sort before grouping
!
for label, group in groupby(samples, key=itemgetter(0)):
    ids = [s[1] for s in group]
    print(f"{label}: {ids}")

```

OUTPUT

```

cat: [0, 2]
dog: [1, 3]

```

zip_longest: pad ragged sequences

Plain `zip` stops at the shortest input; `zip_longest` runs to the longest and fills gaps with a `fillvalue`. This is exactly how you pad variable-length sequences into a rectangular batch.

PAD SEQUENCES TO EQUAL LENGTH

```
from itertools import zip_longest

seqs = [[1, 2, 3], [4], [5, 6]]
padded = list(zip_longest(*seqs, fillvalue=0))
# transpose back to per-sequence rows
rows = [list(col) for col in zip(*padded)]
print("padded batch:", rows)
```

OUTPUT

```
padded batch: [[1, 2, 3], [4, 0, 0], [5, 6, 0]]
```

starmap: apply over pre-zipped args

`starmap` is like `map` but unpacks each item as the function's arguments, i.e. `f(*args)`. Perfect when your data already comes as tuples of arguments, such as (prediction, label) pairs.

MAP A 2-ARG FUNCTION OVER PAIRS

```
from itertools import starmap

pairs = [(0.9, 1), (0.2, 0), (0.6, 1)]
def loss(p, y):
    return round((p - y) ** 2, 3)
print("per-sample loss:", list(starmap(loss, pairs)))
```

OUTPUT

```
per-sample loss: [0.01, 0.04, 0.16]
```

pairwise: sliding windows of two

`pairwise` (Python 3.10+) yields overlapping consecutive pairs, the simplest sliding window. Use it to compute step-to-step deltas of a loss curve or gaps between checkpoints.

DELTA BETWEEN CONSECUTIVE LOSSES


```
from itertools import pairwise

losses = [2.0, 1.6, 1.5, 1.1]
deltas = [round(b - a, 2) for a, b in pairwise(losses)]
print("step-to-step change:", deltas)
```

OUTPUT

```
step-to-step change: [-0.4, -0.1, -0.4]
```

cycle + islice: repeat a finite stream

`cycle` repeats an iterable forever; bound it with `islice` to draw a fixed number of items. Cycling class labels is a quick way to build a round-robin assignment or a repeating schedule.

ROUND-ROBIN OVER A SMALL SET

```
from itertools import cycle, islice

folds = cycle(["train", "val"])          # infinite alternation
assignment = list(islice(folds, 5))
print("fold assignment:", assignment)
```

OUTPUT

```
fold assignment: ['train', 'val', 'train', 'val', 'train']
```

takewhile and dropwhile

`takewhile` yields items until the predicate first fails, then stops; `dropwhile` skips items until it first fails, then yields the rest. Handy for trimming a warmup prefix or reading an early-stopping run.

SPLIT A STREAM AT A THRESHOLD

```

from itertools import takewhile, dropwhile

acc = [0.1, 0.3, 0.5, 0.49, 0.6]
rising = list(takewhile(lambda a: a < 0.5, acc))
rest = list(dropwhile(lambda a: a < 0.5, acc))
print("before 0.5:", rising)
print("from 0.5 :", rest)

```

OUTPUT

```

before 0.5: [0.1, 0.3]
from 0.5 : [0.5, 0.49, 0.6]

```

tee: fork an iterator

tee splits one iterator into several independent ones, so you can traverse a stream twice, for example to compute a running mean and a max over the same data. Do not keep using the original iterator after teeing it.

TWO PASSES OVER ONE STREAM

```

from itertools import tee

data = iter([3, 1, 4, 1, 5])
a, b = tee(data, 2)                                # two independent copies
total = sum(a)
peak = max(b)
print(f"mean={total / 5:.1f}, max={peak}")

```

OUTPUT

```

mean=2.8, max=5

```

Why it matters for AI: `itertools` is the pure-Python toolkit for data pipelines. `chain` merges shards, `islice` bounds infinite generators, `product` enumerates hyperparameter grids, and `batched` builds mini-batches, all lazily and without loading everything into memory. `accumulate` builds schedules, `groupby` buckets samples by label, `zip_longest` pads ragged batches, and `pairwise` gives sliding windows over a loss curve.

PART III

Iteration, Generators and Data Pipelines

CHAPTER 12

Iterables, Iterators and the Iterator Protocol

Every `for` loop in Python, including the one that walks your training data, is built on the *iterator protocol*. Understanding it lets you write custom data loaders that plug straight into `for batch in loader:` syntax.

`iter()` and `next()`

`iter(obj)` asks an *iterable* for an *iterator*; `next(it)` pulls the next value and raises `StopIteration` when it is exhausted. A `for` loop is just this in disguise.

DRIVING AN ITERATOR BY HAND

```
data = [10, 20, 30]
it = iter(data)
print(next(it))
print(next(it))
print(next(it))
try:
    next(it)
except StopIteration:
    print("StopIteration: stream exhausted")
```

OUTPUT

```
10
20
30
StopIteration: stream exhausted
```

How a `for`-loop really works

A `for` loop calls `iter()` once, then `next()` repeatedly, catching `StopIteration` to stop. This desugaring is worth seeing once so iterators feel less magical.

FOR-LOOP DESUGARED

```
def manual_for(iterable, body):
    it = iter(iterable)
    while True:
        try:
            item = next(it)
        except StopIteration:
            break
        body(item)

manual_for(["a", "b", "c"], lambda x: print("got", x))
```

OUTPUT

```
got a
got b
got c
```

`__iter__` and `__next__`: a custom iterator

An object is iterable if it defines `__iter__`; it is an iterator if it also defines `__next__`. Here is a mini data loader that yields shuffled batch *indices*, the same shape as a real [DataLoader](#).

A MINIMAL INDEX DATALOADER

```
import random

class BatchSampler:
    def __init__(self, n, batch_size, seed=0):
        self.idx = list(range(n))
        random.Random(seed).shuffle(self.idx)
        self.batch_size = batch_size

    def __iter__(self):
        self.pos = 0
        return self

    def __next__(self):
        if self.pos >= len(self.idx):
            raise StopIteration
        batch = self.idx[self.pos:self.pos + self.batch_size]
        self.pos += self.batch_size
        return batch

for batch in BatchSampler(7, batch_size=3):
    print("batch indices:", batch)
```

OUTPUT

```
batch indices: [4, 2, 1]
batch indices: [0, 5, 3]
batch indices: [6]
```

Iterable vs iterator: not the same thing

The distinction trips people up. An *iterable* can produce a fresh iterator every time you call `iter()` on it, so a `list` supports many independent passes. An *iterator* is a one-shot cursor: calling `iter()` on it returns *itself*, so two names share the same position and one pass exhausts it.

ONE IS REUSABLE, ONE IS NOT

```

data = [1, 2, 3]
a = iter(data)
b = iter(data)                                # independent cursor
print("independent iterators:", a is b)
print("first of a:", next(a), "first of b:", next(b))

it = iter(data)
print("iter(iterator) is itself:", iter(it) is it)

```

OUTPUT

```

independent iterators: False
first of a: 1 first of b: 1
iter(iterator) is itself: True

```

iter(callable, sentinel): read until a marker

The two-argument form of `iter()` calls a zero-argument *callable* repeatedly and stops when the result equals the *sentinel*. It is the clean way to drain a source, such as fixed-size chunks from a file, until an end marker appears, with no `while True / break` boilerplate.

CHUNK A STREAM UNTIL SENTINEL

```

import io

buffer = io.StringIO("abcdefghij")
# read 4 chars at a time; "" (EOF) is the sentinel that stops iteration
read_chunk = lambda: buffer.read(4)
for chunk in iter(read_chunk, ""):
    print("chunk:", chunk)

```

OUTPUT

```

chunk: abcd
chunk: efgh
chunk: ij

```


Why it matters for AI: PyTorch [Dataset/DataLoader](#), Hugging Face streaming datasets and tf.data all speak the iterator protocol. Implement `__iter__`/`__next__` and your data source works with `for batch in loader:` and `enumerate` for free.

CHAPTER 13

Generators and yield: Streaming Datasets and Mini-Batches

A *generator* is the easiest way to build an iterator: write a normal function but use `yield` instead of `return`. Execution pauses at each `yield` and resumes on the next request, so values are produced lazily, one at a time, using almost no memory.

yield basics

Calling a generator function does not run it; it returns a generator object. Each `next()` runs until the following `yield`.

A GENERATOR COUNTS LAZILY

```
def warmup_lrs(base, steps):
    for s in range(1, steps + 1):
        yield base * s / steps          # produced on demand

gen = warmup_lrs(1e-3, 4)
print("type:", type(gen).__name__)
for lr in gen:
    print(f"lr = {lr:.4e}")
```

OUTPUT

```
type: generator
lr = 2.5000e-04
lr = 5.0000e-04
lr = 7.5000e-04
lr = 1.0000e-03
```

Laziness = memory efficiency

A generator never holds the whole sequence in memory. This lets you stream a dataset of millions of rows and process it one

record at a time, which a list of the same data could not fit.

STREAM A HUGE DATASET LAZILY

```
import sys

def read_huge_dataset(n):
    for i in range(n):                # pretend each line is read from disk
        yield f"sample_{i}"

stream = read_huge_dataset(10_000_000)
# only the generator object exists, not 10M strings
print("generator size (bytes):", sys.getsizeof(stream))
# consume just the first three without materialising the rest
for _ in range(3):
    print(next(stream))
```

OUTPUT

```
generator size (bytes): 208
sample_0
sample_1
sample_2
```

A mini-batch generator

Generators shine as batchers: pull items from an upstream iterable and `yield` fixed-size lists. Because it is lazy, it works on infinite or on-disk streams too.

GROUP A STREAM INTO BATCHES

```
def batches(iterable, size):
    batch = []
    for item in iterable:
        batch.append(item)
        if len(batch) == size:
            yield batch
            batch = []
    if batch:
        # yield the short final batch
        yield batch

for b in batches(range(7), 3):
    print("batch:", b)
```

OUTPUT

```
batch: [0, 1, 2]
batch: [3, 4, 5]
batch: [6]
```

Generators are single-use

A generator carries its own execution state and is exhausted after one full pass. To iterate again (a new epoch) you must create a fresh generator, a common source of silent empty second epochs.

EXHAUSTION AFTER ONE PASS

```
def gen():
    yield from range(3)

g = gen()
print("epoch 1:", list(g))
print("epoch 2:", list(g))          # empty: already exhausted
```

OUTPUT

```
epoch 1: [0, 1, 2]
epoch 2: []
```

Two-way generators with .send()

`yield` is also an expression: `x = yield value` sends `value` out and, on the next `.send(v)`, resumes with `x == v`. This turns a generator into a tiny stateful coroutine, ideal for a running metric that updates as batches arrive without a class.

A RUNNING-AVERAGE GENERATOR

```
def running_average():
    total, count = 0.0, 0
    avg = None
    while True:
        x = yield avg                # emit current avg, wait for next
        or next value
        total += x
        count += 1
        avg = total / count

loss = running_average()
next(loss)                        # prime: advance to the first yield
for batch_loss in [1.0, 0.6, 0.2, 0.4]:
    print(f"loss={batch_loss:.1f}  avg={loss.send(batch_loss):.3f}")
```

OUTPUT

```
loss=1.0  avg=1.000
loss=0.6  avg=0.800
loss=0.2  avg=0.600
loss=0.4  avg=0.550
```

.throw() and .close(): the generator lifecycle

Beyond `.send()`, `.throw(exc)` raises an exception *inside* the generator at the paused `yield` (letting it clean up or skip a bad record), and `.close()` raises `GeneratorExit` so `finally` blocks run, releasing files or sockets. This is the whole generator lifecycle.

THROW, CLOSE AND FINALLY

```

def worker():
    try:
        while True:
            item = yield
            print("processed", item)
    except ValueError as e:
        print("recovered from:", e)
    finally:
        print("cleanup: releasing resources")

w = worker()
next(w)                                # prime
w.send("a")
try:
    w.throw(ValueError("bad record"))    # caught inside the gen
    erator
except StopIteration:
    print("generator finished after throw")
w.close()                              # no-op: already closed

```

OUTPUT

```

processed a
recovered from: bad record
cleanup: releasing resources
generator finished after throw

```

Infinite streams tamed with islice

A generator can loop forever; `itertools.islice` slices a lazy stream without materialising it, giving you exactly k items. This is the standard way to take a bounded sample from an endless source such as an augmentation pipeline.

ISLICE A NEVER-ENDING STREAM

```
import itertools

def augment_forever(seed):
    i = 0
    while True:
        yield f"{seed}_aug{i}"
        i += 1

first_five = list(itertools.islice(augment_forever("img"), 5))
print(first_five)
```

OUTPUT

```
['img_aug0', 'img_aug1', 'img_aug2', 'img_aug3', 'img_aug4']
```

An epoch loader that reshuffles each pass

Because a generator is single-use, wrap the logic in a function you call once per epoch. Here each call reshuffles and yields fresh mini-batches, exactly how a training loop requests a new iterator every epoch.

ONE FRESH GENERATOR PER EPOCH

```
import random

def epoch_loader(data, batch_size, seed):
    order = list(data)
    random.Random(seed).shuffle(order)
    for i in range(0, len(order), batch_size):
        yield order[i:i + batch_size]

data = list(range(6))
for epoch in range(2):
    batches = epoch_loader(data, batch_size=2, seed=epoch)
    print(f"epoch {epoch}:", list(batches))
```

OUTPUT

```
epoch 0: [[4, 2], [1, 0], [5, 3]]
epoch 1: [[2, 3], [5, 0], [4, 1]]
```

Why it matters for AI: streaming data loaders, tokenizing corpora that do not fit in RAM, and yielding mini-batches are all generator patterns. Remember that a generator is single-use, so wrap it in a function you call once per epoch.

CHAPTER 14

Generator Expressions, `yield from` and Lazy Pipelines

Generator expressions give you a generator with comprehension syntax, and `yield from` delegates to a sub-generator. Together they let you assemble multi-stage data pipelines that stay lazy from end to end.

Generator expression vs list comprehension

Swap the `[]` of a list comprehension for `()` and you get a generator expression: same syntax, but it computes items on demand instead of building the whole list. For large data the memory difference is enormous.

`()` IS LAZY, `[]` IS EAGER

```
import sys

n = 1_000_000
listcomp = [x * x for x in range(n)]      # builds 1M ints now
genexpr = (x * x for x in range(n))      # builds nothing yet

print("list bytes:", sys.getsizeof(listcomp))
print("gen  bytes:", sys.getsizeof(genexpr))
# feed a genexpr straight into a reducer, no intermediate list
print("sum of squares:", sum(x * x for x in range(n)))
```

OUTPUT

```
list bytes: 8448728
gen  bytes: 208
sum of squares: 333332833333500000
```

Chaining generators into a pipeline

Because each stage consumes and produces a lazy stream, you can wire *read* -> *clean* -> *tokenize* -> *batch* together. Data flows one record at a time, so peak memory is just one record

plus one batch.

READ -> CLEAN -> TOKENIZE -> BATCH

```
def read(lines):
    for line in lines:
        yield line

def clean(stream):
    for line in stream:
        yield line.strip().lower()

def tokenize(stream):
    for line in stream:
        yield line.split()

def batch(stream, size):
    buf = []
    for item in stream:
        buf.append(item)
        if len(buf) == size:
            yield buf; buf = []
    if buf:
        yield buf

raw = [" Hello WORLD ", "Deep Learning", " GPUs go BRRR "]
pipeline = batch(tokenize(clean(read(raw))), size=2)
for b in pipeline:
    print(b)
```

OUTPUT

```
[['hello', 'world'], ['deep', 'learning']]
[['gpus', 'go', 'brrr']]
```

yield from: delegation and flattening

`yield from sub` yields every item of `sub` in turn, forwarding cleanly without a manual loop. It is the natural way to flatten nested shards into one stream.

FLATTEN SHARDS WITH YIELD FROM

```
def flatten(shards):
    for shard in shards:
        yield from shard          # splice each shard's items in

shards = [["a", "b"], ["c"], ["d", "e", "f"]]
print("flattened:", list(flatten(shards)))
```

OUTPUT

```
flattened: ['a', 'b', 'c', 'd', 'e', 'f']
```

Recursive yield from for nested structure

Because `yield from` accepts any iterable, including another call to the same generator, it flattens arbitrarily nested structures in a few lines, handy for ragged config or nested shard directories.

DEEP-FLATTEN A NESTED LIST

```
def deep_flatten(obj):
    for item in obj:
        if isinstance(item, list):
            yield from deep_flatten(item)    # recurse
        else:
            yield item

nested = [1, [2, [3, 4], 5], [[6]], 7]
print("deep flat:", list(deep_flatten(nested)))
```

OUTPUT

```
deep flat: [1, 2, 3, 4, 5, 6, 7]
```

Memory: a full pipeline stays flat

The payoff of composing generators is that peak memory does not grow with the dataset. Below, the lazy pipeline processes a large stream while holding only a running total; the eager list version allocates every intermediate at once.

LAZY VS EAGER PEAK MEMORY

```
import sys

n = 1_000_000
# eager: materialise all squares, then all evens, then sum
eager = [x for x in [v * v for v in range(n)] if x % 2 == 0]
print("eager list bytes:", sys.getsizeof(eager))

# lazy: one value in flight at a time, nothing materialised
lazy = (x for x in (v * v for v in range(n)) if x % 2 == 0)
print("lazy gen bytes: ", sys.getsizeof(lazy))
print("both sums equal:", sum(eager) == sum(lazy))
```

OUTPUT

```
eager list bytes: 4167352
lazy gen bytes:    208
both sums equal: True
```

The same idea in the frameworks

These generator patterns are exactly what production data APIs expose. A [torch IterableDataset](#) implements `__iter__` as a generator, and [tf.data.Dataset.from_generator](#) wraps one directly; both then add batching, shuffling and prefetching.

ILLUSTRATIVE FRAMEWORK WRAPPERS

```
# Illustrative only (frameworks not required to run this book).
from torch.utils.data import IterableDataset

class Stream(IterableDataset):
    def __iter__(self):
        yield from clean(read(open("corpus.txt")))

# import tensorflow as tf
# ds = tf.data.Dataset.from_generator(
#     lambda: clean(read(open("corpus.txt"))),
#     output_types=tf.string,
# ).batch(32).prefetch(1)
```

Why it matters for AI: lazy pipelines are how you preprocess corpora larger than memory. Use generator expressions to avoid materialising intermediates, compose stages as generators, and use `yield from` to flatten shards, keeping peak memory to a single record plus a batch.

CHAPTER 15

Building a Streaming Data Pipeline End to End

This chapter combines the pieces into one small but realistic pipeline: a resource-safe reader, lazy cleaning and tokenizing stages, batching, and a live metric fed with `.send()`. Every stage is a generator, so the whole thing streams and stays flat in memory.

contextlib for resource-safe iteration

A data source usually owns a resource (a file, socket or DB cursor) that must close even if iteration stops early or raises. `contextlib.contextmanager` lets you write that open/yield/close guarantee as a generator, and the `finally` block always runs.

A GENERATOR-BASED CONTEXT MANAGER

```
from contextlib import contextmanager

@contextmanager
def open_shard(name, rows):
    print("open", name)
    try:
        yield iter(rows)          # hand the caller a lazy c ←
        cursor
    finally:
        print("close", name)      # runs even on early break ←
        / error

with open_shard("shard-0", [" A ", "b"]) as cur:
    for row in cur:
        print("row:", row.strip())
```

OUTPUT

```
open shard-0  
row: A  
row: b  
close shard-0
```

The full worked pipeline

Now we wire the stages together. `read` streams rows from the managed shard, `clean` and `tokenize` transform lazily, `batch` groups, and a `running_average` coroutine tracks mean tokens-per-batch as batches flow past, no list of the corpus ever exists.

READ TO METRIC, FULLY LAZY

```

from contextlib import contextmanager

@contextmanager
def shard(rows):
    try:
        yield iter(rows)
    finally:
        print("shard closed")

def clean(stream):
    for line in stream:
        s = line.strip().lower()
        if s:
            yield s
            # drop blank lines lazily

def tokenize(stream):
    for line in stream:
        yield line.split()

def batch(stream, size):
    buf = []
    for item in stream:
        buf.append(item)
        if len(buf) == size:
            yield buf; buf = []
    if buf:
        yield buf

def running_average():
    total, count, avg = 0, 0, None
    while True:
        x = yield avg
        total += x; count += 1
        avg = total / count

rows = [" Hello WORLD ", "", "Deep Learning here", " GPUs go ←
        BRRR "]
mean_tokens = running_average()
next(mean_tokens)
# prime the metric

with shard(rows) as src:
    pipeline = batch(tokenize(clean(src)), size=2)
    for i, b in enumerate(pipeline):
        n_tokens = sum(len(sent) for sent in b)
        avg = mean_tokens.send(n_tokens)
        print(f"batch {i}: {b} avg_tokens={avg:.1f}")

```


OUTPUT

```
batch 0: [['hello', 'world'], ['deep', 'learning', 'here']] avg_tokens=5.0  
batch 1: [['gpu', 'go', 'brrr']] avg_tokens=4.0  
shard closed
```

Why it matters for AI: real ingestion code looks exactly like this: resource-safe readers, composed lazy transforms, batching, and streaming metrics. Master the generator toolkit and you can preprocess corpora far larger than RAM while guaranteeing every file handle is released.

PART IV

Objects and Abstractions

CHAPTER 16

Classes, Dunder Methods and `__call__`: the `nn.Module` Pattern

A **class** bundles data (attributes) with behaviour (methods). Every neural-network layer you will ever use, `nn.Linear`, `nn.Conv2d`, a whole `nn.Module`, is just a class instance that holds parameters and knows how to transform an input.

`__init__` and instance vs class attributes

`__init__` runs when you build an instance. Attributes set on `self` are *per-instance*; attributes set on the class body are *shared* by every instance, handy for constants like a default learning rate.

INSTANCE VS CLASS ATTRIBUTES

```
class Neuron:
    activation = "relu"          # class attribute, shared

    def __init__(self, n_inputs):
        self.weights = [0.0] * n_inputs  # instance attribute

a, b = Neuron(3), Neuron(5)
print("a.weights:", a.weights)
print("b.weights:", b.weights)
print("shared activation:", a.activation, b.activation)
Neuron.activation = "gelu"      # change once, seen everywhere
print("after change:", a.activation, b.activation)
```

OUTPUT

```
a.weights: [0.0, 0.0, 0.0]
b.weights: [0.0, 0.0, 0.0, 0.0, 0.0]
shared activation: relu relu
after change: gelu gelu
```

`__repr__` makes objects printable

Dunder (double-underscore) methods hook into Python syntax. `__repr__` controls how an object shows up in the REPL and in logs, priceless when you print a model or a config.

A READABLE `__REPR__`

```
class Linear:
    def __init__(self, in_features, out_features):
        self.in_features = in_features
        self.out_features = out_features

    def __repr__(self):
        return f"Linear(in={self.in_features}, out={self.out_features})"

layer = Linear(768, 256)
print(layer)
print([Linear(4, 4), Linear(4, 1)])
```

OUTPUT

```
Linear(in=768, out=256)
[Linear(in=4, out=4), Linear(in=4, out=1)]
```

`__call__`: why `layer(x)` works

When an object defines `__call__`, you can invoke the instance like a function. This is exactly why `output = layer(x)` works in PyTorch instead of `layer.forward(x)`, the framework routes the call through `__call__`. Here is a pure-Python `Linear` layer that computes $y = xW + b$ and is *callable*.

A CALLABLE LINEAR LAYER (PURE PYTHON)

```

class Linear:
    def __init__(self, W, b):
        self.W = W          # out x in
        self.b = b          # length out

    def __call__(self, x):    # y = W @ x + b
        return [sum(w_i * x_j for w_i, x_j in zip(row, x)) + b_
                i
                for row, b_i in zip(self.W, self.b)]

layer = Linear(W=[[1.0, 0.0], [0.5, 0.5]], b=[0.1, -0.2])
x = [2.0, 4.0]
print("layer(x) =", layer(x))  # calls __call__, just like PyTorch

```

OUTPUT

```
layer(x) = [2.1, 2.8]
```

`__str__` vs `__repr__`: two audiences

`__repr__` is the *unambiguous* developer view (used in the REPL, logs and inside containers); `__str__` is the *friendly* user view (used by `print` and `str`). If you only define one, define `__repr__`, `str` falls back to it.

ONE OBJECT, TWO RENDERINGS

```

class Tensor:
    def __init__(self, shape):
        self.shape = shape
    def __repr__(self):
        return f"Tensor(shape={self.shape})"  # for devs/logs
    def __str__(self):
        dims = "x".join(map(str, self.shape))
        return f"<{dims} tensor>"  # for humans

t = Tensor([32, 768])
print(str(t))          # uses __str__
print(repr(t))         # uses __repr__
print([t])             # containers use __repr__

```

OUTPUT

```
<32x768 tensor>
Tensor(shape=[32, 768])
[Tensor(shape=[32, 768])]
```

`__eq__` and `__hash__` travel together

`__eq__` defines value equality; `__hash__` lets an object live in a `set` or `dict` key. If two objects are equal they *must* hash equal, so define both from the same fields. Beware: defining `__eq__` alone makes Python set `__hash__` to `None`, silently dropping hashability.

`__EQ__ PLUS __HASH__ FROM THE SAME FIELDS`

```
class Token:
    def __init__(self, text, idx):
        self.text, self.idx = text, idx
    def __repr__(self):
        return f"Token({self.text!r}, {self.idx})"
    def __eq__(self, other):
        return (isinstance(other, Token)
                and (self.text, self.idx)
                    == (other.text, other.idx))
    def __hash__(self):
        return hash((self.text, self.idx))

a, b = Token("cat", 5), Token("cat", 5)
print("equal?", a == b)
print("same hash?", hash(a) == hash(b))
print("set dedups:", len({a, b}))  # collapses to one
```

OUTPUT

```
equal? True
same hash? True
set dedups: 1
```

`__EQ__ WITHOUT __HASH__ DROPS HASHABILITY`

```

class NoHash:
    def __init__(self, v):
        self.v = v
    def __eq__(self, other):
        return self.v == other.v
    # no __hash__ defined -> Python sets __hash__ = None

try:
    {NoHash(1)}                                # now unhashable
except TypeError as e:
    print("TypeError:", e)

```

OUTPUT

```

TypeError: cannot use 'NoHash' as a set element (unhashable type:
e: 'NoHash')

```

`__len__` and `__getitem__`: make a Dataset

Defining `__len__` and `__getitem__` makes an object behave like a sequence: `len(obj)` works, `obj[i]` works, and, because `__getitem__` accepts integer indices from 0 upward, the object becomes *iterable* too. This is exactly the PyTorch `Dataset` protocol.

AN INDEXABLE, ITERABLE DATASET

```

class Dataset:
    def __init__(self, samples):
        self.samples = samples
    def __len__(self):
        return len(self.samples)
    def __getitem__(self, i):
        return self.samples[i]

ds = Dataset([("a", 0), ("b", 1), ("c", 0)])
print("len:", len(ds))
print("index:", ds[1])                # __getitem__
for text, label in ds:                # iterable for free
    print("sample:", text, label)

```

OUTPUT

```
len: 3
index: ('b', 1)
sample: a 0
sample: b 1
sample: c 0
```

`__slots__` saves memory on many objects

By default each instance carries a `__dict__` to hold attributes. Declaring `__slots__` replaces it with a fixed set of slots, cutting per-instance memory (useful when you create millions of tiny records) and blocking accidental new attributes.

`__SLOTS__` REMOVES THE PER-INSTANCE `__DICT__`

```
class Small:
    __slots__ = ("x", "y")          # fixed attributes, no __dict__
    def __init__(self, x, y):
        self.x, self.y = x, y

class Big:
    def __init__(self, x, y):
        self.x, self.y = x, y

s, b = Small(1, 2), Big(1, 2)
print("slots __dict__?", hasattr(s, "__dict__"))
print("plain __dict__?", hasattr(b, "__dict__"))
try:
    s.z = 3                        # cannot add new attrs
except AttributeError as e:
    print("AttributeError:", e)
```

OUTPUT

```
slots __dict__? False
plain __dict__? True
AttributeError: 'Small' object has no attribute 'z' and no __dict__
    for setting new attributes
```

classmethod and staticmethod

A `@classmethod` receives the class as `cls` and is the idiomatic

way to write *alternate constructors* like `from_config`. A `@static method` receives neither `self` nor `cls`, it is a plain utility that lives in the class namespace for organisation.

FROM_CONFIG CONSTRUCTOR AND A STATIC HELPER

```
class TrainConfig:
    def __init__(self, lr, batch_size):
        self.lr, self.batch_size = lr, batch_size
    def __repr__(self):
        return (f"TrainConfig(lr={self.lr}, "
                f"batch_size={self.batch_size})")

    @classmethod
    def from_config(cls, d):          # alternate constructor
        return cls(lr=d["lr"],
                    batch_size=d.get("batch_size", 32))

    @staticmethod
    def is_valid_lr(lr):              # no self/cls needed
        return 0 < lr < 1

cfg = TrainConfig.from_config({"lr": 1e-3})
print(cfg)
print("valid lr?", TrainConfig.is_valid_lr(1e-3))
```

OUTPUT

```
TrainConfig(lr=0.001, batch_size=32)
valid lr? True
```

Pitfall: a shared mutable class attribute

Because class attributes are shared, a *mutable* one (a list or dict) is shared across all instances. Mutating it through one instance changes it for every instance, a bug that hides for a long time. Keep per-instance state in `__init__`.

THE SHARED-MUTABLE-CLASS-ATTRIBUTE TRAP

```

class Logger:
    history = []                # shared mutable class attribute
    r!

    def log(self, msg):
        self.history.append(msg) # mutates the shared list

a, b = Logger(), Logger()
a.log("from a")
b.log("from b")
print("a sees:", a.history)    # both entries leak in
print("same list?", a.history is b.history)

```

OUTPUT

```

a sees: ['from a', 'from b']
same list? True

```

Why it matters for AI: the whole `model(x)` ergonomics of PyTorch and Keras rests on `__call__`. Understanding it demystifies why layers are 'called' and lets you write your own composable, callable components.

CHAPTER 17

Inheritance, `super()`, Abstract Base Classes and Properties

Inheritance lets a class reuse and specialise another. In practice you subclass a framework base (`nn.Module`, a scikit-learn `BaseEstimator`) and override one or two methods.

Subclassing and `super()`

`super().__init__(...)` runs the parent's setup so the child does not have to re-implement it, the same call you write at the top of every custom `nn.Module.__init__`.

SUPER() CALLS THE PARENT

```
class Layer:
    def __init__(self, name):
        self.name = name

    def forward(self, x):
        raise NotImplementedError

class ReLU(Layer):
    def __init__(self):
        super().__init__("relu")           # run parent setup
        self.calls = 0

    def forward(self, x):                   # override
        self.calls += 1
        return [max(0.0, v) for v in x]

r = ReLU()
print(r.name, "->", r.forward([-1.0, 2.0, -3.0]))
print("forward calls:", r.calls)
```

OUTPUT

```
relu -> [0.0, 2.0, 0.0]
forward calls: 1
```

Abstract base classes define a contract

`abc.ABC` plus `@abstractmethod` forces every subclass to implement required methods. Instantiating an incomplete subclass fails loudly, which is how frameworks guarantee your model actually has a `forward`.

`@ABSTRACTMETHOD` ENFORCES THE INTERFACE

```
from abc import ABC, abstractmethod

class Model(ABC):
    @abstractmethod
    def forward(self, x): ...

class Identity(Model):
    def forward(self, x):
        return x

print("Identity works:", Identity().forward(42))
try:
    Model()                                # abstract, cannot instantiate
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
Identity works: 42
TypeError: Can't instantiate abstract class Model without an im
    plementation for abstract method 'forward'
```

`@property` for computed attributes

A `@property` looks like an attribute but runs code on access, perfect for derived values such as a model's parameter count, which should never fall out of sync with the weights.

`NUM_PARAMETERS` AS A PROPERTY

```
class Linear:
    def __init__(self, in_f, out_f):
        self.in_f, self.out_f = in_f, out_f

    @property
    def num_parameters(self):
        return self.in_f * self.out_f + self.out_f  # weights
            + bias

layer = Linear(768, 256)
print("params:", layer.num_parameters)  # accessed like an attribute
```

OUTPUT

```
params: 196864
```

An unfinished subclass still fails

Abstract methods are checked at *instantiation*: even a concrete-looking subclass cannot be created until it implements every `@abstractmethod`. This catches a half-written model long before the training loop runs.

MISSING METHOD BLOCKS INSTANTIATION

```
from abc import ABC, abstractmethod

class Model(ABC):
    @abstractmethod
    def forward(self, x): ...
    @abstractmethod
    def parameters(self): ...

class Half(Model):
    def forward(self, x):
        return x
    # forgets parameters()

try:
    Half()
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
TypeError: Can't instantiate abstract class Half without an implementation for abstract method 'parameters'
```

@property with a validating setter

Pair a `@property` getter with a matching `@x.setter` to run validation on every assignment. Storing the real value in a 'private' `_lr` lets the public `lr` reject out-of-range hyper-parameters the moment they are set.

A SETTER THAT VALIDATES LR IN [0, 1]

```
class Optimizer:
    def __init__(self, lr):
        self._lr = 0.0
        self.lr = lr                # goes through the setter

    @property
    def lr(self):
        return self._lr

    @lr.setter
    def lr(self, value):
        if not 0 <= value <= 1:
            raise ValueError(f"lr must be in [0, 1], got {value}")
        self._lr = value

opt = Optimizer(0.01)
print("lr:", opt.lr)
opt.lr = 0.5
print("updated lr:", opt.lr)
try:
    opt.lr = 5.0
except ValueError as e:
    print("rejected:", e)
```

OUTPUT

```
lr: 0.01
updated lr: 0.5
rejected: lr must be in [0, 1], got 5.0
```

Multiple inheritance, mixins and the MRO

A class can inherit from several parents. *Mixins*, small classes that add one capability, are a common use: combine a `SerializeMixin` and a base `Module` to compose behaviour. Python resolves which method wins via the **MRO** (method resolution order), visible as `__mro__`.

MIXINS AND `__MRO__`

```
class Module:
    def forward(self, x):
        return x

class SerializeMixin:
    def save(self):
        return f"saved {type(self).__name__}"

class CountMixin:
    def n_params(self):
        return 0

class Net(SerializeMixin, CountMixin, Module):
    pass

net = Net()
print("forward:", net.forward(9))
print("save:", net.save())
print("mro:", [c.__name__ for c in Net.__mro__])
```

OUTPUT

```
forward: 9
save: saved Net
mro: ['Net', 'SerializeMixin', 'CountMixin', 'Module', 'object'
]
```

Why it matters for AI: every custom model you write subclasses a base and calls `super().__init__()`. ABCs catch missing methods before training starts, and properties keep derived stats (param counts, output shapes) always correct.

CHAPTER 18

Dataclasses for Configs and Records

Training runs are drowning in configuration: learning rate, batch size, number of layers. A `@dataclass` turns a plain class into a typed record with a free `__init__`, `__repr__` and equality, ideal for hyper-parameter configs.

A typed config with defaults

@DATACLASS WITH DEFAULTS

```
from dataclasses import dataclass

@dataclass
class TrainConfig:
    lr: float = 3e-4
    batch_size: int = 32
    epochs: int = 10

cfg = TrainConfig(lr=1e-3)
print(cfg)                    # auto __repr__
print("batch_size:", cfg.batch_size)
print("equal?", cfg == TrainConfig(lr=1e-3))
```

OUTPUT

```
TrainConfig(lr=0.001, batch_size=32, epochs=10)
batch_size: 32
equal? True
```

Mutable defaults need `default_factory`

A list or dict default must be created per instance with `field(default_factory=...)`, otherwise every config would *share* the same list, a classic bug.

FIELD(DEFAULT_FACTORY=LIST)


```

from dataclasses import dataclass, field, asdict

@dataclass
class ModelConfig:
    hidden_sizes: list = field(default_factory=lambda: [256, 128])
    dropout: float = 0.1

cfg = ModelConfig()
cfg.hidden_sizes.append(64)
print(cfg)
print("as dict:", asdict(cfg))    # easy to serialise to JSON

```

OUTPUT

```

ModelConfig(hidden_sizes=[256, 128, 64], dropout=0.1)
as dict: {'hidden_sizes': [256, 128, 64], 'dropout': 0.1}

```

frozen=True and __post_init__ validation

frozen=True makes a config immutable (hashable, safe to share), and **__post_init__** runs validation right after construction so bad hyper-parameters fail immediately.

IMMUTABLE, SELF-VALIDATING CONFIG

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Config:
    lr: float
    def __post_init__(self):
        if not 0 < self.lr < 1:
            raise ValueError(f"lr out of range: {self.lr}")

print(Config(lr=0.01))
try:
    Config(lr=5.0)
except ValueError as e:
    print("rejected:", e)
try:
    Config(lr=0.01).lr = 0.02    # frozen, cannot mutate
except Exception as e:
    print(type(e).__name__, "on assignment")

```

OUTPUT

```

Config(lr=0.01)
rejected: lr out of range: 5.0
FrozenInstanceError on assignment

```

order=True and per-field field() options

`order=True` generates comparison methods so instances sort. Per field you can opt out of behaviours: `compare=False` excludes a field from equality/ordering, and `repr=False` hides it from the printout. Here checkpoints sort purely by score.

SORTABLE CHECKPOINTS WITH FIELD() TUNING

```

from dataclasses import dataclass, field

@dataclass(order=True)
class Checkpoint:
    score: float
    path: str = field(compare=False)      # ignored when sorting
    notes: str = field(default="", repr=False)

a = Checkpoint(0.91, "a.pt")
b = Checkpoint(0.95, "b.pt", notes="best so far")
print("b better?", b > a)                # compares score only
print("best:", max([a, b]))              # notes hidden from repr

```

OUTPUT

```

b better? True
best: Checkpoint(score=0.95, path='b.pt')

```

asdict/astuple and slots=True

`asdict` and `astuple` recursively convert a dataclass for serialisation or unpacking. Passing `slots=True` (Python 3.10+) builds the class with `__slots__`, saving memory when you hold many records, exactly like the manual `__slots__` earlier.

ASDICT, ASTUPLE AND SLOTS=TRUE

```

from dataclasses import dataclass, asdict, astuple

@dataclass(slots=True)
class Point:
    x: float
    y: float

p = Point(1.0, 2.0)
print("as dict:", asdict(p))
print("as tuple:", astuple(p))
print("has __dict__?", hasattr(p, "__dict__")) # False

```

OUTPUT

```
as dict: {'x': 1.0, 'y': 2.0}  
as tuple: (1.0, 2.0)  
has __dict__? False
```

Why it matters for AI: Hugging Face [TrainingArguments](#) and most research configs are dataclasses. They give you typed, self-documenting, serialisable hyper-parameters with almost no boilerplate, and validation catches typos before a GPU-hour is wasted.

CHAPTER 19

Enums and NamedTuples

Magic strings ('`cuda`', '`adam`') and bare tuples are easy to mistype and hard to read. Enums and named tuples give those values names and structure.

Enum for fixed choices

An `enum.Enum` defines a closed set of options, a device or an optimizer, so a typo becomes an error instead of a silent wrong run.

ENUM.ENUM FOR DEVICE SELECTION

```
from enum import Enum

class Device(Enum):
    CPU = "cpu"
    CUDA = "cuda"
    MPS = "mps"

def to(device: Device):
    print("moving tensors to", device.value)

to(Device.CUDA)
print("members:", [d.name for d in Device])
print("lookup:", Device("cpu"))
```

OUTPUT

```
moving tensors to cuda
members: ['CPU', 'CUDA', 'MPS']
lookup: Device.CPU
```

NamedTuple for lightweight records

A `typing.NamedTuple` is an immutable record with named fields. It is perfect for a training `Batch`: you get `batch.inputs` instead of `batch[0]`, but it still unpacks and behaves like a tuple.

TYPING.NAMEDTUPLE AS A BATCH

```

from typing import NamedTuple

class Batch(NamedTuple):
    inputs: list
    labels: list
    weight: float = 1.0

b = Batch(inputs=[1, 2, 3], labels=[0, 1, 1])
print("named access:", b.inputs, b.labels, b.weight)
x, y, w = b                                # still unpacks like a tuple
print("unpacked:", x, y, w)
print("as dict:", b._asdict())

```

OUTPUT

```

named access: [1, 2, 3] [0, 1, 1] 1.0
unpacked: [1, 2, 3] [0, 1, 1] 1.0
as dict: {'inputs': [1, 2, 3], 'labels': [0, 1, 1], 'weight': 1.0}

```

auto() and IntEnum

`auto()` fills in values so you do not repeat yourself. An `IntEnum` is an `int`, so members compare and arithmetic like numbers, ideal for ordered levels such as logging verbosity where `INFO` `>=` `DEBUG` should just work.

AUTO() VALUES AND COMPARABLE INTENUM

```

from enum import Enum, IntEnum, auto

class Stage(Enum):
    TRAIN = auto()
    VALID = auto()
    TEST = auto()

class LogLevel(IntEnum):
    DEBUG = 10
    INFO = 20
    WARNING = 30

print("auto values:", [s.value for s in Stage])
print("INFO >= DEBUG?", LogLevel.INFO >= LogLevel.DEBUG)
print("as int:", LogLevel.WARNING + 1)  # behaves like int

```

OUTPUT

```

auto values: [1, 2, 3]
INFO >= DEBUG? True
as int: 31

```

Flag for combinable options

A **Flag** (or **IntFlag**) lets members be combined with `|` and tested with `in`, the natural fit for a set of toggles such as an augmentation pipeline. One value carries several independent choices.

FLAG: OR-COMBINED AUGMENTATIONS

```

from enum import Flag, auto

class Augment(Flag):
    NONE = 0
    FLIP = auto()
    ROTATE = auto()
    CROP = auto()

pipeline = Augment.FLIP | Augment.CROP
print("pipeline:", pipeline)
print("has FLIP?", Augment.FLIP in pipeline)
print("has ROTATE?", Augment.ROTATE in pipeline)

```

OUTPUT

```
pipeline: Augment.FLIP|CROP
has FLIP? True
has ROTATE? False
```

NamedTuple class syntax with methods

The class form of `NamedTuple` adds type hints and even methods while staying an immutable tuple. `_replace` returns a modified copy (never mutating the original) and `_asdict` gives a plain dict.

A NAMEDTUPLE WITH A METHOD AND _REPLACE

```
from typing import NamedTuple

class Point(NamedTuple):
    x: float
    y: float
    def norm(self) -> float:
        return (self.x ** 2 + self.y ** 2) ** 0.5

p = Point(3.0, 4.0)
print("norm:", p.norm())
q = p._replace(y=0.0)          # new immutable copy
print("replaced:", q)
print("original intact:", p)
print("as dict:", p._asdict())
```

OUTPUT

```
norm: 5.0
replaced: Point(x=3.0, y=0.0)
original intact: Point(x=3.0, y=4.0)
as dict: {'x': 3.0, 'y': 4.0}
```

Why it matters for AI: dataloaders, RL transitions and model outputs are often named tuples (PyTorch returns them from many ops). Enums prevent `'cuda'/'Cuda'` typos and make valid options discoverable in your editor.

CHAPTER 20

Decorators: Timing, Caching and no_grad-style Wrappers

A **decorator** wraps a function to add behaviour without touching its body. You have already seen `@dataclass` and `@property`; the AI ecosystem is full of them: `@torch.no_grad()`, `@tf.function`, `@functools.lru_cache`.

A timing decorator with `functools.wraps`

`functools.wraps` copies the original function's name and docstring onto the wrapper so tracebacks and `help()` stay meaningful.

@TIMEIT MEASURES A FUNCTION

```
import time, functools

def timeit(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        out = fn(*args, **kwargs)
        dt = time.perf_counter() - t0
        print(f"{fn.__name__} took {dt*1e3:.2f} ms")
        return out
    return wrapper

@timeit
def matmul_size(n):
    return sum(i * i for i in range(n))

print("result:", matmul_size(100_000))
print("name preserved:", matmul_size.__name__)
```

OUTPUT

```
matmul_size took 3.89 ms
result: 333328333350000
name preserved: matmul_size
```

Caching repeated work

`functools.lru_cache` memoises results, turning an expensive recomputation into a dictionary lookup, useful for tokeniser lookups or pure feature functions.

@LRU_CACHE MEMOISES

```
from functools import lru_cache

calls = {"n": 0}

@lru_cache(maxsize=None)
def fib(n):
    calls["n"] += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print("fib(30) =", fib(30))
print("unique calls:", calls["n"])    # linear, not exponential
print(fib.cache_info())
```

OUTPUT

```
fib(30) = 832040
unique calls: 31
CacheInfo(hits=28, misses=31, maxsize=None, currsize=31)
```

A decorator that takes arguments

Add a layer of nesting and a decorator can accept parameters, for example a `@retry(n)` that re-runs a flaky call, exactly what you want around an unreliable model-serving API.

@RETRY(N) WITH AN ARGUMENT

```

import functools

def retry(times):
    def deco(fn):
        @functools.wraps(fn)
        def wrapper(*a, **k):
            for attempt in range(1, times + 1):
                try:
                    return fn(*a, **k)
                except Exception as e:
                    print(f"attempt {attempt} failed: {e}")
                    raise RuntimeError("all retries exhausted")
            return wrapper
        return deco

state = {"n": 0}

@retry(3)
def flaky():
    state["n"] += 1
    if state["n"] < 3:
        raise ValueError("temporary")
    return "ok"

print("result:", flaky())

```

OUTPUT

```

attempt 1 failed: temporary
attempt 2 failed: temporary
result: ok

```

In real code you rarely write these, you *apply* them. The snippet below is **illustrative** (PyTorch is not installed): `@torch.no_grad()` wraps a function so it runs without building the autograd graph, saving memory during evaluation.

FRAMEWORK DECORATORS (ILLUSTRATIVE)

```
import torch

@torch.no_grad()                # disables gradient tracking for
    the call
def evaluate(model, loader):
    model.eval()
    return sum(accuracy(model(x), y) for x, y in loader)

@torch.compile                  # JIT-optimize the function/module
def forward(x):
    return model(x)
```

Stacking decorators, order matters

You can apply several decorators to one function. They wrap *bottom-up*: the one nearest the `def` runs first (innermost), the top one wraps last (outermost). Reading them top-to-bottom tells you the order the wrappers execute around the call.

TWO DECORATORS WRAP BOTTOM-UP

```
import functools

def bold(fn):
    @functools.wraps(fn)
    def w(*a, **k):
        return "<b>" + fn(*a, **k) + "</b>"
    return w

def emph(fn):
    @functools.wraps(fn)
    def w(*a, **k):
        return "<i>" + fn(*a, **k) + "</i>"
    return w

@bold                # applied second -> outermost
@emph                # applied first -> innermost
def greet(name):
    return f"hi {name}"

print(greet("ada"))  # <b><i>hi ada</i></b>
```

OUTPUT

```
<b><i>hi ada</i></b>
```

A class-based decorator

A class whose instances are callable (via `__call__`) can also be a decorator, handy when the wrapper needs to hold state such as a call counter. `functools.update_wrapper` copies the wrapped function's metadata, the class-based twin of `wraps`.

COUNTING CALLS WITH A CLASS DECORATOR

```
import functools

class CountCalls:
    def __init__(self, fn):
        functools.update_wrapper(self, fn)
        self.fn = fn
        self.count = 0
    def __call__(self, *a, **k):
        self.count += 1
        return self.fn(*a, **k)

@CountCalls
def step():
    return "stepped"

step(); step(); step()
print("name:", step.__name__)      # preserved by update_wrapper
r
print("calls:", step.count)
```

OUTPUT

```
name: step
calls: 3
```

Why it matters for AI: decorators are the vocabulary of modern frameworks. Recognising `@no_grad`, `@lru_cache`, `@compile` and `@jit` lets you read library code, and writing your own `@timeit`/`@retry` cleans up training and serving scripts.

CHAPTER 21

Context Managers and the with Statement

A **context manager** guarantees setup and teardown around a block, even if it raises. Files are the classic case, but in AI the pattern is everywhere: `with torch.no_grad():`, `autocast`, distributed scopes, timers.

with for files closes them for you

WITH ALWAYS CLOSES THE FILE

```
import tempfile, os

path = os.path.join(tempfile.gettempdir(), "metrics.txt")
with open(path, "w") as f:
    f.write("epoch,loss\n0,0.91\n1,0.42\n")
# file is guaranteed closed here, even on error

with open(path) as f:
    print(f.read().strip())
os.remove(path)
```

OUTPUT

```
epoch,loss
0,0.91
1,0.42
```

Writing your own with __enter__/_exit__

Any class with `__enter__` and `__exit__` is a context manager. A block timer is a tidy example, `__exit__` runs whether or not the body succeeded.

A TIMER CONTEXT MANAGER

```
import time

class Timer:
    def __init__(self, label):
        self.label = label
    def __enter__(self):
        self.t0 = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc, tb):
        dt = time.perf_counter() - self.t0
        print(f"{self.label}: {dt*1e3:.2f} ms")
        return False # do not suppress exceptions

with Timer("training step"):
    total = sum(i * i for i in range(200_000))
print("total:", total)
```

OUTPUT

```
training step: 7.78 ms
total: 2666646666700000
```

contextlib.contextmanager: the easy way

The `@contextmanager` decorator lets you write a context manager as a generator: code before `yield` is setup, code after is teardown. Here we temporarily set and restore a random seed, a common trick for reproducible sampling.

@CONTEXTMANAGER FOR A TEMPORARY SEED

```

import random
from contextlib import contextmanager

@contextmanager
def temporary_seed(seed):
    state = random.getstate()      # setup: save
    random.seed(seed)
    try:
        yield
    finally:
        random.setstate(state)     # teardown: restore

with temporary_seed(0):
    print("seeded:", [random.randint(0, 9) for _ in range(4)])
with temporary_seed(0):
    print("repeat:", [random.randint(0, 9) for _ in range(4)])

```

OUTPUT

```

seeded: [6, 6, 0, 4]
repeat: [6, 6, 0, 4]

```

The same protocol powers the most common inference idiom. This snippet is **illustrative** (PyTorch not installed): the `with` block disables gradient tracking just for its body, then restores it.

WITH TORCH.NO_GRAD() (ILLUSTRATIVE)

```

import torch

with torch.no_grad():              # __enter__ turns grad off
    logits = model(inputs)         # no autograd graph built -> ↩
    less memory
    preds = logits.argmax(dim=-1)
# __exit__ restores gradient tracking here

```

`__exit__` can handle an exception

`__exit__` receives the exception (type, value, traceback) if the block raised. Returning a truthy value *suppresses* it, so execution continues after the `with`. Returning `False` (or `None`) lets it propagate, as the `Timer` above did.

A CONTEXT MANAGER THAT SWALLOWS ERRORS

```

class ignore_errors:
    def __enter__(self):
        return self
    def __exit__(self, exc_type, exc, tb):
        if exc_type is not None:
            print("suppressed:", exc_type.__name__)
            return True                # truthy -> swallow it

with ignore_errors():
    raise ValueError("boom")          # handled, not raised
print("execution continues")

```

OUTPUT

```

suppressed: ValueError
execution continues

```

contextlib.suppress for expected errors

`contextlib.suppress` is a ready-made context manager that silently ignores the exception types you name, cleaner than an empty `except`. Great for best-effort cleanup like deleting a checkpoint that may not exist.

SUPPRESS A SPECIFIC EXCEPTION

```

import os
from contextlib import suppress

with suppress(FileNotFoundError):
    os.remove("does_not_exist.ckpt")  # missing? no problem
print("cleanup done")

```

OUTPUT

```

cleanup done

```

ExitStack for a dynamic set of resources

When the number of resources is only known at runtime, `contextlib.ExitStack` manages them all and unwinds them in reverse on exit, so you never hand-write nested `with` statements for a

variable list of open shards.

OPEN MANY FILES WITH EXITSTACK

```
import os, tempfile
from contextlib import ExitStack

tmp = tempfile.gettempdir()
paths = [os.path.join(tmp, f"shard_{i}.txt") for i in range(3)]
for p in paths:
    open(p, "w").close()

with ExitStack() as stack:
    files = [stack.enter_context(open(p)) for p in paths]
    print("open files:", len(files))
# every file closed here, in reverse order
for p in paths:
    os.remove(p)
print("all closed and removed")
```

OUTPUT

```
open files: 3
all closed and removed
```

Why it matters for AI: `with torch.no_grad()`, `with autocast()` and `with device:` are all this one protocol. Knowing it means you can write reproducible-seed, timing and resource-cleanup blocks that never leak state.

PART V

Typing and Robustness

CHAPTER 22

Type Hints and the typing Module

Type hints annotate what a function expects and returns. Python does **not** enforce them at runtime, they are for humans, editors and tools like `mypy`, but modern ML libraries lean on them heavily for autocomplete and clarity.

Function and variable annotations

Since Python 3.9 you write built-in generics directly: `list[int]`, `dict[str, float]`, `tuple[int, int]`. They document shapes-of-data without any import.

ANNOTATIONS ARE DOCUMENTATION

```
def normalise(xs: list[float]) -> list[float]:
    total: float = sum(xs)
    return [x / total for x in xs]

probs = normalise([2.0, 3.0, 5.0])
print("probs:", probs, "sum:", sum(probs))
print("annotations:", normalise.__annotations__)
```

OUTPUT

```
probs: [0.2, 0.3, 0.5] sum: 1.0
annotations: {'xs': list[float], 'return': list[float]}
```

Optional, Union and the | syntax

`X | None` (equivalently `Optional[X]`) marks a value that may be missing, a device that defaults to auto-detect, say. `A | B` means 'either type'.

OPTIONAL PARAMETERS WITH | NONE

```
def build_model(hidden: int, device: str | None = None) -> dict:
    :
    if device is None:
        device = "cpu"          # auto-detect fallback
    return {"hidden": hidden, "device": device}

print(build_model(256))
print(build_model(256, device="cuda"))
```

OUTPUT

```
{'hidden': 256, 'device': 'cpu'}
{'hidden': 256, 'device': 'cuda'}
```

Callable and simple generics

Callable types a function argument (an activation or a metric passed in), and a **TypeVar** writes a function that works for *any* type while keeping the relationship between input and output.

CALLABLE AND TYPEVAR

```
from typing import Callable, TypeVar

def apply(fn: Callable[[float], float], xs: list[float]) -> list[
    float]:
    return [fn(x) for x in xs]

relu = lambda x: max(0.0, x)
print("relu:", apply(relu, [-1.0, 0.5, 2.0]))

T = TypeVar("T")
def first(seq: list[T]) -> T:
    return seq[0]

print("first int:", first([10, 20]))
print("first str:", first(["a", "b"]))
```

OUTPUT

```
relu: [0.0, 0.5, 2.0]
first int: 10
first str: a
```

Hints are not enforced at runtime

This surprises newcomers: passing the wrong type still runs. Use hints for tooling, and add explicit checks (next chapter) when you truly need runtime guarantees.

THE HINT IS A LIE THE RUNTIME IGNORES

```
from typing import get_type_hints

def scale(x: int) -> int:
    return x * 3

print("wrong type still runs:", scale("ab"))    # 'ababab'
print("declared hints:", get_type_hints(scale))
```

OUTPUT

```
wrong type still runs: ababab
declared hints: {'x': <class 'int'>, 'return': <class 'int'>}
```

Literal for a fixed set of choices

`Literal` pins a parameter to specific values, a device string, a dtype, a reduction mode. A checker rejects anything outside the set, which catches the classic `device="gpu"` typo before it wastes a run.

CONSTRAIN A PARAM WITH LITERAL

```
from typing import Literal

Device = Literal["cpu", "cuda", "mps"]

def to_device(name: str, device: Device = "cpu") -> str:
    return f"{name} -> {device}"

print(to_device("model", "cuda"))
# A wrong literal still runs, but a type checker flags it:
print(to_device("model", "tpu"))    # mypy: not a valid Device
```

OUTPUT

```
model -> cuda
model -> tpu
```

Type aliases (classic and the 3.12 type statement)

A type alias gives a meaningful name to a repeated shape. The classic form is a plain assignment; Python 3.12+ adds a `type` statement that creates a first-class, lazily-evaluated alias object. Keep the classic form for readers on 3.10/3.11.

NAMING SHAPES WITH ALIASES

```
from typing import TypeAlias

Vector: TypeAlias = list[float]    # classic alias, works on 3.10+

def dot(a: Vector, b: Vector) -> float:
    return sum(x * y for x, y in zip(a, b))

# Python 3.12+ 'type' statement makes a real alias object:
type Matrix = list[list[float]]

print("dot:", dot([1.0, 2.0], [3.0, 4.0]))
print("alias object:", Matrix, "->", Matrix.__value__)
```

OUTPUT

```
dot: 11.0
alias object: Matrix -> list[list[float]]
```

Final, Any vs object, and cast

`Final` marks a constant a checker should stop you reassigning. `Any` opts out of type checking entirely, while `object` accepts any value yet keeps you honest, you must narrow it before using type-specific methods. `cast` tells the checker 'trust me, it is this type' and does nothing at runtime.

FINAL, ANY, OBJECT AND CAST

```

from typing import Final, Any, cast

MAX_TOKENS: Final = 512          # reassigning is flagged by checker
print("MAX_TOKENS:", MAX_TOKENS)

def describe(x: object) -> str:   # accepts anything, stays strict
    return f"{type(x).__name__}={x!r}"

raw: Any = "42"                  # Any: checker asks no questions
n = cast(int, int(raw))          # cast: pure annotation, no-op at run
print(describe(raw), "| cast ->", n + 1)

```

OUTPUT

```

MAX_TOKENS: 512
str='42' | cast -> 43

```

Reading annotations at runtime

`__annotations__` holds the raw annotations (possibly as strings), while `get_type_hints` resolves them into real type objects. Config systems and serialisers use this to build themselves from a class definition.

INTROSPECTING A CONFIG CLASS

```

from typing import get_type_hints

class Config:
    lr: float = 0.001
    steps: int = 1000

print("raw    :", Config.__annotations__)
print("hints :", get_type_hints(Config))

```

OUTPUT

```

raw    : {'lr': <class 'float'>, 'steps': <class 'int'>}
hints  : {'lr': <class 'float'>, 'steps': <class 'int'>}

```


Overloading with @overload

`@overload` declares several typed signatures for one function so a checker knows the return type depends on the argument type. Only the final, unannotated implementation runs; the stub bodies (...) are erased.

PRECISE RETURN TYPES VIA OVERLOAD

```
from typing import overload

@overload
def encode(x: str) -> list[int]: ...
@overload
def encode(x: list[str]) -> list[list[int]]: ...

def encode(x):
    # the single real implementation
    if isinstance(x, str):
        return [ord(c) for c in x]
    return [[ord(c) for c in s] for s in x]

print("str ->", encode("hi"))
print("list ->", encode(["hi", "yo"]))
```

OUTPUT

```
str -> [104, 105]
list -> [[104, 105], [121, 111]]
```

Why it matters for AI: framework APIs (PyTorch, Hugging Face, Pydantic-based configs) are richly typed, so hints power your editor's autocomplete and catch shape/argument mistakes before you launch an expensive run. Combine them with dataclasses for self-documenting configs.

CHAPTER 23

Protocols and Structural Typing (Duck Typing, Checked)

Python has always been duck-typed: if it has a `.forward()`, treat it as a model. `typing.Protocol` makes that idea *checked* without inheritance, you describe the methods an object must have, and any object that has them structurally satisfies the protocol. This is how you type 'anything with a `.forward()` or `.encode()` method'.

Defining a Protocol

A `Protocol` lists method signatures with empty `(...)` bodies. A class does **not** need to subclass it, it just needs matching methods. Checkers verify the fit; nothing changes at runtime.

AN INTERFACE WITHOUT INHERITANCE

```
from typing import Protocol

class SupportsForward(Protocol):
    def forward(self, x: float) -> float: ...

class Linear:                                # note: does NOT inherit anything
    def __init__(self, w: float) -> None:
        self.w = w
    def forward(self, x: float) -> float:
        return self.w * x

def run(model: SupportsForward, x: float) -> float:
    return model.forward(x)    # any object with .forward fits

print("out:", run(Linear(2.0), 3.0))
```

OUTPUT

```
out: 6.0
```

runtime_checkable and isinstance

Add `@runtime_checkable` and you can use `isinstance` against the protocol. The check only verifies that the method *names* exist, not their signatures, so treat it as a cheap smoke test, not a guarantee.

ISINSTANCE AGAINST A PROTOCOL

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Tokenizer(Protocol):
    def encode(self, text: str) -> list[int]: ...

class CharTokenizer:
    def encode(self, text: str) -> list[int]:
        return [ord(c) for c in text]

tok = CharTokenizer()
print("is Tokenizer :", isinstance(tok, Tokenizer))
print("encoded      :", tok.encode("hi"))
print("int is Tokenizer:", isinstance(42, Tokenizer))
```

OUTPUT

```
is Tokenizer : True
encoded      : [104, 105]
int is Tokenizer: False
```

Protocols vs abstract base classes

An ABC uses *nominal* typing: a class opts in by subclassing and cannot be instantiated until it implements every abstract method. A Protocol uses *structural* typing: no subclassing, the shape is enough. Use ABCs for a base you control and want to share code from; use Protocols to accept third-party objects you do not own.

- ABC: explicit inheritance, enforced at instantiation.
- Protocol: implicit fit, enforced by the type checker.
- Both let one function accept many interchangeable types.

AN ABC ENFORCES IMPLEMENTATION

```

from abc import ABC, abstractmethod

class Loss(ABC):
    @abstractmethod
    def __call__(self, pred: float, target: float) -> float: ...

class MSE(Loss):
    def __call__(self, pred: float, target: float) -> float:
        return (pred - target) ** 2

print("mse:", MSE()(3.0, 1.0))
try:
    Loss() # abstract: cannot instantiate
except TypeError as e:
    print("abstract:", e)

```

OUTPUT

```

mse: 4.0
abstract: Can't instantiate abstract class Loss without an impl ←
          ementation for abstract method '__call__'

```

Why it matters for AI: model zoos, tokenizers and datasets come from many libraries that share no base class. Protocols let you write `def train(model: SupportsForward, ...)` and accept a PyTorch module, a wrapper, or a mock in tests, anything with the right methods, without forcing everyone into one hierarchy.

CHAPTER 24

TypedDict and Typed Configs / Messages

LLM tooling passes dictionaries everywhere: chat messages, JSON configs, API payloads. `typing.TypedDict` types the *keys and value types* of a plain `dict`, so your editor autocompletes `msg["role"]` and a checker catches a misspelled key, while at runtime it stays an ordinary dict.

Typing a chat message

A chat turn is `{"role": ..., "content": ...}`. A `TypedDict` names those keys once and reuses them across the whole conversation list.

TYPED CHAT MESSAGES

```
from typing import TypedDict

class Message(TypedDict):
    role: str
    content: str

chat: list[Message] = [
    {"role": "system", "content": "You are helpful."},
    {"role": "user", "content": "Hello!"},
]
for m in chat:
    # still a plain dict at runtime
    print(f"{m['role']}>6}: {m['content']}")
```

OUTPUT

```
system: You are helpful.
user: Hello!
```

Required, NotRequired and total=False

By default every key is required. `NotRequired` (3.11+) marks an optional key while keeping the rest mandatory, and `total=False`

`e` flips the default so *all* keys become optional, handy for a config 'patch' that overrides a few fields.

OPTIONAL KEYS IN CONFIGS

```
from typing import TypedDict, NotRequired

class TrainConfig(TypedDict):
    lr: float          # required
    epochs: int         # required
    seed: NotRequired[int] # may be omitted

cfg: TrainConfig = {"lr": 1e-3, "epochs": 10}
full: TrainConfig = {"lr": 1e-3, "epochs": 10, "seed": 42}
print("cfg :", cfg)
print("full:", full)

class Patch(TypedDict, total=False):
    lr: float
    epochs: int

patch: Patch = {"lr": 5e-4}    # any subset is allowed
print("patch:", patch)
```

OUTPUT

```
cfg : {'lr': 0.001, 'epochs': 10}
full: {'lr': 0.001, 'epochs': 10, 'seed': 42}
patch: {'lr': 0.0005}
```

TypedDict vs dataclass vs Pydantic

A `TypedDict` is the right tool when the data *must* stay a dict (it came from JSON, or an API expects one). A `@dataclass` (from Part IV) is better when you want a real object with methods, defaults and dot access. Neither validates at runtime, a `TypedDict` will happily hold a wrong-typed value.

- `TypedDict`: dict in, dict out; zero runtime cost; no validation.
- `dataclass`: object with attributes, defaults and methods.
- `Pydantic` (third-party): parses and *validates* at runtime, the de-facto choice for LLM/API schemas, shown here for context only.

PYDANTIC VALIDATES AT RUNTIME

```
# Illustrative only: Pydantic is a third-party library.  
from pydantic import BaseModel
```

```
class Message(BaseModel):  
    role: str  
    content: str
```

```
# Unlike TypedDict, this RAISES on bad data at runtime:  
Message(role="user", content=123)  # ValidationError
```

Why it matters for AI: the OpenAI/Anthropic chat format is a list of typed dicts, and training configs are nested JSON. TypedDict documents those shapes for free and turns silent key typos into editor warnings; when you need real validation of untrusted input, reach for Pydantic on top.

CHAPTER 25

Errors, Exceptions and Custom Exception Types

AI code fails in predictable ways: mismatched tensor shapes, missing files, out-of-memory. Handling exceptions well means your training loop can log, recover, or fail with a message that actually tells you what went wrong.

try / except / else / finally

`else` runs only if no exception fired; `finally` always runs, ideal for releasing a resource such as a GPU handle or a file.

THE FULL TRY STRUCTURE

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("cannot divide by zero")
        return None
    else:
        print("division ok")
        return result
    finally:
        print("cleanup always runs")

print("=>", safe_divide(10, 2))
print("=>", safe_divide(10, 0))
```

OUTPUT

```
division ok
cleanup always runs
=> 5.0
cannot divide by zero
cleanup always runs
=> None
```


Catch specific exceptions, not everything

Catch the narrowest exception you can handle. A bare `except:` hides real bugs; naming the type keeps unexpected errors loud.

HANDLE ONLY WHAT YOU EXPECT

```
config = {"lr": 0.001}
for key in ["lr", "batch_size"]:
    try:
        print(f"{key} = {config[key]}")
    except KeyError:
        print(f"{key} missing, using default")
```

OUTPUT

```
lr = 0.001
batch_size missing, using default
```

Custom exceptions for domain errors

Defining your own exception type makes failures self-describing and lets callers catch exactly your error. A shape-mismatch check is the canonical example.

A SHAPEMISMATCHERROR

```
class ShapeMismatchError(ValueError):
    """Raised when two tensor shapes are incompatible."""

def matmul_shapes(a, b):
    if a[1] != b[0]:
        raise ShapeMismatchError(f"cannot matmul {a} with {b}")
    return (a[0], b[1])

print("ok:", matmul_shapes((4, 8), (8, 2)))
try:
    matmul_shapes((4, 8), (16, 2))
except ShapeMismatchError as e:
    print("caught:", e)
```

OUTPUT

```
ok: (4, 2)
caught: cannot matmul (4, 8) with (16, 2)
```

Exception chaining with `raise ... from`

`raise NewError(...) from original` preserves the underlying cause, so a friendly, high-level message keeps the low-level traceback for debugging.

WRAP A LOW-LEVEL ERROR CLEARLY

```
class DataLoadError(Exception):
    pass

def load_batch(raw):
    try:
        return int(raw)
    except ValueError as e:
        raise DataLoadError(f"bad sample: {raw!r}") from e

try:
    load_batch("not-a-number")
except DataLoadError as e:
    print("high level:", e)
    print("caused by :", type(e.__cause__).__name__)
```

OUTPUT

```
high level: bad sample: 'not-a-number'
caused by : ValueError
```

Re-raising with a bare `raise`

Sometimes you want to log an error and still let it propagate. A bare `raise` (no argument) inside an `except` re-throws the *same* exception with its original traceback intact, do not write `raise e`, which resets the traceback.

LOG, THEN RE-RAISE UNCHANGED

```

import logging
logging.basicConfig(level=logging.INFO, format="%(message)s",
                    force=True)

def checkpoint(step):
    try:
        if step < 0:
            raise ValueError("negative step")
        return f"saved@{step}"
    except ValueError:
        logging.info("logging, then re-raising...")
        raise # bare: keep the original traceback ←
        ack

try:
    checkpoint(-1)
except ValueError as e:
    print("caller saw:", e)

```

OUTPUT

```

caller saw: negative step
logging, then re-raising...

```

Suppressing expected errors with suppress

When an exception is genuinely fine to ignore, `contextlib.suppress` reads better than an empty `except` block and makes the intent explicit.

CONTEXTLIB.SUPPRESS

```

from contextlib import suppress

cache = {"warm": True}
# Delete a key that may not exist, no try/except noise:
with suppress(KeyError):
    del cache["cold"]
print("cache still fine:", cache)

with suppress(FileNotFoundError):
    open("no_such_checkpoint.pt")
print("missing checkpoint ignored")

```

OUTPUT

```
cache still fine: {'warm': True}
missing checkpoint ignored
```

A custom exception hierarchy

Group related failures under one base class. Callers can catch the *base* to handle the whole family, or a specific subclass when they need finer control, exactly how framework error hierarchies are built.

CATCH A WHOLE FAMILY AT THE BASE

```
class TrainingError(Exception):
    """Base for anything that can go wrong during training."""

class DataError(TrainingError):
    pass

class OptimizerError(TrainingError):
    pass

def step(kind):
    if kind == "data":
        raise DataError("corrupt batch")
    if kind == "optim":
        raise OptimizerError("nan gradient")
    return "ok"

for kind in ["data", "optim", "fine"]:
    try:
        print(kind, "->", step(kind))
    except TrainingError as e:      # one catch for the family
        print(kind, "-> caught", type(e).__name__, ":", e)
```

OUTPUT

```
data -> caught DataError : corrupt batch
optim -> caught OptimizerError : nan gradient
fine -> ok
```

Gathering failures with `ExceptionGroup` / `except*`

When several concurrent tasks fail, you want *all* the errors, not just the first. Python 3.11 added `ExceptionGroup` to bundle them and `except*` to handle each type in the bundle separately, the model behind `asyncio.TaskGroup`.

BUNDLE AND SPLIT ERRORS WITH EXCEPT*

```
def run_task(i):
    if i % 2 == 0:
        raise ValueError(f"task {i} bad value")
        raise KeyError(f"task {i} missing key")

errors = []
for i in range(3):
    # imagine these ran concurrently
    try:
        run_task(i)
    except Exception as e:
        errors.append(e)

try:
    raise ExceptionGroup("some tasks failed", errors)
except* ValueError as eg:
    print("value errors:", len(eg.exceptions))
except* KeyError as eg:
    print("key errors  :", len(eg.exceptions))
```

OUTPUT

```
value errors: 2
key errors  : 1
```

Why it matters for AI: the most common PyTorch error you will ever read is a shape mismatch. Validating shapes early with a clear custom exception, and chaining low-level errors into readable ones, turns cryptic stack traces into a two-second fix.

CHAPTER 26

Logging Instead of print for Training Runs

`print` is fine for a quick script, but real training loops need timestamps, severity levels and the ability to silence or redirect output. The `logging` module gives you all of that with one configuration call.

Levels and basicConfig

Messages have levels (`DEBUG` < `INFO` < `WARNING` < `ERROR`). You set a threshold once; anything below it is dropped, so you can dial verbosity up or down without deleting log lines.

CONFIGURE LOGGING ONCE

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s | %(message)s",
)
log = logging.getLogger("train")

log.debug("this is hidden (below INFO)")
log.info("starting training")
log.warning("learning rate looks high")
```

OUTPUT

```
INFO | starting training
WARNING | learning rate looks high
```

Logging per-step metrics

Inside a loop, logging structured metrics beats printing: you get consistent formatting and can later route the same lines to a file or a service without touching the loop body.

A TINY TRAINING LOOP WITH LOGGING

```
import logging

logging.basicConfig(level=logging.INFO,
                    format="%(levelname)s | step=%(message)s")
log = logging.getLogger("loop")

loss = 1.0
for step in range(1, 4):
    loss *= 0.5                # pretend the model improves
    log.info("%d loss=%.4f", step, loss)
log.info("done, final loss=%.4f", loss)
```

OUTPUT

```
INFO | step=1 loss=0.5000
INFO | step=2 loss=0.2500
INFO | step=3 loss=0.1250
INFO | step=done, final loss=0.1250
```

Timestamps and richer formats

Adding `asctime` and the logger name to the format is what makes logs from long runs actually searchable after the fact.

TIMESTAMPED, NAMED LOGS

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(name)s] %(levelname)s: %(message)s",
    datefmt="%H:%M:%S",
    force=True,                # override any earlier config
)
logging.getLogger("data").info("loaded %d samples", 50000)
logging.getLogger("model").warning("checkpoint dir is nearly full")
```

OUTPUT

```
17:14:53 [data] INFO: loaded 50000 samples
17:14:53 [model] WARNING: checkpoint dir is nearly full
```

Logging exceptions with tracebacks

Inside an `except` block, `log.exception(...)` records the message at `ERROR` level *and* attaches the full traceback, so a failed step is captured in your logs without crashing the run. `log.error(msg, exc_info=True)` does the same at a chosen level.

CAPTURE A FAILURE AND CONTINUE

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s: %(message)s",
    force=True,
)
log = logging.getLogger("train")

try:
    weights = 1 / 0          # simulate a step blowing up
except ZeroDivisionError:
    log.exception("training step failed")  # ERROR + traceback
    log.info("recovered and continued")
```

OUTPUT

```
ERROR: training step failed
Traceback (most recent call last):
  File "<string>", line 11, in <module>
    weights = 1 / 0          # simulate a step blowing up
    ~~~~~
ZeroDivisionError: division by zero
INFO: recovered and continued
```

Why it matters for AI: training runs last hours and produce thousands of lines. Levels let you keep `DEBUG` detail available but quiet by default, and structured, timestamped logs are what you grep through when a run diverges at 3 a.m. Frameworks and experiment trackers all build on `logging`.

PART VI

Data, Files and Serialization

CHAPTER 27

File I/O and pathlib

Training runs live and die by files: datasets on disk, checkpoints, logs, configs. Python's `open()` plus the modern `pathlib` module give you a clean, cross-platform way to read, write and discover them without fragile string surgery on paths.

Reading and writing with a context manager

Always open files inside a `with` block. The file is closed automatically even if an exception fires mid-write, so you never leak a half-flushed checkpoint. The mode string picks the operation: `'w'` write, `'r'` read, `'a'` append.

WRITE THEN READ A TEXT FILE

```
import tempfile, os

path = os.path.join(tempfile.mkdtemp(), "notes.txt")
with open(path, "w") as f:
    f.write("epoch 1 loss 0.42\n")
    f.write("epoch 2 loss 0.31\n")

with open(path) as f:
    content = f.read()
print(content, end="")
os.remove(path)
```

OUTPUT

```
epoch 1 loss 0.42
epoch 2 loss 0.31
```

Iterating lines lazily

A file object is itself an iterator: looping over it yields one line at a time without loading the whole file into memory. This is how you stream a multi-gigabyte `.jsonl` dataset that would never fit in RAM.

STREAM A LARGE FILE LINE BY LINE

```
import tempfile, os

path = os.path.join(tempfile.mkdtemp(), "data.txt")
with open(path, "w") as f:
    for i in range(5):
        f.write(f"sample-{i}\n")

# Never loads the whole file at once - one line per iteration.
with open(path) as f:
    for lineno, line in enumerate(f, start=1):
        print(lineno, line.strip())
os.remove(path)
```

OUTPUT

```
1 sample-0
2 sample-1
3 sample-2
4 sample-3
5 sample-4
```

pathlib: paths as objects

`pathlib.Path` treats a path as an object with useful attributes and methods. The `/` operator joins path parts, `.stem/.suffix` split a filename, and `.mkdir(parents=True, exist_ok=True)` creates directory trees safely.

BUILDING AND INSPECTING PATHS

```

from pathlib import Path
import tempfile

root = Path(tempfile.mkdtemp())
ckpt_dir = root / "checkpoints"
ckpt_dir.mkdir(parents=True, exist_ok=True)

ckpt = ckpt_dir / "model_epoch3.pt"
ckpt.write_bytes(b"fake-weights")
print("name :", ckpt.name)
print("stem :", ckpt.stem)
print("suffix:", ckpt.suffix)
print("exists:", ckpt.exists())
print("parent:", ckpt.parent.name)

```

OUTPUT

```

name : model_epoch3.pt
stem : model_epoch3
suffix: .pt
exists: True
parent: checkpoints

```

Globbering for checkpoints

`Path.glob` finds files by pattern, perfect for locating every checkpoint or shard in a directory. Combine it with `sorted` and `max` to grab the latest one.

FIND ALL CHECKPOINTS, PICK THE NEWEST

```

from pathlib import Path
import tempfile

d = Path(tempfile.mkdtemp())
for epoch in (1, 5, 12, 3):
    (d / f"ckpt_{epoch:03d}.pt").write_bytes(b"x")

ckpts = sorted(d.glob("ckpt_*.pt"))
print("found:", [p.name for p in ckpts])
latest = max(ckpts, key=lambda p: int(p.stem.split("_")[1]))
print("latest:", latest.name)

```

OUTPUT

```
found: ['ckpt_001.pt', 'ckpt_003.pt', 'ckpt_005.pt', 'ckpt_012.
      pt']
latest: ckpt_012.pt
```

Whole-file helpers and directory walking

For small files, `Path.write_text/read_text` (and the binary `write_bytes/read_bytes`) do the open-write-close dance in a single call - no `with` block needed. To discover files, `Path.iterdir` yields the entries of one directory while `Path.rglob` walks a whole tree recursively, which is how you gather every shard scattered under a nested dataset root.

ONE-SHOT READS AND A RECURSIVE WALK

```
from pathlib import Path
import tempfile

root = Path(tempfile.mkdtemp())
(root / "train").mkdir()
(root / "val").mkdir()
(root / "config.json").write_text('{"lr": 0.001}')
(root / "train" / "shard_0.jsonl").write_text('{"x": 1}\n')
(root / "val" / "shard_0.jsonl").write_text('{"x": 2}\n')

print("config:", (root / "config.json").read_text())
print("top level:")
for entry in sorted(root.iterdir()):
    kind = "dir " if entry.is_dir() else "file"
    print(" ", kind, entry.name)

shards = sorted(p.relative_to(root) for p in root.rglob("*.jsonl"))
print("all shards:", [str(s) for s in shards])
```

OUTPUT

```
config: {"lr": 0.001}
top level:
  file config.json
  dir train
  dir val
all shards: ['train/shard_0.jsonl', 'val/shard_0.jsonl']
```

Why it matters for AI: dataset loaders stream files line by line to stay within memory, and resuming training means globbing a checkpoint directory and loading the newest file. `pathlib` makes both robust across Linux, macOS and Windows.

CHAPTER 28

JSON and CSV: Configs and Datasets

Two text formats dominate practical AI work: **JSON** for configs, API payloads and `.jsonl` datasets, and **CSV** for tabular data. Both ship in the standard library, so you can read and write them without any dependency.

JSON: strings to objects and back

`json.dumps` serialises a Python object to a string (use `indent=` for a readable config file); `json.loads` parses it back. The `dump/load` pair does the same directly to and from file objects.

SERIALISE AND PARSE A CONFIG

```
import json

config = {"model": "gpt", "layers": 12, "lr": 3e-4, "dropout": 0.1}

text = json.dumps(config, indent=2)
print(text)

restored = json.loads(text)
print("layers x2 =", restored["layers"] * 2)
```

OUTPUT

```
{
  "model": "gpt",
  "layers": 12,
  "lr": 0.0003,
  "dropout": 0.1
}
layers x2 = 24
```

JSON Lines: one example per line

The `.jsonl` format stores one JSON object per line. It is the

de-facto standard for LLM fine-tuning datasets because you can stream it line by line and append new records cheaply.

WRITE AND READ A .JSONL DATASET

```
import json, io

records = [
    {"prompt": "2+2?", "completion": "4"},
    {"prompt": "capital of France?", "completion": "Paris"},
]

buf = io.StringIO()
for r in records:
    buf.write(json.dumps(r) + "\n")

buf.seek(0)
for line in buf:
    obj = json.loads(line)
    print(obj["prompt"], "->", obj["completion"])
```

OUTPUT

```
2+2? -> 4
capital of France? -> Paris
```

Formatting options and custom encoders

`json.dumps` takes formatting flags worth knowing: `indent=` pretty-prints, `sort_keys=True` makes output deterministic so two configs diff cleanly, and `ensure_ascii=False` keeps accented or non-Latin characters readable instead of escaping them to `\\uXXXX`. For a type JSON does not understand - a `set`, a `datetime`, a `dataclass` - pass a `default=` function that converts it into something serialisable.

SORTED KEYS, UNICODE AND A CUSTOM DEFAULT


```

import json
from dataclasses import dataclass, asdict

@dataclass
class RunConfig:
    name: str
    tags: set

def encode(obj):
    if isinstance(obj, set):
        return sorted(obj)          # sets are not JSON-native
    raise TypeError(f"cannot serialise {type(obj).__name__}")

cfg = RunConfig(name="expérience", tags={"nlp", "gpu"})

text = json.dumps(asdict(cfg), indent=2, sort_keys=True,
                  ensure_ascii=False, default=encode)
print(text)

```

OUTPUT

```

{
  "name": "expérience",
  "tags": [
    "gpu",
    "nlp"
  ]
}

```

CSV with DictReader / DictWriter

The `csv` module handles quoting and escaping for you. `DictWriter` writes rows from dictionaries and `DictReader` yields each row as a dict keyed by the header, which keeps column access readable.

ROUND-TRIP A SMALL DATASET

```
import csv, io

rows = [
    {"text": "great movie", "label": 1},
    {"text": "boring plot", "label": 0},
]

buf = io.StringIO()
writer = csv.DictWriter(buf, fieldnames=["text", "label"])
writer.writeheader()
writer.writerows(rows)

buf.seek(0)
reader = csv.DictReader(buf)
pos = sum(1 for row in reader if row["label"] == "1")
print("CSV written:")
print(buf.getvalue(), end="")
print("positive examples:", pos)
```

OUTPUT

```
CSV written:
text,label
great movie,1
boring plot,0
positive examples: 1
```

Fields that contain commas and quotes

Real text data is full of commas, quotes and newlines. The `csv` module quotes such fields on write and unquotes them on read, so a value with a comma is never split across two columns. When reading or writing a CSV *file* (not an in-memory buffer), always open it with `newline=""` so the module - not the OS - controls line endings.

ROUND-TRIP A FIELD WITH COMMAS AND QUOTES

```
import csv, io

rows = [{"text": 'He said, "hi", loudly', "label": "greeting"}]

buf = io.StringIO()
writer = csv.DictWriter(buf, fieldnames=["text", "label"])
writer.writeheader()
writer.writerows(rows)

print("raw csv line:")
print(buf.getvalue(), end="")

buf.seek(0)
row = next(csv.DictReader(buf))
print("parsed text :", row["text"])
print("still 2 cols:", len(row))
```

OUTPUT

```
raw csv line:
text,label
"He said, ""hi"", loudly",greeting
parsed text : He said, "hi", loudly
still 2 cols: 2
```

Why it matters for AI: fine-tuning datasets are almost always `.jsonl`, hyperparameter configs are JSON or YAML, and classic tabular ML lives in CSV. Note that `csv` reads every field as a **string** - cast to `int/float` yourself before training.

CHAPTER 29

pickle and Model / Artifact Serialization

JSON only handles simple types. To save an arbitrary Python object - a fitted scaler, a vocabulary, a dict of NumPy arrays - you reach for **pickle**, Python's native binary serialisation format. It is what most ML checkpoint formats are built on under the hood.

Pickling arbitrary objects

`pickle.dumps` turns almost any object into bytes and `pickle.loads` reconstructs it. Here we save a small preprocessing artifact: a vocabulary plus normalisation statistics.

ROUND-TRIP A PREPROCESSING ARTIFACT

```
import pickle

artifact = {
    "vocab": {"<pad>": 0, "cat": 1, "dog": 2},
    "mean": 0.5,
    "std": 0.25,
}

blob = pickle.dumps(artifact)
print("serialised bytes:", len(blob))

restored = pickle.loads(blob)
print("vocab size:", len(restored["vocab"]))
print("dog id    :", restored["vocab"]["dog"])
```

OUTPUT

```
serialised bytes: 85
vocab size: 3
dog id    : 2
```

Pickling NumPy arrays to disk

NumPy arrays pickle cleanly, so a dict of weights survives a save/load round-trip exactly. In real code you would prefer `numpy.save/load (.npy)` for plain arrays, but pickle wins when you need to bundle arrays together with other Python objects.

SAVE A WEIGHT DICT, THEN RELOAD IT

```
import pickle, tempfile, os
import numpy as np

weights = {"w": np.arange(6).reshape(2, 3), "b": np.zeros(2)}

path = os.path.join(tempfile.mkdtemp(), "weights.pkl")
with open(path, "wb") as f:      # note the binary mode "wb"
    pickle.dump(weights, f)

with open(path, "rb") as f:
    loaded = pickle.load(f)

print("shapes:", {k: v.shape for k, v in loaded.items()})
print("equal :", np.array_equal(weights["w"], loaded["w"]))
os.remove(path)
```

OUTPUT

```
shapes: {'w': (2, 3), 'b': (2,)}
equal : True
```

Protocol versions and size

Pickle has several wire formats. Higher protocols are faster and more compact; `pickle.DEFAULT_PROTOCOL` is what `dumps` uses unless you override it, and `pickle.HIGHEST_PROTOCOL` is the newest your interpreter supports. Pin a protocol number when a file must also be readable by an older Python.

COMPARE PICKLE PROTOCOLS

```
import pickle

obj = {"weights": list(range(1000)), "step": 42}

print("default protocol:", pickle.DEFAULT_PROTOCOL)
print("highest protocol:", pickle.HIGHEST_PROTOCOL)
for proto in (0, pickle.DEFAULT_PROTOCOL, pickle.HIGHEST_PROTOCOL):
    size = len(pickle.dumps(obj, protocol=proto))
    print(f"protocol {proto}: {size} bytes")
```

OUTPUT

```
default protocol: 5
highest protocol: 5
protocol 0: 5928 bytes
protocol 5: 2783 bytes
protocol 5: 2783 bytes
```

Why untrusted pickles are dangerous

Unpickling is not passive data loading: an object can define `__reduce__` to run *any* callable the moment it is loaded. The demo below is deliberately harmless - it only calls `print` - but the very same mechanism could delete files or spawn a shell. This is why you must never `pickle.load` bytes from a source you do not trust.

A PICKLE THAT RUNS CODE ON LOAD

```
import pickle

class Exploit:
    def __reduce__(self):
        # The returned callable + args run during unpickling.
        return (print, ("[!] arbitrary code ran on load",))

blob = pickle.dumps(Exploit())
print("loading the blob triggers the payload:")
pickle.loads(blob) # prints the message - could be anything
```

OUTPUT

```
loading the blob triggers the payload:  
[!] arbitrary code ran on load
```

The framework equivalent

Deep-learning frameworks wrap pickle in their own save/load helpers. The snippet below is *illustrative* (PyTorch is not installed here) but shows the identical mental model: one call to persist state, one to restore it.

ILLUSTRATIVE: TORCH.SAVE / TORCH.LOAD

```
import torch  
  
# Save just the learned parameters (the recommended pattern).  
torch.save(model.state_dict(), "model.pt")  
  
# Later, rebuild the architecture, then load weights into it.  
model = MyModel()  
model.load_state_dict(torch.load("model.pt"))  
model.eval()
```

Safer alternatives for weights

For pure numeric weights, prefer formats that store data *without* embedded code. [numpy.save/.npz](#) bundles plain arrays, [joblib](#) compresses large arrays efficiently (though it is still pickle-based), and [safetensors](#) has become the default for shared model weights precisely because loading it can never execute code. The snippet is illustrative only.

ILLUSTRATIVE: SAFETENSORS AND JOBLIB

```
import numpy as np
from safetensors.numpy import save_file, load_file
import joblib

# safetensors: weights only, no arbitrary code on load.
save_file({"w": np.zeros((2, 3))}, "model.safetensors")
weights = load_file("model.safetensors")

# joblib: efficient for big arrays, but still pickle-based.
joblib.dump(weights, "weights.joblib")
weights = joblib.load("weights.joblib")
```

Why it matters for AI - and a security warning: **never unpickle a file you did not create**. Unpickling can execute arbitrary code, so a malicious checkpoint downloaded from the internet is a remote-code-execution risk. This is exactly why safer formats like [safetensors](#) (weights only, no code) and [joblib](#) (efficient for large arrays) exist.

PART VII

Concurrency and Performance

CHAPTER 30

Threads, Processes and concurrent.futures for Data Loading

Data loading is often the real bottleneck in training: the GPU sits idle while the CPU decodes images or tokenises text. Python gives you two kinds of parallelism to hide that latency, and choosing between them comes down to one acronym: the **GIL**.

The GIL: threads vs processes

The Global Interpreter Lock lets only one thread run Python bytecode at a time. So **threads** win for I/O-bound work (downloading, reading files) because the GIL is released while waiting, but **processes** are needed for CPU-bound work (heavy pure-Python computation) since each process has its own interpreter and GIL.

- **Threads** - I/O-bound: downloading shards, reading files, calling APIs.
- **Processes** - CPU-bound: heavy pixel/audio decoding, pure-Python math.
- Most tensor math already runs in C and releases the GIL, so threads go further than beginners expect.

The demo below makes the GIL visible. We run the *same* CPU-bound loop four times, first sequentially and then across four threads. Because only one thread executes Python bytecode at a time, the threaded version is **no faster** - it may even be slower from lock contention. This is the single most important thing to internalise before reaching for threads.

THE GIL BLOCKS CPU-BOUND THREADS

```

import time
from concurrent.futures import ThreadPoolExecutor

def cpu_task(n):
    total = 0
    for i in range(n):          # pure-Python work: holds the GIL
        total += i * i
    return total

N = 1_000_000

start = time.perf_counter()
for _ in range(4):
    cpu_task(N)
seq = time.perf_counter() - start

start = time.perf_counter()
with ThreadPoolExecutor(max_workers=4) as pool:
    list(pool.map(cpu_task, [N] * 4))
thr = time.perf_counter() - start

print(f"sequential: {seq:.2f}s")
print(f"4 threads: {thr:.2f}s (no speedup - the GIL serialises it)")

```

OUTPUT

```

sequential: 0.19s
4 threads: 0.19s (no speedup - the GIL serialises it)

```

ThreadPoolExecutor.map for parallel I/O

`concurrent.futures` gives a single clean API for both pools. `executor.map` applies a function across an iterable concurrently and returns results **in input order** - ideal for preprocessing a batch of samples. We simulate I/O latency with `time.sleep`.

PARALLEL PREPROCESSING WITH A THREAD POOL

```

import time
from concurrent.futures import ThreadPoolExecutor

def load_sample(i):
    time.sleep(0.1)          # pretend this is a network / disk
    read
    return i * i

items = range(8)

start = time.perf_counter()
with ThreadPoolExecutor(max_workers=8) as pool:
    results = list(pool.map(load_sample, items))
elapsed = time.perf_counter() - start

print("results:", results)
print(f"8 x 0.1s work done in {elapsed:.2f}s (concurrent, not 0
    .8s)")

```

OUTPUT

```

results: [0, 1, 4, 9, 16, 25, 36, 49]
8 x 0.1s work done in 0.10s (concurrent, not 0.8s)

```

submit + as_completed

When you want each result the moment it is ready (say, to update a progress bar) use `submit` to schedule work and `as_completed` to iterate over futures as they finish, in **completion order** rather than submission order.

PROCESS RESULTS AS THEY FINISH

```
import time, random
from concurrent.futures import ThreadPoolExecutor, as_completed

def fetch(url):
    time.sleep(random.uniform(0.02, 0.12))
    return f"{url} -> 200 OK"

urls = [f"shard-{i}" for i in range(5)]

with ThreadPoolExecutor(max_workers=5) as pool:
    futures = {pool.submit(fetch, u): u for u in urls}
    for fut in as_completed(futures):
        print("done:", fut.result())
```

OUTPUT

```
done: shard-4 -> 200 OK
done: shard-1 -> 200 OK
done: shard-0 -> 200 OK
done: shard-3 -> 200 OK
done: shard-2 -> 200 OK
```

Exceptions travel through the future

A worker that raises does not crash the pool - the exception is stored on its future and **re-raised** when you call `future.result()`. Wrap that call in `try/except` so one bad sample cannot abort the whole batch; a robust data pipeline logs the failure and keeps the good results.

HANDLING FAILURES PER FUTURE

```

from concurrent.futures import ThreadPoolExecutor, as_completed

def load_sample(i):
    if i == 3:
        raise ValueError(f"corrupt record {i}")
    return i * 10

good, failed = [], []
with ThreadPoolExecutor(max_workers=4) as pool:
    futures = {pool.submit(load_sample, i): i for i in range(6)}
    for fut in as_completed(futures):
        i = futures[fut]
        try:
            good.append(fut.result())      # re-raises here if i
            t failed
        except ValueError as exc:
            failed.append((i, str(exc)))

print("loaded:", sorted(good))
print("skipped:", failed)

```

OUTPUT

```

loaded: [0, 10, 20, 40, 50]
skipped: [(3, 'corrupt record 3')]

```

Switching to processes for CPU work

The API is identical - just swap the executor class. The one catch is that process pools re-import your module, so the launch code must sit under an `if __name__ == '__main__':` guard. The snippet below is *illustrative* (process pools are awkward inside tiny scripts, but this is exactly the shape you use in a real training script).

ILLUSTRATIVE: PROCESSPOOLEXECUTOR FOR CPU-BOUND WORK

```
from concurrent.futures import ProcessPoolExecutor

def heavy_decode(chunk):
    # Pure-Python CPU work that the GIL would otherwise seriali
    se.
    return sum(i * i for i in range(chunk))

if __name__ == "__main__":          # required for process pools
    with ProcessPoolExecutor(max_workers=4) as pool:
        totals = list(pool.map(heavy_decode, [10_000] * 4))
    print(totals)
```

Why it matters for AI: this is exactly what a PyTorch [DataLoader\(num_workers=N\)](#) does under the hood - fan out sample loading across workers so the GPU never starves. Reach for threads on I/O-bound pipelines and processes only when the work is genuinely CPU-bound Python.

CHAPTER 31

async / await for Concurrent LLM and API Calls

When you call an LLM API, almost all the time is spent waiting for the network. **async/await** lets a single thread juggle hundreds of in-flight requests, making it the standard tool for batching prompts, building agents and serving inference.

Defining and running a coroutine

`async def` defines a *coroutine*; calling it returns an object that does nothing until awaited. `asyncio.run` starts the event loop and drives one coroutine to completion. `await` yields control while waiting, so other tasks can proceed.

A FIRST COROUTINE

```
import asyncio

async def call_model(prompt):
    await asyncio.sleep(0.1)           # stand-in for a network round-trip
    return f"response to: {prompt!r}"

async def main():
    reply = await call_model("hello")
    print(reply)

asyncio.run(main())
```

OUTPUT

```
response to: 'hello'
```

gather: fire many requests at once

`asyncio.gather` schedules many coroutines concurrently and waits for all of them, returning results in order. Ten requests that each take 0.1s finish in about 0.1s total, not 1.0s - the

whole point of `async`.

CONCURRENT REQUESTS WITH GATHER

```
import asyncio, time

async def call_model(prompt):
    await asyncio.sleep(0.1)
    return prompt.upper()

async def main():
    prompts = [f"q{i}" for i in range(10)]
    start = time.perf_counter()
    results = await asyncio.gather(*(call_model(p) for p in prompts))
    elapsed = time.perf_counter() - start
    print("results:", results)
    print(f"10 requests in {elapsed:.2f}s (concurrent, not 1.0s)")

asyncio.run(main())
```

OUTPUT

```
results: ['Q0', 'Q1', 'Q2', 'Q3', 'Q4', 'Q5', 'Q6', 'Q7', 'Q8', 'Q9']
10 requests in 0.10s (concurrent, not 1.0s)
```

Semaphore: respecting rate limits

Real APIs cap how many requests you may have in flight. An `asyncio.Semaphore` bounds concurrency: acquire it before each call and only `N` run at once, while the rest wait their turn. This is the idiomatic way to stay under a rate limit.

BOUNDING CONCURRENCY TO N AT A TIME

```

import asyncio, time

sem = asyncio.Semaphore(3)          # at most 3 concurrent call
s

async def call_model(i):
    async with sem:
        await asyncio.sleep(0.1)
        return i

async def main():
    start = time.perf_counter()
    results = await asyncio.gather(*(call_model(i) for i in range(9)))
    elapsed = time.perf_counter() - start
    # 9 tasks, 3 at a time, 0.1s each => ~3 waves => ~0.3s
    print("results:", results)
    print(f"9 calls, limit 3, took {elapsed:.2f}s (~3 waves)")

asyncio.run(main())

```

OUTPUT

```

results: [0, 1, 2, 3, 4, 5, 6, 7, 8]
9 calls, limit 3, took 0.30s (~3 waves)

```

as_completed: stream results as they land

`gather` returns everything at once, in order. When responses have uneven latency and you want to act on each the instant it arrives - update a UI, write to disk, feed a downstream step - use `asyncio.as_completed`. It yields awaitables in **completion order**, so the fastest calls surface first.

STREAMING RESULTS WITH AS_COMPLETED

```
import asyncio, random

async def call_model(i):
    await asyncio.sleep(random.uniform(0.02, 0.12))
    return f"result-{i}"

async def main():
    tasks = [call_model(i) for i in range(5)]
    for coro in asyncio.as_completed(tasks):
        print("arrived:", await coro)

asyncio.run(main())
```

OUTPUT

```
arrived: result-4
arrived: result-1
arrived: result-2
arrived: result-3
arrived: result-0
```

wait_for: bounding slow calls with a timeout

A hung request must not stall your pipeline. `asyncio.wait_for` cancels the coroutine and raises `TimeoutError` once the deadline passes, letting you retry or fall back. Here the call would take 1s but we cap it at 0.1s.

TIMEOUTS AND FALLBACKS

```
import asyncio

async def slow_call():
    await asyncio.sleep(1.0)           # a call that hangs
    return "real answer"

async def main():
    try:
        answer = await asyncio.wait_for(slow_call(), timeout=0.1)
    except asyncio.TimeoutError:
        answer = "fallback (timed out)"
    print(answer)

asyncio.run(main())
```

OUTPUT

```
fallback (timed out)
```

Async generators: consuming a token stream

Streaming LLM responses arrive token by token. An **async generator** (an `async def` that `yields`) models this: it `awaits` the next chunk from the network and yields it, and the caller consumes it with `async for`. The loop stays responsive between tokens, so you can render output as it is produced.

CONSUMING AN ASYNC TOKEN STREAM

```
import asyncio

async def stream_tokens(text):
    for token in text.split():
        await asyncio.sleep(0.02)    # wait for the next chunk
        yield token

async def main():
    pieces = []
    async for token in stream_tokens("the model streams its reply"):
        pieces.append(token)
        print(token, end=" ", flush=True)
    print("\n[assembled]", " ".join(pieces))

asyncio.run(main())
```

OUTPUT

```
the model streams its reply
[assembled] the model streams its reply
```

What a real async LLM client looks like

The pattern is identical with a real SDK: build an async client, wrap each call in a semaphore, and [gather](#) them. The snippet below is *illustrative* only - no network call is made - but it mirrors the shape of production batch-inference code.

ILLUSTRATIVE: CONCURRENT REAL API CALLS

```

import asyncio
from some_llm_sdk import AsyncClient      # e.g. an async Anthr ←
    opic/OpenAI client

client = AsyncClient()
sem = asyncio.Semaphore(5)                # honour the provider' ←
    s rate limit

async def ask(prompt):
    async with sem:
        resp = await client.messages.create(
            model="claude-opus-4-8",
            max_tokens=256,
            messages=[{"role": "user", "content": prompt}],
        )
        return resp.content[0].text

async def main(prompts):
    return await asyncio.gather(*(ask(p) for p in prompts))

answers = asyncio.run(main(["Summarise X", "Translate Y", "Clas ←
    sify Z"]))

```

Why it matters for AI: batch inference, agent loops and retrieval pipelines all issue many network calls whose cost is latency, not CPU. [asyncio.gather](#) for concurrency plus a [Semaphore](#) for rate limiting is the backbone of fast, well-behaved LLM clients.

CHAPTER 32

Choosing Concurrency: I/O-bound vs CPU-bound

Three tools, one question: *is the work waiting or computing?* Get that right and the choice almost makes itself. **I/O-bound** work waits on the network or disk; **CPU-bound** work burns processor cycles in pure Python.

A decision guide

- **Many I/O-bound calls (APIs, LLMs, DB)** - prefer *async*: one thread juggles thousands of awaits with the least overhead.
- **I/O-bound with blocking libraries** (no *async* support) - use *threads* via `ThreadPoolExecutor`; the GIL is released while they wait.
- **CPU-bound pure-Python work** - use *processes* via `ProcessPoolExecutor`; only separate interpreters sidestep the GIL.
- **Already-vectorised C work** (NumPy, PyTorch) - it releases the GIL, so threads or even a plain loop are often enough.

Measuring the difference

The demo below runs the same batch of I/O-bound tasks sequentially and then through a thread pool. For work dominated by waiting, concurrency turns near-linear time into near-constant time - the exact win you get from *async* or threads.

SEQUENTIAL VS CONCURRENT I/O

```

import time
from concurrent.futures import ThreadPoolExecutor

def fetch(i):
    time.sleep(0.05)          # I/O-bound: waiting, not computing
    return i

n = 10
start = time.perf_counter()
seq = [fetch(i) for i in range(n)]
seq_t = time.perf_counter() - start

start = time.perf_counter()
with ThreadPoolExecutor(max_workers=n) as pool:
    conc = list(pool.map(fetch, range(n)))
conc_t = time.perf_counter() - start

print(f"sequential: {seq_t:.2f}s")
print(f"concurrent: {conc_t:.2f}s")
print(f"speedup:    {seq_t / conc_t:.1f}x")

```

OUTPUT

```

sequential: 0.50s
concurrent: 0.06s
speedup:    9.0x

```

Why it matters for AI: an inference pipeline mixes both worlds - network-bound LLM calls (async), blocking client libraries (threads) and CPU-bound tokenisation or image decoding (processes). Naming each stage 'waiting' or 'computing' tells you which tool keeps the GPU fed and the latency low.

PART VIII

The Numeric AI Stack

CHAPTER 33

NumPy Arrays, dtypes and Vectorization

NumPy is the foundation of the entire Python AI stack. A tensor in PyTorch, TensorFlow or JAX is essentially a NumPy array with a GPU and autograd bolted on, so the mental model you build here transfers directly. The core object is the **ndarray**: a contiguous block of numbers with a single **dtype** and a **shape**.

Creating arrays

You rarely type numbers by hand. **zeros**, **ones**, **arange** and **linspace** build the weight matrices, index ranges and learning-rate schedules you actually use.

WAYS TO MAKE AN ARRAY

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print("from list:\n", a)
print("zeros:", np.zeros(3))
print("ones (2x2):\n", np.ones((2, 2)))
print("arange:", np.arange(0, 10, 2))
print("linspace:", np.linspace(0.0, 1.0, 5))
```

OUTPUT

```
from list:
[[1 2 3]
 [4 5 6]]
zeros: [0. 0. 0.]
ones (2x2):
[[1. 1.]
 [1. 1.]]
arange: [0 2 4 6 8]
linspace: [0.    0.25 0.5   0.75 1.   ]
```

dtype: why models use float32

Every array has one `dtype`. Deep-learning models default to `float32` (not Python's 64-bit float) because it halves memory and doubles throughput on GPUs with negligible accuracy loss.

Knowing when a cast happens saves you from silent slowdowns.

DYPES AND CASTING

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
print("default dtype:", x.dtype)           # float64
w = np.ones(3, dtype=np.float32)
print("model weights:", w.dtype)
print("bytes each:", x.nbytes, "vs", w.nbytes)
ids = np.array([5, 1, 9], dtype=np.int64) # token ids are ints
print("ids dtype:", ids.dtype)
```

OUTPUT

```
default dtype: float64
model weights: float32
bytes each: 24 vs 12
ids dtype: int64
```

shape, ndim, size and reshape

The `shape` is the grammar of tensor code. A batch of images might be `(32, 3, 224, 224)`; a batch of token embeddings (`batch, seq_len, dim`). `reshape` rearranges the same data without copying it, and `-1` means "infer this axis".

INSPECTING AND RESHAPING

```
import numpy as np

a = np.arange(12)
print("shape:", a.shape, "ndim:", a.ndim, "size:", a.size)
b = a.reshape(3, 4)
print("reshaped:\n", b)
print("flatten with -1:", b.reshape(-1).shape)
print("add batch axis:", a.reshape(1, -1).shape)
```

OUTPUT

```

shape: (12,) ndim: 1 size: 12
reshaped:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
flatten with -1: (12,)
add batch axis: (1, 12)

```

Views vs copies: a slice shares memory

This trips up nearly everyone once. A basic slice returns a **view** onto the *same* memory, not a copy, so mutating the view mutates the original. `reshape` and `ravel` also return views when they can. Call `.copy()` when you need an independent array, and `np.newaxis` to insert a length-1 axis (the classic way to add a batch dimension).

A VIEW MUTATES THE ORIGINAL

```

import numpy as np

a = np.arange(6)
view = a[:3]           # a slice is a VIEW, not a copy
view[0] = 99           # mutating the view...
print("original changed:", a)    # ...changes a!
safe = a[:3].copy()    # .copy() breaks the link
safe[0] = -1
print("original intact:", a)
print("ravel gives a flat view:", a.reshape(2, 3).ravel().shape)
)
row = a[np.newaxis, :] # add a leading axis
print("newaxis shape:", row.shape)
print("astype to int8 bytes:", a.astype(np.int8).nbytes)

```

OUTPUT

```

original changed: [99  1  2  3  4  5]
original intact: [99  1  2  3  4  5]
ravel gives a flat view: (6,)
newaxis shape: (1, 6)
astype to int8 bytes: 6

```

Vectorization beats Python loops

The golden rule of numeric Python: *never loop over elements if an array operation exists*. Vectorized operations run in optimized C, so they are typically 10-100x faster and read like the math they implement.

LOOP VS VECTORIZED (SAME RESULT, ONE IS FAST)

```
import numpy as np, time

x = np.arange(1_000_000, dtype=np.float64)

t0 = time.perf_counter()
loop = [v * 2.0 + 1.0 for v in x]      # slow, pure Python
t1 = time.perf_counter()
vec = x * 2.0 + 1.0                    # fast, vectorized C
t2 = time.perf_counter()

print("same result:", np.allclose(loop, vec))
print(f"loop:  {t1 - t0:.4f}s")
print(f"vector:{t2 - t1:.4f}s")
```

OUTPUT

```
same result: True
loop:  0.1234s
vector:0.0026s
```

Aggregations along an axis

Reductions like `sum`, `mean` and `max` take an `axis` argument. `axis=0` collapses rows (per-feature stats), `axis=1` collapses columns (per-sample stats). This is how you compute batch statistics and pool over sequence dimensions.

REDUCTIONS WITH AXIS

```
import numpy as np

batch = np.array([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]]) # (2 samples, 3 features)
print("grand mean:", batch.mean())
print("per-feature mean (axis=0):", batch.mean(axis=0))
print("per-sample sum (axis=1):", batch.sum(axis=1))
print("argmax per sample:", batch.argmax(axis=1))
```

OUTPUT

```
grand mean: 3.5
per-feature mean (axis=0): [2.5 3.5 4.5]
per-sample sum (axis=1): [ 6. 15.]
argmax per sample: [2 2]
```

Why it matters for AI: every forward pass is array creation, reshaping and vectorized math over a fixed dtype. If you can read shapes and reach for an axis-aware reduction instead of a for-loop, you can read almost any model's code.

CHAPTER 34

Broadcasting, Indexing and Linear Algebra: the Math of a Layer

A neural network layer is mostly one line of linear algebra: $y = X @ W + b$. To write and debug it you need three skills: **broadcasting** (combining arrays of different shapes), **indexing** (pulling out the elements you want), and the **matmul** operator `@`.

Broadcasting: adding a bias to a batch

Broadcasting lets NumPy stretch a smaller array across a larger one without copying. Adding a bias vector of shape `(features,)` to a batch of shape `(batch, features)` just works, because the trailing dimensions match.

BROADCASTING A BIAS ACROSS A BATCH

```
import numpy as np

batch = np.zeros((3, 4))          # 3 samples, 4 features
bias = np.array([10, 20, 30, 40]) # one bias per feature
print("batch + bias:\n", batch + bias)
# column vector broadcasts the other way:
scale = np.array([[1], [2], [3]]) # shape (3, 1) -> per-sample
print("per-sample scale:\n", (batch + bias) * scale)
```

OUTPUT

```
batch + bias:
[[10. 20. 30. 40.]
 [10. 20. 30. 40.]
 [10. 20. 30. 40.]]
per-sample scale:
[[ 10.  20.  30.  40.]
 [ 20.  40.  60.  80.]
 [ 30.  60.  90. 120.]]
```

Slicing and boolean masking

Slices select contiguous regions; boolean masks select elements by condition. Masking is how you apply a ReLU, filter padded tokens, or clip outliers, all without a loop.

SLICES AND MASKS

```
import numpy as np

a = np.arange(1, 13).reshape(3, 4)
print("first row:", a[0])
print("last column:", a[:, -1])
print("top-left 2x2:\n", a[:2, :2])

mask = a > 6
print("mask:\n", mask)
relu = np.where(a - 6 > 0, a - 6, 0)    # a mini ReLU
print("relu(a-6):\n", relu)
```

OUTPUT

```
first row: [1 2 3 4]
last column: [ 4  8 12]
top-left 2x2:
[[1 2]
 [5 6]]
mask:
[[False False False False]
 [False False  True  True]
 [ True  True  True  True]]
relu(a-6):
[[0 0 0 0]
 [0 0 1 2]
 [3 4 5 6]]
```

Fancy indexing: gathering rows

Indexing with an array of integers gathers arbitrary rows in any order, with repeats. This is exactly an **embedding lookup**: token ids index into an embedding table.

EMBEDDING LOOKUP BY FANCY INDEXING


```
import numpy as np

embed = np.round(np.arange(15).reshape(5, 3) / 10, 1) # vocab=5, dim=3
print("embedding table:\n", embed)
token_ids = np.array([0, 2, 2, 4])
print("looked up:\n", embed[token_ids]) # shape (4, 3)
```

OUTPUT

```
embedding table:
[[0.  0.1 0.2]
 [0.3 0.4 0.5]
 [0.6 0.7 0.8]
 [0.9 1.  1.1]
 [1.2 1.3 1.4]]
looked up:
[[0.  0.1 0.2]
 [0.6 0.7 0.8]
 [0.6 0.7 0.8]
 [1.2 1.3 1.4]]
```

The matmul operator @ and a linear layer

@ is matrix multiplication. Combined with broadcasting for the bias, it is a complete dense layer. Watch the shapes: (batch, in) @ (in, out) -> (batch, out).

A BATCHED LINEAR LAYER $Y = X @ W + B$

```
import numpy as np
rng = np.random.default_rng(0)

X = rng.normal(size=(2, 3)) # batch=2, in_features=3
W = rng.normal(size=(3, 4)) # in=3, out=4
b = np.zeros(4) # one bias per output

y = X @ W + b
print("X:", X.shape, " W:", W.shape, " -> y:", y.shape)
print("y:\n", np.round(y, 3))
```

OUTPUT

```
X: (2, 3)  W: (3, 4)  ->  y: (2, 4)
y:
[[-0.552 -0.355 -0.13  -0.333]
 [ 0.02  -0.188  0.975 -0.13  ]]
```

Batched matmul and einsum

`@` also broadcasts over leading **batch** axes: a (B, m, k) array times a (k, n) array gives (B, m, n) , one matmul per batch element with no loop. When the index bookkeeping gets hairy, `np.einsum` spells out exactly which axes multiply and which get summed. The attention scores $Q @ K^T$ are a one-liner: "`qd,kd->qk`" says "multiply over the shared dim `d`, keep queries and keys".

BATCHED LAYER AND ATTENTION SCORES

```
import numpy as np
rng = np.random.default_rng(1)

X = rng.normal(size=(8, 2, 3))    # 8 batches of (2, 3)
W = rng.normal(size=(3, 4))       # shared (3, 4)
Y = X @ W                         # broadcasts over the batch axis
    is
print("X:", X.shape, "@ W:", W.shape, "->", Y.shape)

Q = rng.normal(size=(2, 3))       # 2 queries, dim 3
K = rng.normal(size=(4, 3))       # 4 keys,    dim 3
scores = np.einsum("qd,kd->qk", Q, K) # Q @ K^T
print("attention scores:", scores.shape)
print(np.round(scores, 2))
```

OUTPUT

```
X: (8, 2, 3) @ W: (3, 4) -> (8, 2, 4)
attention scores: (2, 4)
[[ 1.84  0.64 -0.09  1.41]
 [ 0.66  0.1   0.93 -0.37]]
```

Top-k predictions with argmax and argsort

After a forward pass you have a vector of scores per sample. `ar`

`gmax` gives the single predicted class; `argsort` (reversed) gives the ranked indices for a **top-k** shortlist, exactly what beam search and retrieval systems return.

ARGMAX AND TOP-K

```
import numpy as np

logits = np.array([0.1, 3.2, 0.4, 2.1, 1.0])
print("argmax (top-1):", int(np.argmax(logits)))
top2 = np.argsort(logits)[::-1][:2]      # indices of the 2 largest
    est
print("top-2 indices:", top2)
print("top-2 values :", logits[top2])
```

OUTPUT

```
argmax (top-1): 1
top-2 indices: [1 3]
top-2 values : [3.2 2.1]
```

A little linear algebra

`np.linalg` gives you norms (for gradient clipping and regularization), dot products (for cosine similarity), and solvers. Cosine similarity between embeddings is one of the most common operations in retrieval and RAG systems.

COSINE SIMILARITY BETWEEN TWO VECTORS

```
import numpy as np

u = np.array([1.0, 2.0, 2.0])
v = np.array([2.0, 0.0, 1.0])
cos = (u @ v) / (np.linalg.norm(u) * np.linalg.norm(v))
print("dot:", u @ v)
print("|u|:", round(float(np.linalg.norm(u)), 3))
print("cosine similarity:", round(float(cos), 3))
```

OUTPUT

```
dot: 4.0
|u|: 3.0
cosine similarity: 0.596
```

Why it matters for AI: forward passes, attention scores and similarity search are all matmuls plus broadcast adds. Fancy indexing *is* the embedding layer. Get comfortable reading shapes through an @ and you can trace data through any model.

CHAPTER 35

Random, Statistics and Activation Functions from Scratch

Randomness (for weight initialization and sampling) and statistics (for normalization) sit at the heart of training. In this chapter we build the classic activation and loss functions in pure NumPy so you know exactly what a framework computes under the hood.

Reproducible randomness

Always seed your generator. The modern API is `np.random.default_rng(seed)`, which gives an isolated stream, so your experiments are reproducible. Weight initialization typically draws from a scaled normal distribution.

SEEDED INIT AND SAMPLING

```
import numpy as np
rng = np.random.default_rng(0)

W = rng.normal(0, 0.02, size=(2, 3)) # GPT-style small init
print("weights:\n", np.round(W, 3))
print("uniform:", np.round(rng.uniform(size=3), 3))
print("random choice:", rng.choice([10, 20, 30], size=4))
```

OUTPUT

```
weights:
[[ 0.003 -0.003  0.013]
 [ 0.002 -0.011  0.007]]
uniform: [0.607 0.729 0.544]
random choice: [20 30 10 30]
```

Statistics and normalization

Normalizing inputs (zero mean, unit variance) makes training

stable. The z-score below is the same computation LayerNorm and BatchNorm perform, minus the learnable scale and shift.

Z-SCORE NORMALIZATION (A MINI LAYERNORM)

```
import numpy as np
rng = np.random.default_rng(1)

x = rng.normal(5, 3, size=(2, 4))      # not centered
mu = x.mean(axis=1, keepdims=True)
sd = x.std(axis=1, keepdims=True)
norm = (x - mu) / (sd + 1e-5)
print("mean before:", np.round(x.mean(axis=1), 2))
print("mean after :", np.round(norm.mean(axis=1), 6))
print("std  after :", np.round(norm.std(axis=1), 3))
```

OUTPUT

```
mean before: [5.15 6.05]
mean after  : [-0.  0.]
std  after  : [1. 1.]
```

Sigmoid and ReLU

Activations are just elementwise functions over an array. Vectorization means the whole tensor is transformed at once.

TWO ACTIVATION FUNCTIONS

```
import numpy as np

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def relu(z):
    return np.maximum(0.0, z)

z = np.array([-2.0, -0.5, 0.0, 0.5, 2.0])
print("sigmoid:", np.round(sigmoid(z), 3))
print("relu   :", relu(z))
```

OUTPUT

```
sigmoid: [0.119 0.378 0.5   0.622 0.881]
relu   : [0.  0.  0.  0.5 2. ]
```

Numerically stable softmax

Softmax turns logits into probabilities. The naive version overflows on large logits, so the standard trick is to subtract the max first, this changes nothing mathematically but keeps `exp` in range. Every framework does exactly this.

SOFTMAX WITH THE MAX-SUBTRACTION TRICK

```
import numpy as np

def softmax(logits, axis=-1):
    z = logits - logits.max(axis=axis, keepdims=True) # stability
    e = np.exp(z)
    return e / e.sum(axis=axis, keepdims=True)

logits = np.array([[2.0, 1.0, 0.1],
                  [0.0, 0.0, 5.0]])
probs = softmax(logits)
print("probs:\n", np.round(probs, 3))
print("rows sum to 1:", np.round(probs.sum(axis=1), 6))
```

OUTPUT

```
probs:
[[0.659 0.242 0.099]
 [0.007 0.007 0.987]]
rows sum to 1: [1. 1.]
```

Why the naive softmax fails

It is worth *seeing* the overflow. `float64` maxes out around `1e308`, and `exp(1000)` is astronomically larger, so it becomes `inf` and `inf/inf` is `nan`. Subtracting the max shifts the largest logit to 0, so `exp` stays in `[0, 1]` while the ratios are unchanged. The related **log-sum-exp** trick computes the log of the denominator stably, and `np.clip` bounds values into a safe range.

NAIVE VS STABLE SOFTMAX ON LARGE LOGITS

```

import numpy as np

logits = np.array([1000.0, 1001.0, 1002.0]) # large logits

def naive(z):
    e = np.exp(z) # exp(1002) overflows -> inf
    return e / e.sum()

def stable(z):
    z = z - z.max() # shift: mathematically identical
    e = np.exp(z)
    return e / e.sum()

with np.errstate(over="ignore", invalid="ignore"):
    print("naive :", naive(logits)) # nan from inf/inf
print("stable:", np.round(stable(logits), 3))

m = logits.max() # log-sum-exp, computed stable
y
lse = m + np.log(np.exp(logits - m).sum())
print("logsumexp:", round(float(lse), 3))
print("clip to [0,1]:", np.clip([-3.0, 0.5, 9.0], 0.0, 1.0))

```

OUTPUT

```

naive : [nan nan nan]
stable: [0.09  0.245 0.665]
logsumexp: 1002.408
clip to [0,1]: [0.  0.5 1. ]

```

Cross-entropy loss

Cross-entropy measures how far predicted probabilities are from the true labels. It combines fancy indexing (to pull the probability of the correct class) with a log and a mean, the loss that trains most classifiers and language models.

CROSS-ENTROPY FROM PROBABILITIES


```

import numpy as np

probs = np.array([[0.7, 0.2, 0.1],
                  [0.1, 0.1, 0.8]])
labels = np.array([0, 2])          # correct class per sample
le = 1
n = len(labels)
correct_p = probs[np.arange(n), labels]    # fancy indexing
loss = -np.mean(np.log(correct_p + 1e-9))
print("prob of correct class:", correct_p)
print("cross-entropy loss:", round(float(loss), 4))

```

OUTPUT

```

prob of correct class: [0.7 0.8]
cross-entropy loss: 0.2899

```

Why it matters for AI: softmax, ReLU and cross-entropy are the building blocks of nearly every model. Written in NumPy they are a few lines each, the framework versions add autograd and GPU support but compute the same thing, stability tricks included.

CHAPTER 36

pandas Series and DataFrame: Loading Datasets

Before data becomes tensors it usually lives in a table: CSVs, logs, feature stores. **pandas** is the standard tool for loading, inspecting and slicing that tabular data. Its two types are the 1-D [Series](#) and the 2-D [DataFrame](#) (a dict of aligned Series sharing an index).

Series: a labeled array

A SERIES IS VALUES PLUS AN INDEX

```
import pandas as pd

s = pd.Series([0.9, 0.4, 0.7], index=["cat", "dog", "fox"])
print(s)
print("by label:", s["dog"])
print("mean:", round(s.mean(), 3))
print("above 0.5:\n", s[s > 0.5])
```

OUTPUT

```
cat    0.9
dog    0.4
fox    0.7
dtype: float64
by label: 0.4
mean: 0.667
above 0.5:
  cat    0.9
  fox    0.7
dtype: float64
```

Building a DataFrame

You will usually read a DataFrame with `pd.read_csv(...)`, but constructing one from a dict is the clearest way to learn it. Think of each key as a column of a dataset.

A SMALL DATASET AS A DATAFRAME

```
import pandas as pd

df = pd.DataFrame({
    "text_len": [12, 45, 7, 33, 20],
    "label":    [0, 1, 0, 1, 0],
    "source":   ["web", "book", "web", "book", "web"],
})
print(df)
print("\ndtypes:\n", df.dtypes)
print("\nshape:", df.shape)
```

OUTPUT

```
   text_len  label source
0        12     0    web
1        45     1   book
2         7     0    web
3        33     1   book
4        20     0    web
```

```
dtypes:
text_len    int64
label       int64
source      str
dtype: object
```

```
shape: (5, 3)
```

Reading a CSV (from a string)

In practice you call `pd.read_csv("data.csv")`. To keep this example self-contained we hand `read_csv` an in-memory file via `io.StringIO`, the parsing is identical. Notice how pandas infers each column's dtype automatically from the raw text.

READ_CSV FROM AN IN-MEMORY STRING

```
import io
import pandas as pd

CSV = """id,text_len,label,source
1,12,0,web
2,45,1,book
3,7,0,web
4,33,1,book
5,20,0,web"""

df = pd.read_csv(io.StringIO(CSV))
print(df.head(3))
print("\ndtypes:\n", df.dtypes)
```

OUTPUT

```
   id  text_len  label source
0    1         12      0    web
1    2         45      1  book
2    3          7      0    web

dtypes:
id          int64
text_len    int64
label       int64
source      str
dtype: object
```

Inspecting: head and describe

The first thing you do with any dataset is look at it. `head` shows the top rows; `describe` gives summary statistics for the numeric columns, an instant sanity check on ranges and scale.

QUICK LOOK AT THE DATA

```
import pandas as pd

df = pd.DataFrame({
    "text_len": [12, 45, 7, 33, 20],
    "label":    [0, 1, 0, 1, 0],
})
print(df.head(2))
print("\nsummary:\n", df.describe().round(2))
```

OUTPUT

```
   text_len  label
0        12      0
1        45      1

summary:
      text_len  label
count      5.00  5.00
mean      23.40  0.40
std       15.57  0.55
min        7.00  0.00
25%       12.00  0.00
50%       20.00  0.00
75%       33.00  1.00
max       45.00  1.00
```

Selecting with loc and iloc

`loc` selects by label, `iloc` selects by integer position. Pick columns with a list, and rows with a slice or a mask. This is how you separate features from labels before training.

ROWS, COLUMNS, FEATURES VS LABELS

```
import pandas as pd

df = pd.DataFrame({
    "text_len": [12, 45, 7, 33, 20],
    "n_words":  [2, 8, 1, 6, 4],
    "label":    [0, 1, 0, 1, 0],
})
X = df[["text_len", "n_words"]]      # feature columns
y = df["label"]                     # target column
print("features:\n", X.iloc[:2])
print("\nfirst label by iloc:", y.iloc[0])
print("label via loc:", df.loc[3, "label"])
```

OUTPUT

```
features:
   text_len  n_words
0         12         2
1         45         8

first label by iloc: 0
label via loc: 1
```

Filtering rows by condition

Boolean masks work on DataFrames just like on NumPy arrays, and you can combine conditions with `&` and `|` (each condition in parentheses). This is your everyday tool for subsetting data.

BOOLEAN FILTERING

```
import pandas as pd

df = pd.DataFrame({
    "text_len": [12, 45, 7, 33, 20],
    "label":    [0, 1, 0, 1, 0],
})
long_pos = df[(df["text_len"] > 15) & (df["label"] == 1)]
print(long_pos)
print("\ncount per label:\n", df["label"].value_counts())
```

OUTPUT

```
   text_len  label
1         45      1
3         33      1

count per label:
label
0      3
1      2
Name: count, dtype: int64
```

Why it matters for AI: real projects spend most of their time in data wrangling, not modeling. [head/describe/loc/masks](#) are how you understand and slice a dataset before a single tensor is created, and `df[features]`, `df[target]` is the split every training script begins with.

CHAPTER 37

pandas Cleaning, Grouping and Feature Engineering

Raw data is messy: missing values, wrong scales, categorical strings a model cannot consume. This chapter covers the transforms that turn a table into model-ready features.

Missing values

`isna` finds gaps, `fillna` imputes them (often with the column mean), and `dropna` removes them. Leaving NaNs in will poison your gradients, so this is step one of cleaning.

DETECT AND FILL MISSING VALUES

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"age": [25, np.nan, 40, np.nan, 30]})
print("missing per column:\n", df.isna().sum())
df["age"] = df["age"].fillna(df["age"].mean())
print("\nafter fillna:\n", df.round(1))
```

OUTPUT

```
missing per column:
age      2
dtype: int64

after fillna:
age
0    25.0
1    31.7
2    40.0
3    31.7
4    30.0
```

New columns with vectorized ops and apply

Feature engineering is creating new columns from existing

ones. Prefer vectorized expressions; use `map/apply` for logic that does not vectorize cleanly. Both return a new Series you assign back to the frame.

DERIVED FEATURES

```
import pandas as pd

df = pd.DataFrame({"chars": [40, 10, 200, 75]})
df["words"] = (df["chars"] / 5).round().astype(int)  # vectorized
df["bucket"] = df["chars"].apply(
    lambda c: "long" if c > 100 else "short"  # apply
)
print(df)
```

OUTPUT

	chars	words	bucket
0	40	8	short
1	10	2	short
2	200	40	long
3	75	15	short

Grouping and aggregation

`groupby` splits rows into groups and aggregates each, the table equivalent of a pivot report. It answers questions like "what is the mean label per data source?", essential for spotting bias and class imbalance.

GROUPBY + AGG

```
import pandas as pd

df = pd.DataFrame({
    "source": ["web", "book", "web", "book", "web"],
    "len":     [12, 45, 7, 33, 20],
    "label":   [0, 1, 0, 1, 0],
})
summary = df.groupby("source").agg(
    n=("label", "size"),
    mean_len=("len", "mean"),
    pos_rate=("label", "mean"),
)
print(summary.round(2))
```

OUTPUT

	n	mean_len	pos_rate
source			
book	2	39.0	1.0
web	3	13.0	0.0

One-hot encoding categoricals

Models need numbers, not strings. `pd.get_dummies` expands a categorical column into 0/1 indicator columns, one per category, the standard encoding for non-ordinal features.

GET_DUMMIES FOR CATEGORICAL FEATURES

```
import pandas as pd

df = pd.DataFrame({"color": ["red", "blue", "red", "green"]})
onehot = pd.get_dummies(df, columns=["color"], dtype=int)
print(onehot)
```

OUTPUT

	color_blue	color_green	color_red
0	0	0	1
1	1	0	0
2	0	0	1
3	0	1	0

Merging two tables

Features often live in separate tables, an events log and a user profile, keyed by a shared id. `merge` is pandas' SQL-style join: `how="left"` keeps every row on the left and attaches matching columns from the right. `map` then turns a categorical string into a numeric code in one pass.

MERGE TWO FRAMES AND MAP A CATEGORY

```
import pandas as pd

events = pd.DataFrame({
    "user_id": [1, 2, 1, 3],
    "clicks": [4, 1, 2, 7],
})
users = pd.DataFrame({
    "user_id": [1, 2, 3],
    "plan": ["free", "pro", "free"],
})
joined = events.merge(users, on="user_id", how="left")
joined["is_pro"] = joined["plan"].map({"free": 0, "pro": 1})
print(joined)
print("\nclicks by plan:\n", joined.groupby("plan")["clicks"].sum())
```

OUTPUT

```
   user_id  clicks  plan  is_pro
0         1      4  free        0
1         2      1   pro        1
2         1      2  free        0
3         3      7  free        0

clicks by plan:
plan
free    13
pro      1
Name: clicks, dtype: int64
```

Scaling a numeric column

Features on wildly different scales make optimization hard. Standardizing each numeric column to zero mean and unit

variance, exactly as in the NumPy chapter, is a routine final step before handing data to a model.

STANDARDIZE A FEATURE COLUMN

```
import pandas as pd

df = pd.DataFrame({"income": [30000, 80000, 55000, 120000]})
col = df["income"]
df["income_z"] = (col - col.mean()) / col.std()
print(df.round(3))
```

OUTPUT

	income	income_z
0	30000	-1.075
1	80000	0.228
2	55000	-0.423
3	120000	1.270

Why it matters for AI: fillna, get_dummies and standardization are the classic tabular preprocessing pipeline, and scikit-learn transformers do exactly these steps. groupby is your fastest way to audit a dataset for imbalance and leakage before training.

CHAPTER 38

Visualization and the Deep-Learning Frameworks

This closing chapter connects the Python features from the whole book to the frameworks you will actually use. The punchline: PyTorch and friends invent very little new *language*, they lean on dunder methods, decorators, context managers and operator overloading you have already seen.

A framework **tensor** is a NumPy `ndarray` with three additions: it can live on a GPU (device placement, `x.to("cuda")` in PyTorch or `jax.device_put` in JAX), it can record operations for **autograd** (so a backward pass computes gradients automatically), and its ops are dispatched to fused kernels. Shapes, dtypes, broadcasting and `@` behave exactly as they do in NumPy, which is why everything you practised above transfers unchanged.

Plotting a loss curve (matplotlib, illustrative)

matplotlib is the default way to visualize training. The snippet below is illustrative (matplotlib is not installed in this build), but this is the exact idiom for saving a loss curve to a file during training.

ILLUSTRATIVE: SAVE A TRAINING LOSS CURVE

```
import matplotlib.pyplot as plt    # illustrative, not executed
    here

steps = list(range(0, 100, 10))
loss = [2.5, 1.8, 1.3, 1.0, 0.8, 0.7, 0.6, 0.55, 0.52, 0.5]

plt.figure(figsize=(5, 3))
plt.plot(steps, loss, marker="o")
plt.xlabel("step"); plt.ylabel("loss"); plt.title("Training loss")
plt.tight_layout()
plt.savefig("loss.png", dpi=150)    # write to disk, view later
```

The nn.Module pattern is just `__init__` + `__call__`

A PyTorch layer is a class: `__init__` registers parameters, `forward` defines the math, and calling the instance `layer(x)` works because `nn.Module` implements `__call__` (which then invokes `forward`). This is the exact dunder pattern from the objects chapter, illustrative below since torch is not installed.

ILLUSTRATIVE: A REAL PYTORCH MODULE

```
import torch                                # illustrative, not executed
    here
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, d_in, d_hidden, d_out):
        super().__init__()
        self.fc1 = nn.Linear(d_in, d_hidden)
        self.fc2 = nn.Linear(d_hidden, d_out)

    def forward(self, x):
        x = torch.relu(self.fc1(x))    # same relu as our NumPy
        one
        return self.fc2(x)

model = MLP(3, 8, 2)
y = model(torch.randn(4, 3))          # __call__ -> forward
print(y.shape)                        # torch.Size([4, 2])
```

Decorators and context managers in real training

Turning off gradient tracking during evaluation uses the same two features from earlier chapters: `@torch.no_grad()` as a **decorator** on a function, or `with torch.no_grad():` as a **context manager**. Same tool, two syntaxes.

ILLUSTRATIVE: NO_GRAD AS DECORATOR AND CONTEXT

```
import torch                                # illustrative, not executed ←
    here

@torch.no_grad()                            # decorator form
def evaluate(model, x):
    return model(x).argmax(dim=-1)

# ... inside a training loop ...
for step in range(100):
    with torch.no_grad():                   # context-manager form
        val_preds = model(val_x)          # no autograd graph built ←
        t
    # train step here would use gradients as normal
```

A verified gradient-descent step in NumPy

To keep this chapter grounded, here is a real (executed) training step for linear regression in pure NumPy: forward pass, mean-squared-error loss, analytic gradient, and a weight update. Watch the loss fall, this is what autograd automates.

ONE EPOCH LOOP OF GRADIENT DESCENT

```

import numpy as np
rng = np.random.default_rng(0)

X = rng.normal(size=(50, 3))
true_w = np.array([2.0, -1.0, 0.5])
y = X @ true_w + 0.1 * rng.normal(size=50)  # noisy targets

w = np.zeros(3)
lr = 0.1
for step in range(0, 101):
    pred = X @ w  # forward
    err = pred - y
    loss = np.mean(err ** 2)  # MSE
    grad = 2.0 / len(y) * (X.T @ err)  # gradient
    w -= lr * grad  # update
    if step % 25 == 0:
        print(f"step {step:3d}  loss={loss:.4f}")
print("learned w:", np.round(w, 2))

```

OUTPUT

```

step   0  loss=6.0123
step  25  loss=0.0094
step  50  loss=0.0087
step  75  loss=0.0087
step 100  loss=0.0087
learned w: [ 2.02 -1.02  0.5 ]

```

A whole tiny MLP forward pass in NumPy

Here is the payoff of the entire part: a complete 2-layer classifier, `linear -> relu -> linear -> softmax`, using only the pieces built earlier, `matmul`, `broadcasting`, `ReLU` and the `stable softmax`. The illustrative PyTorch `MLP` above compiles to this exact arithmetic; a framework merely adds `autograd` and a `GPU`.

EXECUTED: 2-LAYER MLP FORWARD PASS


```

import numpy as np
rng = np.random.default_rng(0)

def relu(z):
    return np.maximum(0.0, z)

def softmax(z, axis=-1):
    z = z - z.max(axis=axis, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=axis, keepdims=True)

X = rng.normal(size=(4, 3))          # batch=4, in=3
W1 = rng.normal(size=(3, 5)) * 0.5   # layer 1: 3 -> 5
b1 = np.zeros(5)
W2 = rng.normal(size=(5, 2)) * 0.5   # layer 2: 5 -> 2
b2 = np.zeros(2)

h = relu(X @ W1 + b1)                # linear -> relu
logits = h @ W2 + b2                 # linear
probs = softmax(logits)              # -> class probabilities
print("logits shape:", logits.shape)
print("probs:\n", np.round(probs, 3))
print("row sums:", np.round(probs.sum(axis=1), 6))
print("predicted class:", probs.argmax(axis=1))

```

OUTPUT

```

logits shape: (4, 2)
probs:
[[0.51  0.49 ]
 [0.501 0.499]
 [0.569 0.431]
 [0.43  0.57 ]]
row sums: [1.  1.  1.  1.]
predicted class: [0 0 0 1]

```

The whole book, in one sentence

- **Containers & comprehensions** build datasets and batches.
- **Generators** stream data too big for memory.
- **Dunder methods** (`__call__`) make models callable like functions.

- **Decorators & context managers** toggle behavior like `no_grad`.
- **Dataclasses & type hints** describe configs and tensors.
- **NumPy** vectorization is the math every framework accelerates.

Why it matters for AI: once you see that `model(x)` is `__call__`, that `@torch.no_grad()` is a decorator, and that a tensor is an ndarray with autograd, framework code stops looking like magic and starts looking like the Python you already know.

PART IX

Capstone: A Tiny Transformer LLM

CHAPTER 39

Building and Training a Tiny Transformer LLM

This capstone pulls the whole book together: classes and `__call__` (Part IV), tensors and matrix math (Part VIII) and a training loop all combine into a small **GPT-style transformer** you can train on a laptop CPU in about ten seconds. It is a *character-level language model*: it reads text one character at a time and learns to predict the next character. Train it on a passage and it learns that passage's style well enough to continue it.

These two capstone chapters are the only ones that need PyTorch. Install it first with `pip install torch` (the CPU build is enough). Everything here runs on CPU; no GPU is required.

Step 1 - a character tokenizer

A model works on numbers, not text, so first we map each character to an integer id and back. Real LLMs use subword tokenizers (BPE), but characters keep the vocabulary tiny and the idea identical: `encode` turns text into ids, `decode` turns ids back into text.

A CHARACTER-LEVEL TOKENIZER

```
# A character tokenizer is the simplest possible tokenizer: the
# vocabulary
# is just the set of distinct characters, and each maps to an i
# nteger id.
text = "WildFly"
chars = sorted(set("Downloading WildFly and running it."))
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for i, c in enumerate(chars)}
ids = [stoi[c] for c in text]
print("vocab size :", len(chars))
print("encode      :", ids)
print("decode back:", "".join(itos[i] for i in ids))
```

OUTPUT

```
vocab size : 17
encode      : [4, 8, 9, 6, 3, 9, 16]
decode back: WildFly
```

Step 2 - the causal mask

A language model must predict the next token from **only the tokens before it**, never from the future. Self-attention enforces this with a *causal mask* that blocks each position from attending to later positions, the upper triangle of the attention matrix.

THE LOOK-AHEAD MASK (1 = BLOCKED)

```
import torch
# The causal mask stops each position from attending to future
# tokens.
# True means "blocked". Row t can only see columns 0..t (the lo
# wer triangle).
T = 5
mask = torch.triu(torch.ones(T, T, dtype=torch.bool), diagonal=
1)
print(mask.int())
```

OUTPUT

```
tensor([[0, 1, 1, 1, 1],
        [0, 0, 1, 1, 1],
        [0, 0, 0, 1, 1],
        [0, 0, 0, 0, 1],
        [0, 0, 0, 0, 0]], dtype=torch.int32)
```

Step 3 - the model, in one screen

The transformer is small enough to read top to bottom. Each input id becomes a learned **token embedding**; a **position embedding** tells the model where each character sits. Two **blo**cks follow, each doing masked multi-head self-attention (letting characters exchange information) then a small MLP, both wrapped in residual connections with **LayerNorm**. A final linear **h**ead turns the result into one score (logit) per vocabulary character.

- `nn.Embedding` - lookup tables for tokens and positions.
- `nn.MultiheadAttention` - the attention mechanism, with our causal `attn_mask`.
- residual `x = x + sublayer(x)` - keeps gradients flowing through a deep stack.
- `F.cross_entropy` - the next-character prediction loss.

Step 4 - train it

Training draws random chunks of the corpus, asks the model to predict each chunk shifted by one character, and nudges the weights with `AdamW`. Watch the loss fall from ~ 3.8 (random guessing over the vocabulary) to well under 0.1 as the model memorises the passage. The full program below is what actually ran to produce the output panel.

TRAIN A CHARACTER-LEVEL GPT ON THE WILDFLY GUIDE

```

import os, tempfile, time
import torch
import torch.nn as nn
from torch.nn import functional as F

torch.manual_seed(1337)

# 1. A tiny corpus taken from the WildFly Installation Guide.
CORPUS = (
    "Downloading the WildFly release zip and unzipping it is th
        e "
    "traditional way to install a complete WildFly server with
        support "
    "for both standalone and managed domain operating modes. "
    "Galleon provisioning tooling allows you to construct a cus
        tomized "
    "WildFly installation according to your application needs.
        "
    "This method suits those concerned about server size and me
        mory "
    "footprint who only need specific Jakarta or MicroProfile A
        PIs. "
    "The WildFly Maven Plugin enables developers building appli
        cations "
    "with Maven to provision customized installations directly
        within "
    "their build process. A bootable JAR contains both a custom
        ized "
    "WildFly server and your deployment, and can be run with a
        simple "
    "Java command, making it ideal for microservices architectu
        res."
)

# 2. Character-level tokenizer: every character is a token.
chars = sorted(set(CORPUS))
vocab_size = len(chars)
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for i, c in enumerate(chars)}
encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: "".join(itos[i] for i in ids)
data = torch.tensor(encode(CORPUS), dtype=torch.long)
print(f"corpus chars: {len(CORPUS)} | vocab: {vocab_size}")

# 3. Tiny, laptop-friendly hyperparameters.
block_size = 64          # context length (chars the model can

```

```

    look back on)
batch_size = 32
n_embd = 96          # embedding width
n_head = 3           # attention heads
n_layer = 2          # transformer blocks
steps = 600
lr = 3e-3

def get_batch():
    ix = torch.randint(0, len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i + block_size] for i in ix])
    y = torch.stack([data[i + 1:i + block_size + 1] for i in ix])
    return x, y          # y is x shifted by one (next char)

# 4. One transformer block: causal self-attention + a small MLP
#     each wrapped in a residual connection with LayerNorm.
class Block(nn.Module):
    def __init__(self):
        super().__init__()
        self.ln1 = nn.LayerNorm(n_embd)
        self.attn = nn.MultiheadAttention(n_embd, n_head, batch_first=True)
        self.ln2 = nn.LayerNorm(n_embd)
        self.mlp = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd), nn.GELU(),
            nn.Linear(4 * n_embd, n_embd),
        )

    def forward(self, x):
        T = x.size(1)
        # causal mask: position t may not attend to positions > t
        mask = torch.triu(torch.ones(T, T, dtype=torch.bool), diagonal=1)
        h = self.ln1(x)
        a, _ = self.attn(h, h, h, attn_mask=mask, need_weights=False)
        x = x + a          # residual
        x = x + self.mlp(self.ln2(x)) # residual
        return x

# 5. The full model: token + position embeddings, blocks, and a head

```



```

# that projects back to vocab-sized logits.
class TinyGPT(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok = nn.Embedding(vocab_size, n_embd)
        self.pos = nn.Embedding(block_size, n_embd)
        self.blocks = nn.ModuleList([Block() for _ in range(n_l ←
            ayer)])
        self.lnf = nn.LayerNorm(n_embd)
        self.head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx, targets=None):
        T = idx.size(1)
        x = self.tok(idx) + self.pos(torch.arange(T))
        for b in self.blocks:
            x = b(x)
        logits = self.head(self.lnf(x))
        loss = None
        if targets is not None:
            loss = F.cross_entropy(
                logits.view(-1, vocab_size), targets.view(-1))
        return logits, loss

model = TinyGPT()
print(f"parameters: {sum(p.numel() for p in model.parameters()) ←
    :,,}")

# 6. Training loop: predict the next character, minimise cross- ←
entropy.
opt = torch.optim.AdamW(model.parameters(), lr=lr)
t0 = time.time()
for step in range(steps):
    x, y = get_batch()
    _, loss = model(x, y)
    opt.zero_grad()
    loss.backward()
    opt.step()
    if step % 150 == 0 or step == steps - 1:
        print(f"step {step:4d} | loss {loss.item():.3f}")
print(f"trained in {time.time() - t0:.1f}s")

# 7. Save a checkpoint the inference chapter will load.
ckpt = os.path.join(tempfile.gettempdir(), "tiny_wildfly_llm.pt ←
    ")
torch.save({"model": model.state_dict(), "chars": chars,
    "config": dict(block_size=block_size, n_embd=n_embd ←
    ,

```

```
        n_head=n_head, n_layer=n_layer)}, ckpt)
print("saved checkpoint:", os.path.basename(ckpt))

# 8. Greedy sanity check: does it reproduce the guide's style?
model.eval()
idx = torch.tensor([encode("WildFly")], dtype=torch.long)
for _ in range(80):
    logits, _ = model(idx[:, -block_size:])
    nxt = torch.argmax(logits[:, -1, :], dim=-1, keepdim=True)
    idx = torch.cat([idx, nxt], dim=1)
print("sample:", repr(decode(idx[0].tolist())))
```

OUTPUT

```
corpus chars: 753 | vocab: 37
parameters: 237,157
step    0 | loss 3.835
step  150 | loss 0.265
step  300 | loss 0.087
step  450 | loss 0.076
step  599 | loss 0.076
trained in 8.5s
saved checkpoint: tiny_wildfly_llm.pt
sample: 'WildFly release zip and unzipping it is the traditiona
        l way to install a complete WildF'
```

The final line shows greedy continuation from the prompt **WildFly**: after a few seconds of training the model reproduces the guide's wording almost verbatim, which is exactly what we expect when a model with enough capacity overfits a very small corpus.

Why it matters for AI: every large language model, from a few-million-parameter demo to a frontier model, is this same loop at scale, predict the next token, backpropagate the cross-entropy loss, repeat, on far more data, layers and compute. Understanding this 100-line version demystifies the rest.

CHAPTER 40

Running Inference with the Trained Transformer

Training produced a checkpoint; **inference** is where we put it to work. Text generation is *autoregressive*: feed the prompt, take the model's prediction for the next character, append it, and feed the longer sequence back in, one character at a time. This chapter loads the saved model and generates text three different ways.

Loading a saved model

A checkpoint stores the learned weights (`state_dict`), not the code. To use it you re-create the same architecture and load the weights into it, then switch to `eval()` mode. We also saved the vocabulary and config so the tokenizer and model shapes match exactly.

Greedy, temperature and top-k sampling

The model outputs a score for every possible next character. How you pick from those scores controls the output:

- **Greedy** - always take the highest-scoring character. Deterministic, but repetitive.
- **Temperature** - divide the logits by a temperature before sampling. Below 1.0 makes the model more confident and focused; above 1.0 makes it more random and creative.
- **Top-k** - sample only from the k most likely characters, cutting off the unlikely long tail that produces gibberish.

These are the very same knobs (`temperature`, `top_k`, `top_p`) you set on a real LLM API. The program below loads the checkpoint and shows all three strategies.

GENERATE TEXT FROM THE TRAINED MODEL

```

import os, tempfile
import torch
import torch.nn as nn
from torch.nn import functional as F

torch.manual_seed(0)

# 1. Load the checkpoint saved by the training chapter.
ckpt_path = os.path.join(tempfile.gettempdir(), "tiny_wildfly_lm.pt")
if not os.path.exists(ckpt_path):
    print("no checkpoint found - run the training chapter first")
    raise SystemExit(0)

# weights_only=False uses pickle; safe here because WE wrote the file.
# (See Part VI: never unpickle a checkpoint from an untrusted source.)
ckpt = torch.load(ckpt_path, weights_only=False)
chars = ckpt["chars"]
cfg = ckpt["config"]
vocab_size = len(chars)
block_size = cfg["block_size"]
n_embd, n_head, n_layer = cfg["n_embd"], cfg["n_head"], cfg["n_layer"]

stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for i, c in enumerate(chars)}
encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: "".join(itos[i] for i in ids)

# 2. Re-declare the same architecture so the weights can be loaded into it.
class Block(nn.Module):
    def __init__(self):
        super().__init__()
        self.ln1 = nn.LayerNorm(n_embd)
        self.attn = nn.MultiheadAttention(n_embd, n_head, batch_first=True)
        self.ln2 = nn.LayerNorm(n_embd)
        self.mlp = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd), nn.GELU(),
            nn.Linear(4 * n_embd, n_embd),
        )

    def forward(self, x):

```

```

    T = x.size(1)
    mask = torch.triu(torch.ones(T, T, dtype=torch.bool), diagonal=1)
    h = self.ln1(x)
    a, _ = self.attn(h, h, h, attn_mask=mask, need_weights=False)
    x = x + a
    return x + self.mlp(self.ln2(x))

class TinyGPT(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok = nn.Embedding(vocab_size, n_embd)
        self.pos = nn.Embedding(block_size, n_embd)
        self.blocks = nn.ModuleList([Block() for _ in range(n_layers)])
        self.lnf = nn.LayerNorm(n_embd)
        self.head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx):
        T = idx.size(1)
        x = self.tok(idx) + self.pos(torch.arange(T))
        for b in self.blocks:
            x = b(x)
        return self.head(self.lnf(x))

model = TinyGPT()
model.load_state_dict(ckpt["model"])
model.eval()
print("loaded", sum(p.numel() for p in model.parameters()), "parameters")

# 3. Autoregressive generation: predict one char, append it, repeat.
@torch.no_grad()
def generate(prompt, n=90, temperature=1.0, top_k=None, greedy=False):
    idx = torch.tensor([encode(prompt)], dtype=torch.long)
    for _ in range(n):
        logits = model(idx[:, -block_size:])[ :, -1, :] # last position
        if greedy:
            nxt = torch.argmax(logits, dim=-1, keepdim=True)
        else:
            logits = logits / temperature # sharpen/flat ten
            if top_k is not None: # keep only to k

```

```
    p-k choices
    v, _ = torch.topk(logits, top_k)
    logits[logits < v[:, [-1]]] = -float("inf")
    probs = F.softmax(logits, dim=-1)
    nxt = torch.multinomial(probs, num_samples=1)
    idx = torch.cat([idx, nxt], dim=1)
    return decode(idx[0].tolist())

print("\n-- greedy (deterministic) --")
print(generate("Galleon", greedy=True))
print("\n-- temperature=0.8 sampling --")
print(generate("Galleon", temperature=0.8))
print("\n-- temperature=1.0, top_k=5 --")
print(generate("The WildFly", temperature=1.0, top_k=5))
```

OUTPUT

```
loaded 237157 parameters

-- greedy (deterministic) --
Galleon provisioning tooling allows you to construct a customiz ←
ed WildFly installation according

-- temperature=0.8 sampling --
Galleon provisioning tooling allows you to construct a customiz ←
ed installations directly within t

-- temperature=1.0, top_k=5 --
The WildFly Maven Plugin enables developers building applicatio ←
ns with Maven to provision customized
```

Greedy decoding reproduces the guide's text deterministically; the sampled variants stay on-topic but vary their wording. On a corpus this small the model mostly recites what it memorised, exactly how a large model, trained on far more text, instead learns general language patterns it can recombine into novel sentences.

Why it matters for AI: serving a model is this load-then-generate loop, and the sampling strategy is what separates a flat, repetitive assistant from a fluent one. The decoding knobs you just implemented by hand are precisely the [temperature](#) and [top_k/top_p](#) parameters exposed by every production LLM API.